

PENGEMBANGAN SISTEM INFORMASI ORGANISASI INFINITE DENGAN LARAVEL *HEADLESS*, NEXT.JS, DAN KONTAINER

TUGAS AKHIR SKRIPSI



Ditulis untuk memenuhi sebagian persyaratan guna mendapatkan gelar
Sarjana Teknik
Program Studi Teknologi Informasi

Oleh:
MUHAMMAD NAUFAL KHOIRUL IMAMILHAQ ALHIFDI
NIM 22051130001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026**

PENGEMBANGAN SISTEM INFORMASI ORGANISASI INFINITE DENGAN LARAVEL *HEADLESS*, NEXT.JS, DAN KONTAINER

TUGAS AKHIR SKRIPSI



Ditulis untuk memenuhi sebagian persyaratan guna mendapatkan gelar
Sarjana Teknik
Program Studi Teknologi Informasi

Oleh:
MUHAMMAD NAUFAL KHOIRUL IMAMILHAQ ALHIFDI
NIM 22051130001

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026**

Pengembangan Sistem Informasi Organisasi INFINITE dengan Laravel *Headless*, Next.js, dan Kontainer

Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM 22051130001

ABSTRAK

INFINITE adalah divisi di bawah UKM Rekayasa Teknologi (Restek) UNY yang berfungsi sebagai wadah pengembangan diri mahasiswa di bidang teknologi informasi. Sistem informasi organisasi INFINITE sebelumnya terfragmentasi dalam dua sistem, yaitu Laravel dengan arsitektur monolitik di *shared hosting* dan CMS Strapi yang terpisah, sehingga menimbulkan tantangan pemeliharaan, konsistensi data, inefisiensi biaya operasional, serta ketidakmampuan integrasi dengan aplikasi eksternal dan infrastruktur Kubernetes. Penelitian ini bertujuan mengembangkan sistem informasi organisasi INFINITE yang terkontainerisasi dengan arsitektur *headless* menggunakan Laravel sebagai *backend*, Next.js sebagai *frontend*, dan autentikasi berstandar protokol *OpenID Connect* (OIDC), serta di-*deploy* di Kubernetes dengan pendekatan GitOps menggunakan ArgoCD.

Pengembangan sistem dilakukan menggunakan metode *Research and Development* (R&D) dengan model *Waterfall* melalui empat tahapan: *Requirements Definition*, *System and Software Design*, *Implementation and Unit Testing*, serta *Integration and System Testing*. Pengujian dilakukan berdasarkan standar ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use*, yang mencakup aspek *Functional Suitability*, *Interaction Capability*, *Performance Efficiency*, dan *Beneficialness*.

Hasil penelitian menunjukkan bahwa sistem berhasil dikembangkan dan di-*deploy* sebagai aplikasi terkontainerisasi di Kubernetes, dengan hasil pengujian *Functional Suitability* sebesar 100% (Sangat Layak), *Interaction Capability* 91,73% (Sangat Layak), *Performance Efficiency* pada *frontend* berupa rata-rata skor Lighthouse 90,42 (Baik) dan pada *backend* berupa *Response Time* 1 detik, *Throughput* 53,6 permintaan per detik, *Error Rate* 0%, serta maksimum 189 *Virtual Users* simultan, dan *Beneficialness* 81,30% (Sangat Layak). Berdasarkan hasil tersebut, sistem informasi organisasi INFINITE tergolong layak digunakan sebagai solusi pengelolaan data organisasi secara terintegrasi dan berkelanjutan.

Kata kunci: Sistem Informasi, *Headless*, Kontainer.

Development of INFINITE Organizational Information System with Headless Laravel, Next.js, and Containers

Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM 22051130001

ABSTRACT

INFINITE is a division under the Rekayasa Teknologi (Restek) Student Activity Unit at Universitas Negeri Yogyakarta (UNY) that serves as a platform for student self-development in the field of information technology. The previous INFINITE organizational information system was fragmented into two systems, namely Laravel with a monolithic architecture on shared hosting and a separate Strapi CMS, creating challenges in maintenance, data consistency, operational cost inefficiency, and the inability to integrate with external applications and the Kubernetes infrastructure. This study aims to develop a containerized INFINITE organizational information system with a headless architecture using Laravel as the backend, Next.js as the frontend, and authentication based on the OpenID Connect (OIDC) protocol, deployed on Kubernetes using the GitOps approach with ArgoCD.

System development was carried out using the Research and Development (R&D) method with the Waterfall model through four stages: Requirements Definition, System and Software Design, Implementation and Unit Testing, and Integration and System Testing. Testing was conducted based on the ISO/IEC 25010:2023 standard for Product Quality and ISO/IEC 25019:2023 for Quality in Use, covering the aspects of Functional Suitability, Interaction Capability, Performance Efficiency, and Beneficialness.

The results show that the system was successfully developed and deployed as a containerized application on Kubernetes, with testing results of Functional Suitability at 100% (Very Feasible), Interaction Capability at 91.73% (Very Feasible), Performance Efficiency on the frontend with an average Lighthouse score of 90.42 (Good) and on the backend with a Response Time of 1 second, Throughput of 53.6 requests per second, Error Rate of 0%, and a maximum of 189 simultaneous Virtual Users, and Beneficialness at 81.30% (Very Feasible). Based on these results, the INFINITE organizational information system is considered feasible as a solution for integrated and sustainable organizational data management.

Key words: *Information System, Headless, Container.*

SURAT PERNYATAAN

Saya yang bertandatangan di bawah ini:

Nama : Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM : 22051130001
Program Studi : Teknologi Informasi
Judul Tugas Akhir : Pengembangan Sistem Informasi Organisasi
INFINITE dengan Laravel *Headless*, Next.js, dan
Kontainer

menyatakan bahwa tugas akhir ini benar-benar karya saya sendiri. Sepanjang pengetahuan saya tidak terdapat karya atau pendapat yang ditulis atau diterbitkan orang lain kecuali sebagai acuan kutipan dengan mengikuti tata penulisan karya ilmiah yang telah lazim.

Yogyakarta, 17 Juli 2026

Yang menyatakan,

Muhammad Naufal Khoirul Imamilhaq Alhifdi

NIM. 22051130001

LEMBAR PERSETUJUAN

PENGEMBANGAN SISTEM INFORMASI ORGANISASI INFINITE DENGAN LARAVEL *HEADLESS*, NEXT.JS, DAN KONTAINER

TUGAS AKHIR SKRIPSI

MUHAMMAD NAUFAL KHOIRUL IMAMILHAQ ALHIFDI
NIM 22051130001

Telah disetujui untuk dipertahankan di depan Tim Penguji Tugas Akhir Fakultas
Teknik Universitas Negeri Yogyakarta.
Tanggal: 17 Juli 2026

Koordinator Program Studi,

Dosen Pembimbing,

Nurkhamid, S.Si., M.Kom., Ph.D.
NIP. 19680707 199702 1 001

Muslikhin, S.Pd., M.Pd., Ph.D.
NIP. 19850101 201404 1 001

LEMBAR PENGESAHAN

PENGEMBANGAN SISTEM INFORMASI ORGANISASI INFINITE DENGAN LARAVEL *HEADLESS*, NEXT.JS, DAN KONTAINER

TUGAS AKHIR SKRIPSI

MUHAMMAD NAUFAL KHOIRUL IMAMILHAQ ALHIFDI
NIM 22051130001

Telah dipertahankan di depan Tim Penguji Tugas Akhir
Fakultas Teknik Universitas Negeri Yogyakarta
Tanggal 27 Juli 2026.

TIM PENGUJI

Nama/Jabatan	Tanda Tangan	Tanggal
Muslikhin, S.Pd., M.Pd., Ph.D. (Ketua Penguji/Pembimbing)		
Ir. Bkti Wulandari, S.Pd.T., M.Pd. (Sekretaris)		
Dr. Agus Qomaruddin Munir, S.T., M.Cs. (Penguji Utama)		

Yogyakarta, 27 Juli 2026
Fakultas Teknik, Universitas Negeri Yogyakarta
Dekan,

Prof. Dr. Mutiara Nugraheni, S.TP., M.Si.
NIP. 19770131 200212 2 001

MOTTO

Machines never make mistakes. They operate according to the laws of the universe, laws created by God Himself. It is humans who fail to account for every possible outcome of these laws.

PERSEMBAHAN

Tugas Akhir Skripsi ini dipersembahkan untuk:

Ibu tersayang,
yang doanya merengkuh di tiap arunika,
yang asihnya menyingkap pendar cakrawala.

Ayah tercinta,
yang didikannya menyemai ketajaman nalar,
yang lisannya sarat kedalaman makna.

Kakak-kakak tersayang,
yang selalu menjadi teladan dan pengingat.

Rekan,
yang senantiasa menanamkan ekuitas asa,
yang hadir sebagai jurnal penyesuaian dalam setiap siklus rencana.

Dosen, teman-teman, dan INFINITE,
yang menjadi bagian dari setiap orkestrasi pendewasaan diri,
yang turut melengkapi setiap dependensi dalam perjalanan akademik,
yang pada akhirnya *pipeline* ini sukses mencapai *deployment*.

KATA PENGANTAR

Puji syukur kehadiran Allah Swt. atas segala rahmat dan karunia-Nya, sehingga Tugas Akhir Skripsi ini dapat diselesaikan. Tugas Akhir ini dapat diselesaikan tidak lepas dari peran dan dukungan berbagai pihak. Berkenaan dengan hal tersebut, penulis menyampaikan ucapan terima kasih kepada:

1. Prof. Dr. Mutiara Nugraheni, S.TP., M.Si. selaku Dekan Fakultas Teknik Universitas Negeri Yogyakarta yang telah memberikan dukungan dan fasilitas dalam proses penyelesaian studi.
2. Nurkhamid, S.Si., M.Kom., Ph.D. selaku Ketua Program Studi Teknologi Informasi sekaligus Dosen Pembimbing Akademik yang telah memberikan bimbingan, arahan, dan motivasi selama proses perkuliahan.
3. Muslikhin, S.Pd., M.Pd., Ph.D. selaku dosen pembimbing yang telah banyak memberikan semangat, dorongan, bimbingan, dan masukan selama penyusunan Tugas Akhir Skripsi ini.
4. Muhammad Resa Arif Yudianto, M.Kom. selaku validator instrumen penelitian yang telah memberikan bantuan, masukan, dan saran yang sangat berharga bagi penulis.
5. Ir. Ardy Seto Priambodo, S.T., M.Eng., yang telah menyediakan format Tugas Akhir LaTeX yang memudahkan penulis dalam menyusun Tugas Akhir Skripsi ini.
6. Seluruh dosen dan staf pengajar di Fakultas Teknik Universitas Negeri Yogyakarta yang telah memberikan ilmu pengetahuan, wawasan, dan pengalaman berharga selama masa perkuliahan.
7. Teman-teman anggota dan pengurus INFINITE UKM Rekayasa Teknologi yang telah memberikan semangat dan dukungan selama proses pengembangan dan penyusunan proyek Tugas Akhir Skripsi ini.
8. Semua pihak, secara langsung maupun tidak langsung, yang tidak dapat disebutkan di sini atas bantuan dan perhatiannya selama penyusunan Tugas Akhir Skripsi ini.

Penulis menyadari bahwa Tugas Akhir Skripsi ini masih memiliki banyak potensi untuk dikembangkan dan disempurnakan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun dari berbagai pihak guna kelanjutan pengembangan hasil penelitian di masa yang akan datang. Penulis juga berharap Tugas Akhir Skripsi ini dapat memberikan manfaat dan dorongan bagi pengembangan ilmu pengetahuan, khususnya di bidang Teknologi Informasi modern.

Yogyakarta, 27 Juli 2026

Muhammad Naufal Khoirul Imamilhaq Alhifdi

22051130001

DAFTAR ISI

HALAMAN SAMPUL	i
ABSTRAK	ii
ABSTRACT	iii
SURAT PERNYATAAN	iv
LEMBAR PERSETUJUAN	v
LEMBAR PENGESAHAN	vi
MOTTO	vii
PERSEMBAHAN	viii
KATA PENGANTAR	ix
DAFTAR ISI	xi
DAFTAR SINGKATAN	xiii
DAFTAR GAMBAR	xiv
DAFTAR TABEL	xv
DAFTAR LAMPIRAN	xvi
BAB 1 PENDAHULUAN	1
A. Latar Belakang Masalah	1
B. Identifikasi Masalah	5
C. Batasan Masalah	5
D. Rumusan Masalah	6
E. Tujuan Penelitian	7
F. Manfaat Penelitian	7
1. Manfaat Teoritis	7
2. Manfaat Praktis	8
BAB 2 KAJIAN PUSTAKA	9
A. Kajian Teori	9
1. INFINITE	9
2. Sistem Informasi	10
3. <i>Backend</i> dengan Laravel	11
4. <i>Frontend</i> dengan Next.js	15
5. <i>Deployment</i> dengan Kubernetes dan GitOps	18
6. Autentikasi dan <i>Single Sign-On</i> (SSO)	21
7. Rekayasa Perangkat Lunak	27
B. Hasil Penelitian yang Relevan	38
C. Kerangka Pikir	40
D. Pertanyaan Penelitian	42
BAB 3 METODE PENELITIAN	44
A. Jenis Penelitian	44
B. Metode Penelitian	46
1. <i>Requirements Definition</i>	46

2. <i>System and Software Design</i>	46
3. <i>Implementation and Unit Testing</i>	47
4. <i>Integration and System Testing</i>	48
C. Tempat dan Waktu Penelitian	48
D. Subjek Penelitian	49
E. Teknik Pengumpulan Data	49
1. Wawancara	50
2. Observasi	50
3. Kuesioner	50
F. Instrumen Penelitian	51
1. Instrumen Pengujian <i>Functional Suitability</i>	51
2. Instrumen Pengujian <i>Interaction Capability</i>	51
3. Instrumen Pengujian <i>Performance Efficiency</i>	52
4. Instrumen Pengujian <i>Beneficialness</i>	54
G. Teknik Analisis Data	55
1. Analisis Pengujian <i>Functional Suitability</i>	55
2. Analisis Pengujian <i>Interaction Capability</i> dan <i>Beneficialness</i>	56
3. Analisis Pengujian <i>Performance Efficiency</i>	57
BAB 4 HASIL PENELITIAN DAN PEMBAHASAN	59
A. Hasil Penelitian	59
1. Pengembangan Sistem Informasi	59
2. Cara Kerja Sistem Informasi	84
3. Kualitas Sistem Informasi	90
B. Pembahasan	94
BAB 5 KESIMPULAN DAN SARAN	99
A. Kesimpulan	99
B. Keterbatasan Produk	100
C. Saran	101
DAFTAR PUSTAKA	102
LAMPIRAN	109

DAFTAR SINGKATAN

API	: <i>Application Programming Interface</i>
CI/CD	: <i>Continuous Integration/Continuous Deployment</i>
CNCF	: <i>Cloud Native Computing Foundation</i>
DTO	: <i>Data Transfer Object</i>
EUCS	: <i>End-User Computing Satisfaction</i>
IaaS	: <i>Infrastructure as a Service</i>
JSON	: <i>JavaScript Object Notation</i>
JWT	: <i>JSON Web Token</i>
MVC	: <i>Model-View-Controller</i>
OIDC	: <i>OpenID Connect</i>
ORM	: <i>Object-Relational Mapping</i>
PaaS	: <i>Platform as a Service</i>
PWA	: <i>Progressive Web App</i>
REST	: <i>Representational State Transfer</i>
SaaS	: <i>Software as a Service</i>
SAML	: <i>Security Assertion Markup Language</i>
SEO	: <i>Search Engine Optimization</i>
SPA	: <i>Single-Page Application</i>
SSO	: <i>Single Sign-On</i>
SSR	: <i>Server-Side Rendering</i>
USE	: <i>Usefulness, Satisfaction, and Ease of Use</i>
XML	: <i>Extensible Markup Language</i>

DAFTAR GAMBAR

2.1	Diagram Kerangka Pikir Penelitian	41
3.1	Diagram Model <i>Waterfall</i>	45
4.1	<i>Use Case Diagram</i> Aktor Publik	65
4.2	<i>Class Diagram</i>	66
4.3	<i>Sequence Diagram</i> Alur Autentikasi OIDC	68
4.4	<i>Sequence Diagram</i> Alur Sinkronisasi Data Pengguna dan Hak Akses ke SSO	69
4.5	<i>Sequence Diagram</i> Alur Pengelolaan Data Sistem (CRUD)	70
4.6	Desain Basis Data	71
4.7	Desain Halaman List Data	72
4.8	Desain Halaman Detail	72
4.9	Desain Halaman Formulir Sunting	73
4.10	Tampilan Halaman List Data	81
4.11	Tampilan Halaman Detail	81
4.12	Tampilan Halaman Formulir Sunting	82
4.13	CI/CD dengan GitHub Actions dan ArgoCD	83
4.14	<i>Flowchart</i> Manajemen Data Umum	86
4.15	<i>Flowchart</i> Manajemen Data Anggota	87

DAFTAR TABEL

3.1	Jadwal Pelaksanaan Penelitian	49
3.2	Kriteria Interpretasi Kelayakan <i>Functional Suitability</i>	55
3.3	Kriteria Interpretasi Kelayakan <i>Interaction Capability</i> dan <i>Beneficialness</i>	56
3.4	Kriteria Interpretasi Skor Lighthouse	57
4.1	Identitas Tenaga Ahli Penguji	90
4.2	Skor Pengujian <i>Functional Suitability</i>	91
4.3	Skor Pengujian <i>Interaction Capability</i>	92
4.4	Skor Pengujian <i>Beneficialness</i>	94

DAFTAR LAMPIRAN

Lampiran 1. Kisi-kisi Instrumen	109
Lampiran 2. Instrumen <i>Product Quality</i>	113
Lampiran 3. Instrumen <i>Quality in Use</i>	116
Lampiran 4. Surat Pernyataan Validasi Instrumen	117
Lampiran 5. <i>Use Case Diagram</i> Aktor Terautentikasi	118
Lampiran 6. <i>Use Case Description</i>	124
Lampiran 7. Kode Program Migrasi Tabel <i>users</i>	127
Lampiran 8. Kode Program <i>seeder</i> untuk Data <i>groups</i>	127
Lampiran 9. Kode Program <i>Model</i> User	128
Lampiran 10. Kode Program <i>Custom Resource</i>	130
Lampiran 11. Kode Program <i>Custom Resource Collection</i>	131
Lampiran 12. Kode Program <i>Resource</i> untuk <i>Model</i> User	132
Lampiran 13. Kode Program <i>Resource Collection</i> untuk <i>Model</i> User	132
Lampiran 14. Kode Program <i>Controller</i> <i>UserController</i>	132
Lampiran 15. Kode Program <i>Service</i> <i>OidcService</i>	136
Lampiran 16. Kode Program <i>Service</i> <i>SsoService</i>	139
Lampiran 17. Kode Program <i>Repository</i> <i>StorageRepository</i>	140
Lampiran 18. Kode Program <i>Guard</i> <i>OidcTokenGuard</i>	141
Lampiran 19. Kode Program <i>Guard</i> <i>OidcHeadersGuard</i>	144
Lampiran 20. Kode Program <i>Job</i> <i>SyncAchievementLeaderboards</i>	146
Lampiran 21. Kode Program <i>Cast</i> <i>Blob</i>	148
Lampiran 22. Kode Program Entitas User	148
Lampiran 23. Kode Program Antarmuka Repositori <i>UserRepository</i>	150
Lampiran 24. Kode Program Kelas Kesalahan <i>HttpError</i>	150
Lampiran 25. Kode Program <i>Use Case</i> <i>GetUsers</i>	152
Lampiran 26. Kode Program <i>Data Source</i> <i>InfinityApiDataSource</i>	153
Lampiran 27. Kode Program DTO <i>UserDto</i>	153
Lampiran 28. Kode Program Implementasi Repositori <i>UserRepositoryImpl</i>	154
Lampiran 29. Kode Program <i>Dockerfile Backend</i>	158
Lampiran 30. Kode Program <i>Dockerfile Frontend</i>	160
Lampiran 31. Kode Program <i>Pipeline</i> Rilis <i>Backend</i>	162
Lampiran 32. Kode Program <i>Pipeline</i> Rilis <i>Frontend</i>	168
Lampiran 33. <i>Deployment</i> dengan ArgoCD	174
Lampiran 34. Arsitektur <i>Deployment</i> di Kubernetes	175
Lampiran 35. Hasil Pengujian <i>Functional Suitability</i> dan <i>Interaction Capability</i>	176
Lampiran 36. Hasil Pengujian <i>Performance Efficiency Frontend</i>	201
Lampiran 37. Hasil Pengujian <i>Performance Efficiency Backend</i>	203

BAB I

PENDAHULUAN

A. Latar Belakang Masalah

Unit Kegiatan Mahasiswa (UKM) merupakan wadah pengembangan diri mahasiswa di luar kegiatan akademik yang memiliki peran strategis dalam pendidikan tinggi. Melalui UKM, mahasiswa dapat mengembangkan kemampuan berpikir kritis, keterampilan kepemimpinan, dan sosial yang tidak sepenuhnya diperoleh dari kurikulum formal (Pertiwi et al., 2021). INFINITE adalah salah satu divisi di bawah naungan UKM Rekayasa Teknologi (Restek) Universitas Negeri Yogyakarta (UNY). Organisasi ini berfungsi sebagai wadah bagi mahasiswa untuk mengembangkan diri terutama di bidang teknologi informasi, baik melalui kegiatan pengembangan keterampilan teknis, kompetisi, maupun proyek kolaboratif. INFINITE berperan penting dalam memfasilitasi pengembangan pengetahuan dan kemampuan anggota serta membangun jejaring profesional di kalangan mahasiswa yang memiliki minat dalam teknologi informasi.

Dalam menjalankan fungsinya, INFINITE memiliki kebutuhan pengelolaan informasi yang beragam dan kompleks. Kebutuhan tersebut mencakup: (1) manajemen keanggotaan yang meliputi data profil personal serta data keanggotaan; (2) pengelolaan kepengurusan yang mencakup struktur organisasi dan profil pengurus; (3) pencatatan data perlombaan yang meliputi informasi kompetisi, tim-tim yang dibentuk oleh anggota, proposal pengajuan pendanaan oleh tim, dan dokumentasi prestasi yang diraih; serta; (4) pengumpulan testimoni baik dari alumni tentang dampak INFINITE terhadap karier mereka maupun

testimoni proyek yang pernah dikerjakan oleh anggota. Kebutuhan pengelolaan data yang terintegrasi ini menjadi motivasi utama keberadaan sistem informasi organisasi INFINITE.

Untuk memenuhi kebutuhan tersebut, INFINITE telah memiliki dua sistem informasi berbasis web. Sistem pertama dibangun menggunakan *framework* Laravel dengan arsitektur *monolithic* dan di-*hosting* pada layanan *shared hosting* konvensional. Selain itu, terdapat juga sebagian data yang dikelola melalui sistem kedua yang di-*deploy* menggunakan Strapi, sebuah *Content Management System* (CMS) berbasis Node.js. Sistem Strapi ini dijalankan di server pribadi milik salah satu anggota INFINITE karena layanan *shared hosting* organisasi tidak mendukung *runtime* Node.js. Sistem berbasis Laravel menyimpan data terkait keanggotaan serta perlombaan, sedangkan sistem berbasis Strapi menyimpan data terkait kepengurusan dan testimoni.

Penggunaan dua sistem informasi ini menciptakan permasalahan dan tantangan. Pertama, kedua sistem tersebut menggunakan basis data yang berbeda dan terpisah. Hal ini menciptakan fragmentasi dan potensi inkonsistensi data, sehingga menyulitkan pengelolaan informasi secara menyeluruh. Misalnya data anggota yang juga menjadi pengurus, maka namanya akan dicatat di sistem Laravel pada data anggota, dan juga dicatat di sistem Strapi pada data pengurus secara manual dan rawan terjadi kesalahan penulisan. Kedua, sistem informasi Laravel maupun Strapi belum diintegrasikan dengan sistem SSO INFINITE, sehingga akun pengguna yang digunakan untuk mengakses sistem informasi harus didaftarkan secara terpisah di masing-masing sistem. Sistem Laravel memang

dapat dikembangkan untuk diintegrasikan dengan sistem SSO, tetapi sistem Strapi tidak menyediakan dukungan autentikasi OIDC untuk integrasi SSO pada versi gratisnya, dan penggunaan versi berbayar tidak memungkinkan karena keterbatasan dana organisasi.

Selain itu, seiring dengan perkembangan organisasi, INFINITE telah mulai mengadopsi platform komputasi awan (*cloud computing*) dengan berlangganan *Virtual Private Server* (VPS). Platform ini menggunakan Kubernetes untuk menyediakan abstraksi deklaratif dalam mengelola siklus hidup aplikasi terkontainerisasi, termasuk penjadwalan otomatis, penskalaan horizontal, dan pemulihan mandiri (Burns et al., 2016). Dalam implementasinya, INFINITE menggunakan pendekatan GitOps dengan ArgoCD sebagai alat untuk mengelola *deployment* aplikasi secara deklaratif, di mana Git berfungsi sebagai sumber kebenaran tunggal untuk konfigurasi infrastruktur (Argo Project, 2024; Limoncelli, 2018). Platform ini dibutuhkan karena INFINITE mulai memerlukan aplikasi baru lainnya yang tersedia secara *open-source* dalam bentuk *container*, misalnya Outline yang digunakan untuk mengelola dokumentasi organisasi.

Adopsi platform Kubernetes yang telah dilakukan ini menimbulkan permasalahan tambahan terkait sistem informasi yang ada. Platform baru ini memerlukan langganan infrastruktur VPS baru, akan tetapi sistem informasi Laravel yang ada saat ini masih memerlukan *shared hosting* karena belum memiliki bentuk *image* kontainer yang mampu dijalankan melalui Kubernetes. Pengoperasian dua infrastruktur secara bersamaan—*shared hosting* untuk sistem informasi Laravel dan VPS dengan Kubernetes untuk aplikasi-aplikasi

baru—mengakibatkan penambahan beban biaya yang signifikan dan tidak sepadan.

Masalah lain yang dihadapi terletak pada kebutuhan integrasi aplikasi-aplikasi baru yang di-*deploy* di Kubernetes dengan sistem informasi yang sudah ada. Kebutuhan integrasi ini berupa pertukaran data antar aplikasi, terutama terkait data anggota pengguna aplikasi. Sistem informasi Laravel yang telah ada tidak menyediakan *Application Programming Interface* (API) yang memadai untuk menunjang kebutuhan integrasi tersebut. Arsitektur *monolithic* dengan *rendering* tampilan di sisi server membatasi kemampuan sistem untuk berkomunikasi dengan aplikasi lain secara terprogram.

Berdasarkan permasalahan yang telah diuraikan, dikembangkan sistem informasi organisasi INFINITE yang terkontainerisasi dengan arsitektur *headless* menggunakan Laravel sebagai *backend* dan Next.js sebagai *frontend*. Laravel dipilih sebagai *backend* untuk menyesuaikan dengan teknologi yang telah digunakan pada sistem informasi versi sebelumnya, sehingga memudahkan proses migrasi dan transfer pengetahuan kepada anggota baru. Hal ini sejalan dengan penelitian oleh Yadav et al. (2019) yang menunjukkan bahwa Laravel memiliki kemudahan penggunaan yang menjadikannya sesuai untuk pengembangan aplikasi web, sehingga mudah dipelajari dan digunakan bahkan bagi pengembang pemula. Selain itu, Next.js dipilih sebagai *frontend* karena menyediakan fitur-fitur yang mempermudah pengembangan aplikasi berbasis React, seperti *Server-Side Rendering* (SSR) dan *automatic code splitting* yang dapat meningkatkan performa aplikasi. Hal ini juga ditunjukkan dalam penelitian oleh Silva and Farah (2018), yang membuktikan bahwa Next.js memberikan keunggulan performa melalui fitur

SSR dan optimasi otomatis. Kedua teknologi ini mendukung kebutuhan INFINITE dalam mengembangkan sistem yang dapat dikelola dan dikembangkan oleh anggota baru yang masih pemula, serta memenuhi kriteria sistem yang dapat berjalan di platform Kubernetes, menyediakan API untuk integrasi, dan dapat dikelola secara deklaratif melalui pendekatan GitOps.

B. Identifikasi Masalah

Berdasarkan latar belakang yang telah diuraikan, dapat diidentifikasi beberapa permasalahan sebagai berikut:

1. Sistem informasi yang ada terpecah dalam dua sistem berupa sistem Laravel di *shared hosting* dan sistem Strapi di server terpisah, menciptakan fragmentasi dan potensi inkonsistensi data.
2. Sistem informasi Strapi yang ada tidak mendukung integrasi dengan protokol OIDC untuk memungkinkan autentikasi terpusat dengan sistem SSO.
3. Sistem informasi Laravel yang ada tidak memiliki *image* kontainer untuk dapat di-*deploy* pada platform Kubernetes.
4. Pengoperasian dua infrastruktur secara bersamaan (*shared hosting* dan VPS) mengakibatkan inefisiensi biaya operasional.
5. Sistem informasi Laravel yang ada menggunakan arsitektur *monolithic* yang tidak menyediakan API untuk integrasi dengan aplikasi lain.

C. Batasan Masalah

Dari berbagai masalah yang teridentifikasi, penelitian ini membatasi ruang lingkup pada aspek-aspek berikut:

1. Pengembangan sistem informasi difokuskan pada pengelolaan data organisasi

INFINITE.

2. Arsitektur sistem tetap menggunakan Laravel sebagai *backend* dengan pendekatan *headless* yang menyediakan API dan Next.js sebagai *frontend* untuk antarmuka pengguna.
3. Sistem di-*deploy* sebagai aplikasi terkontainerisasi di platform Kubernetes dengan pendekatan GitOps menggunakan ArgoCD.
4. Integrasi autentikasi menggunakan protokol OIDC dengan sistem SSO INFINITE sebagai *identity provider*.
5. Penelitian ini tidak mencakup migrasi data dari sistem lama ke sistem baru. Proses migrasi data dianggap sebagai tahap terpisah dan dilakukan bersama dengan kepengurusan INFINITE ketika struktur basis data sistem baru telah siap.
6. Penelitian ini tidak mencakup proses penyiapan infrastruktur Kubernetes, sistem SSO, serta *deployment* basis data. Hal tersebut dianggap sudah tersedia dan siap digunakan sebagai bagian dari lingkungan operasional INFINITE.

D. Rumusan Masalah

Berdasarkan identifikasi dan batasan masalah, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana mengembangkan sistem informasi organisasi INFINITE yang terkontainerisasi dengan menggunakan Laravel *headless*, Next.js, dan autentikasi OIDC?
2. Bagaimana menguji sistem informasi organisasi INFINITE yang

dikembangkan untuk memastikan kualitas sistem dan pemenuhan kebutuhan pengguna?

E. Tujuan Penelitian

Sesuai dengan rumusan masalah tersebut, tujuan penelitian ini adalah:

1. Mengembangkan sistem informasi organisasi INFINITE yang terkontainerisasi dengan menggunakan Laravel *headless*, Next.js, dan autentikasi OIDC.
2. Menguji sistem informasi organisasi INFINITE yang dikembangkan untuk memastikan kualitas sistem dan pemenuhan kebutuhan pengguna.

F. Manfaat Penelitian

Hasil penelitian ini diharapkan dapat memberikan manfaat baik secara teoritis maupun praktis.

1. Manfaat Teoritis

Penelitian ini diharapkan dapat memberikan kontribusi pada pengembangan ilmu pengetahuan di bidang rekayasa perangkat lunak, khususnya dalam konteks pengembangan sistem informasi organisasi dengan komponen *backend* dan *frontend* terpisah yang dihubungkan melalui antarmuka API. Penelitian ini juga diharapkan dapat memberikan referensi implementasi integrasi autentikasi berstandar protokol OIDC, yang dapat menjadi acuan bagi pengembang lain yang ingin mengadopsi pendekatan serupa. Selain itu, penelitian ini diharapkan dapat memberikan wawasan tentang penerapan pendekatan *deployment* terkontainerisasi pada Kubernetes dengan GitOps.

2. Manfaat Praktis

- a. Bagi INFINITE: menyediakan sistem informasi terintegrasi yang dapat diakses melalui API dan mendukung integrasi dengan aplikasi lain di ekosistem Kubernetes.
- b. Bagi komunitas akademik: menyediakan studi kasus pengembangan sistem informasi organisasi mahasiswa dengan teknologi komputasi awan modern bagi mahasiswa, dosen, peneliti, dan tenaga kependidikan di lingkungan akademik Universitas Negeri Yogyakarta maupun umum.

BAB II

KAJIAN PUSTAKA

A. Kajian Teori

1. INFINITE

Divisi Teknologi Informasi INFINITE atau lebih sering disebut sebagai INFINITE, adalah satu dari empat divisi di bawah naungan Unit Kegiatan Mahasiswa (UKM) Rekayasa Teknologi (Restek) Universitas Negeri Yogyakarta (UNY). Organisasi ini telah ada sejak tahun 2014, dan berfungsi sebagai wadah bagi mahasiswa untuk mengembangkan diri di bidang teknologi informasi, baik melalui kegiatan pengembangan keterampilan teknis, kompetisi, maupun proyek kolaboratif. INFINITE berperan penting dalam memfasilitasi pengembangan pengetahuan dan kemampuan anggota dan membangun jejaring profesional di kalangan mahasiswa yang memiliki minat dalam teknologi informasi (INFINITE UNY, 2026).

Dalam operasional hariannya, INFINITE mengelola beragam informasi yang kompleks melalui berbagai sistem aplikasi yang dimiliki. Salah satu dari sistem tersebut adalah sistem informasi manajemen organisasi utama yang menjadi pusat pengelolaan data inti organisasi. Sistem ini mencakup berbagai data seperti: (1) data anggota, meliputi: profil akademik personal yaitu nama, Nomor Induk Mahasiswa (NIM), strata, fakultas, dan program studi; kontak yaitu nomor telepon dan email; serta status keanggotaan yaitu *role* atau persona, tanggal mulai dan berakhir, status aktif keanggotaan, dan status istimewa keanggotaan; (2) data pengurus berupa *Core Team* dan *Community Group Administrator*, meliputi:

struktur pengurus berupa divisi; dan profil pengurus berupa jabatan, periode masa jabatan, serta foto pengurus; (3) data kompetisi, meliputi: informasi kompetisi yaitu nama kompetisi, penyelenggara, tipe penyelenggara, tingkat kompetisi, dan cabang kompetisi; tim-tim yang dibentuk oleh anggota untuk mengikuti kompetisi yaitu nama tim, nama ketua, dan nama-nama anggota; proposal pengajuan pendanaan oleh tim yaitu detail kompetisi, tanggal pelaksanaan kompetisi, dokumen bukti lolos kompetisi, dan dokumen proposal pendanaan; serta dokumentasi prestasi yang diraih yaitu detail kompetisi, tanggal pelaksanaan kompetisi, dokumen sertifikat, dan dokumen foto kegiatan; serta (4) data testimoni, baik dari alumni tentang dampak INFINITE terhadap karier yaitu nama alumni, posisi pekerjaan, serta testimoni; maupun testimoni proyek yang pernah dikerjakan oleh anggota yaitu nama proyek, deskripsi proyek, dan dokumentasi foto proyek.

2. Sistem Informasi

Sistem informasi merupakan kombinasi terorganisir dari manusia, perangkat keras, perangkat lunak, jaringan komunikasi, sumber data, serta kebijakan dan prosedur yang menyimpan, mengambil, mengubah, dan menyebarkan informasi dalam suatu organisasi (O'Brien and Marakas, 2012). Menurut Laudon and Laudon (2020), sistem informasi dapat didefinisikan secara teknis sebagai sekumpulan komponen yang saling terkait yang mengumpulkan, memproses, menyimpan, dan mendistribusikan informasi untuk mendukung pengambilan keputusan, koordinasi, dan pengendalian dalam organisasi. Sistem informasi juga membantu manajer dan pekerja dalam menganalisis masalah, memvisualisasikan subjek yang kompleks, dan menciptakan produk baru.

Komponen utama sistem informasi terdiri dari lima elemen fundamental yang saling berinteraksi. Pertama, *hardware* mencakup perangkat fisik seperti komputer, server, dan perangkat jaringan yang digunakan untuk memproses dan menyimpan data. Kedua, *software* meliputi program aplikasi dan sistem operasi yang mengendalikan operasi perangkat keras dan memungkinkan pemrosesan data. Ketiga, data merupakan fakta mentah yang diorganisasi dan diproses menjadi informasi yang bermakna bagi pengguna. Keempat, prosedur adalah serangkaian instruksi dan aturan yang mengatur bagaimana komponen lain bekerja sama untuk memproses data menjadi informasi. Kelima, manusia (*people*) mencakup pengguna akhir, administrator sistem, dan profesional teknologi informasi yang mengoperasikan dan mengelola sistem (O'Brien and Marakas, 2012).

3. Backend dengan Laravel

Laravel adalah kerangka kerja (*framework*) aplikasi web berbasis PHP yang dikembangkan oleh Taylor Otwell pada tahun 2011. Laravel dirancang dengan filosofi pengembangan yang elegan dan ekspresif, menyediakan sintaks yang bersih serta fitur-fitur modern untuk mempercepat pengembangan aplikasi web (Laravel LLC, 2026). Sebagai salah satu kerangka kerja PHP yang paling populer, Laravel menawarkan berbagai fitur bawaan seperti sistem *routing* yang fleksibel, *Object-Relational Mapping* (ORM) melalui Eloquent, sistem migrasi basis data, antrian pekerjaan (*job queues*), serta sistem autentikasi dan otorisasi yang komprehensif. Studi komparatif terhadap kerangka kerja PHP menunjukkan bahwa Laravel memiliki keunggulan dalam hal dokumentasi, komunitas pengembang, dan ekosistem paket pendukung (Laaziri et al., 2019).

3.1. PHP

PHP adalah bahasa pemrograman skrip sisi server yang awalnya dikembangkan oleh Rasmus Lerdorf pada tahun 1994 dengan nama *Personal Home Page Tools*. Seiring perkembangannya, PHP bertransformasi menjadi bahasa pemrograman yang lengkap dan namanya diubah menjadi akronim rekursif *PHP: Hypertext Preprocessor* (The PHP Documentation Group, 2026). PHP dirancang khusus untuk pengembangan web dan dapat disisipkan ke dalam HTML, menjadikannya pilihan populer untuk membangun aplikasi web dinamis.

Dibandingkan dengan bahasa pemrograman *backend* lainnya seperti Python, Java, atau Node.js, PHP memiliki beberapa keunggulan. Pertama, PHP memiliki kurva pembelajaran yang relatif landai dan sintaks yang mudah dipahami. Kedua, ekosistem PHP sangat matang dengan ketersediaan banyak kerangka kerja seperti Laravel, Symfony, dan CodeIgniter. Ketiga, sebagian besar penyedia *hosting* web mendukung PHP secara *native*, memudahkan proses *deployment*. Namun demikian, PHP juga memiliki keterbatasan seperti inkonsistensi penamaan fungsi bawaan dan penanganan *concurrency* yang kurang optimal dibandingkan Node.js. Dalam konteks proyek ini, PHP dipilih untuk menjaga kemudahan transisi dari sistem informasi versi sebelumnya, dan karena kerangka kerja Laravel yang digunakan sudah menyediakan abstraksi yang elegan untuk pengembangan API *backend* serta memiliki dokumentasi dan komunitas yang sangat aktif.

3.2. Arsitektur *Model-View-Controller* (MVC)

MVC adalah pola arsitektur perangkat lunak yang memisahkan aplikasi menjadi tiga komponen utama yang saling terhubung namun memiliki tanggung

jawab yang berbeda. Arsitektur ini pertama kali diperkenalkan oleh Trygve Reenskaug pada tahun 1979 di Xerox PARC dalam konteks pengembangan antarmuka pengguna Smalltalk-80. Konsep MVC kemudian dipopulerkan dan didokumentasikan secara lebih luas oleh Krasner and Pope (1988) dalam publikasi yang menjadi referensi klasik untuk implementasi arsitektur ini. Pemisahan ini bertujuan untuk meningkatkan modularitas, memudahkan pemeliharaan, dan memungkinkan pengembangan paralel oleh tim yang berbeda. Dalam konteks teknologi web modern, arsitektur MVC telah berevolusi dengan berbagai adaptasi untuk memenuhi kebutuhan aplikasi web yang kompleks (Sastypratiwi and Yulianti, 2019).

Komponen pertama, *Model*, bertanggung jawab atas logika bisnis dan pengelolaan data aplikasi. *Model* merepresentasikan struktur data, aturan validasi, dan operasi terhadap basis data. Dalam Laravel, *Model* diimplementasikan melalui Eloquent ORM yang menyediakan antarmuka objek untuk berinteraksi dengan tabel basis data. Komponen kedua, *View*, menangani presentasi data kepada pengguna. *View* menerima data dari *Controller* dan merendernya dalam format yang sesuai untuk ditampilkan. Komponen ketiga, *Controller*, berperan sebagai perantara antara *Model* dan *View*. *Controller* menerima permintaan dari pengguna, memproses logika yang diperlukan dengan memanggil *Model*, dan mengembalikan respons melalui *View* (Laravel LLC, 2026).

Dalam implementasi Laravel *headless*, arsitektur MVC diadaptasi tanpa komponen *Blade templating* untuk menampilkan HTML. Sebagai gantinya, Laravel menggunakan *Resources* dan *Collections* untuk memformat data *Model*

menjadi respons JSON yang konsisten. Laravel menyediakan fitur-fitur seperti *route model binding*, *middleware* untuk autentikasi dan otorisasi, serta *resource controllers* yang membantu untuk pengembangan API (Laravel LLC, 2026).

3.3. *RESTful Application Programming Interface (API)*

Representational State Transfer (REST) adalah gaya arsitektur untuk sistem terdistribusi yang pertama kali diperkenalkan oleh Roy Fielding dalam disertasinya pada tahun 2000 (Fielding, 2000). REST mendefinisikan sekumpulan batasan arsitektural yang, ketika diterapkan secara keseluruhan, menekankan skalabilitas interaksi antar komponen, antarmuka yang seragam, dan *deployment* komponen yang independen. API yang mengikuti prinsip REST disebut sebagai *RESTful API*.

Prinsip utama REST meliputi: (1) arsitektur *client-server* yang memisahkan antarmuka pengguna dari penyimpanan data; (2) *statelessness*, di mana setiap permintaan dari klien harus berisi semua informasi yang diperlukan untuk memproses permintaan; (3) *cacheability* yang memungkinkan respons ditandai sebagai dapat di-cache atau tidak; (4) sistem berlapis (*layered system*) yang memungkinkan penambahan lapisan perantara; dan (5) antarmuka yang seragam menggunakan metode HTTP standar (GET, POST, PUT, DELETE) untuk operasi *Create, Read, Update, Delete* (CRUD) (Richardson et al., 2013).

3.4. *Basis Data PostgreSQL*

PostgreSQL adalah sistem manajemen basis data relasional objek (*Object-Relational Database Management System/ORDBMS*) sumber terbuka yang dikembangkan sejak tahun 1986 sebagai bagian dari proyek POSTGRES di University of California, Berkeley (The PostgreSQL Global Development Group,

2026). PostgreSQL dikenal karena keandalan, integritas data, dan kepatuhannya terhadap standar SQL. Sebagai basis data sumber terbuka, PostgreSQL dapat digunakan, dimodifikasi, dan didistribusikan secara bebas tanpa biaya lisensi.

Fitur-fitur utama PostgreSQL meliputi: (1) kepatuhan penuh terhadap properti *Atomicity, Consistency, Isolation, Durability* (ACID) yang menjamin integritas transaksi; (2) dukungan tipe data yang ekstensif termasuk JSON/JSONB untuk data semi-terstruktur; (3) sistem indeks yang canggih termasuk B-tree, Hash, GiST, dan GIN; (4) *full-text search* bawaan untuk pencarian teks; serta (5) kemampuan ekstensi yang memungkinkan penambahan fungsi kustom (Ferrari and Pirozzi, 2023). Dalam konteks proyek INFINITE, PostgreSQL dipilih karena beberapa alasan: pertama, PostgreSQL telah digunakan sebelumnya untuk beberapa aplikasi internal INFINITE sehingga tidak memerlukan *deployment* baru; kedua, sebagai basis data sumber terbuka, PostgreSQL tidak memerlukan biaya lisensi yang sejalan dengan prinsip pengembangan organisasi mahasiswa; dan ketiga, PostgreSQL memiliki performa yang sangat baik untuk beban kerja transaksional maupun analitis.

4. Frontend dengan Next.js

Next.js adalah kerangka kerja React yang dikembangkan oleh Vercel untuk membangun aplikasi web dengan kemampuan *rendering* yang fleksibel (Vercel Inc., 2026). Next.js memperluas kemampuan React dengan menyediakan fitur-fitur seperti *Server-Side Rendering* (SSR), *Static Site Generation* (SSG), *Incremental Static Regeneration* (ISR), dan sistem *routing* berbasis *file system*. Kombinasi fitur-fitur ini memungkinkan pengembang untuk mengoptimalkan performa

aplikasi sesuai dengan kebutuhan masing-masing halaman.

React.js sendiri adalah pustaka JavaScript untuk membangun antarmuka pengguna yang dikembangkan oleh Meta (sebelumnya Facebook) dan dirilis sebagai sumber terbuka pada tahun 2013 (Meta Platforms Inc., 2026). React menggunakan pendekatan berbasis komponen (*component-based*) yang memungkinkan pengembang membangun UI dari potongan-potongan kode yang terisolasi dan dapat digunakan kembali (Kaushalya and Perera, 2021). Konsep utama React meliputi: (1) *Virtual DOM* yang mengoptimalkan pembaruan UI dengan meminimalkan manipulasi DOM langsung; (2) *JavaScript XML* (JSX) yang memungkinkan penulisan markup dalam kode JavaScript; (3) *unidirectional data flow* yang membuat aliran data lebih mudah diprediksi; serta (4) *hooks* yang memungkinkan penggunaan *state* dan fitur React lainnya dalam komponen fungsional.

4.1. TypeScript

TypeScript adalah bahasa pemrograman sumber terbuka yang dikembangkan dan dipelihara oleh Microsoft sejak tahun 2012 (Microsoft, 2026). TypeScript merupakan *superset* dari JavaScript yang menambahkan sistem tipe statis opsional dan fitur-fitur pemrograman berorientasi objek yang lebih kaya. Kode TypeScript dikompilasi (*transpiled*) menjadi JavaScript murni yang dapat dijalankan di berbagai lingkungan seperti peramban, Node.js, atau *runtime* JavaScript lainnya.

Sistem tipe TypeScript memungkinkan pendeteksian kesalahan pada saat kompilasi alih-alih saat eksekusi, yang secara signifikan meningkatkan keandalan kode (Bierman et al., 2014). Fitur-fitur utama TypeScript meliputi: (1) anotasi tipe

untuk variabel, parameter fungsi, dan nilai kembalian; (2) *interfaces* dan *type aliases* untuk mendefinisikan bentuk objek; (3) *generics* untuk membuat komponen yang dapat bekerja dengan berbagai tipe; (4) *enums* untuk mendefinisikan sekumpulan konstanta bernama; serta (5) dukungan *IntelliSense* yang lebih baik di *Integrated Development Environment* (IDE). Dalam pengembangan aplikasi *frontend* dengan Next.js, penggunaan TypeScript meningkatkan *maintainability* kode, memudahkan *refactoring*, dan memperbaiki kolaborasi tim melalui dokumentasi tipe yang eksplisit.

4.2. *Clean Architecture*

Clean Architecture adalah pendekatan arsitektur perangkat lunak yang diperkenalkan oleh Robert C. Martin (Uncle Bob) yang menekankan pemisahan kepentingan (*separation of concerns*) dan independensi dari kerangka kerja, basis data, serta antarmuka pengguna (Martin, 2017). Prinsip utama *Clean Architecture* adalah *Dependency Rule* yang menyatakan bahwa dependensi kode sumber hanya boleh mengarah ke dalam, menuju kebijakan tingkat tinggi (*high-level policies*).

Arsitektur ini mengorganisasi kode ke dalam beberapa lapisan konsentris dengan tanggung jawab yang berbeda. Lapisan terdalam adalah *Domain* (atau *Entities*) yang berisi logika bisnis inti dan aturan bisnis yang paling umum. Lapisan *Application* (atau *Use Cases*) berisi logika aplikasi spesifik yang mengorkestrasi aliran data ke dan dari entitas. Lapisan *Infrastructure* (atau *Interface Adapters*) berisi implementasi konkret dari antarmuka yang didefinisikan di lapisan dalam, seperti repositori basis data atau klien API. Lapisan terluar adalah *Presentation* (atau *Frameworks and Drivers*) yang berisi detail

implementasi seperti kerangka kerja UI atau pustaka eksternal.

Dalam penelitian ini, *Clean Architecture* diterapkan melalui struktur direktori yang mencerminkan lapisan-lapisan tersebut: direktori `domain` untuk entitas dan aturan bisnis, `application` untuk *use cases* dan layanan aplikasi, `infrastructure` untuk implementasi repositori dan integrasi API, serta `presentation` untuk komponen React dan halaman Next.js. Pemisahan ini meningkatkan modularitas dan memungkinkan perubahan pada satu lapisan tanpa memengaruhi lapisan lainnya.

5. Deployment dengan Kubernetes dan GitOps

5.1. Komputasi Awan

Dalam operasional hariannya, INFINITE telah memanfaatkan komputasi awan untuk menjalankan berbagai sistem aplikasi yang dimiliki. Menurut National Institute of Standards and Technology (NIST), komputasi awan atau *cloud computing* adalah suatu model yang memungkinkan akses secara *on-demand* melalui jaringan kepada sekumpulan sumber daya komputasi bersama yang dapat disediakan dan dilepaskan dengan cepat. Karakteristik utama dari komputasi awan meliputi: (1) layanan mandiri sesuai kebutuhan, pengguna dapat mengalokasikan sumber daya komputasi secara terprogram tanpa interaksi manusia; (2) akses jaringan luas, tersedia melalui jaringan standar yang mendukung berbagai platform pengguna; (3) penggabungan sumber daya, pengelolaan sumber daya dilakukan secara terpusat dengan model *multi-tenant*; (4) elastisitas cepat, memungkinkan penyediaan dan pelepasan sumber daya secara dinamis sesuai beban kerja; dan (5) layanan terukur, optimasi penggunaan sumber daya melalui mekanisme

pengukuran yang transparan (Mell and Grance, 2011). Secara umum, layanan komputasi awan terbagi ke dalam 3 model, yaitu *Infrastructure as a Service* (IaaS), *Platform as a Service* (PaaS), dan *Software as a Service* (SaaS). IaaS menyediakan sumber daya komputasi dasar seperti mesin virtual, penyimpanan, dan jaringan; PaaS menyediakan platform untuk pengembangan aplikasi yang mencakup sistem operasi, *runtime*, dan alat pengembangan; sedangkan SaaS menyediakan aplikasi siap pakai yang dapat diakses melalui web tanpa perlu instalasi lokal (Younis et al., 2024).

5.2. Kontainer dan Kubernetes

Seiring dengan adopsi komputasi awan yang meluas, tantangan utama yang dihadapi organisasi bergeser dari penyediaan infrastruktur ke pengelolaan kompleksitas sistem terdistribusi. Untuk mengatasi tantangan ini, platform orkestrasi seperti Kubernetes telah menjadi fondasi penting dalam membangun dan mengelola aplikasi *cloud-native* yang skalabel dan dapat diandalkan. Kubernetes adalah platform orkestrasi kontainer sumber terbuka yang awalnya dikembangkan oleh Google berdasarkan pengalaman mengelola sistem berskala besar melalui proyek internal Borg (Burns et al., 2016). Platform ini menyediakan abstraksi deklaratif untuk mengelola siklus hidup aplikasi terkontainerisasi, termasuk penjadwalan otomatis, penskalaan horizontal, pemulihan mandiri, penemuan layanan, serta penyeimbangan beban. Kubernetes mengorganisasi sumber daya dalam unit-unit seperti *Pod* (unit terkecil yang dapat dijadwalkan), *Deployment* (manajemen replika dan pembaruan), *Service* (abstraksi jaringan), dan *Namespace* (isolasi logis) (Burns et al., 2019). Hingga kini, adopsi Kubernetes telah meningkat

sangat pesat. Survei Cloud Native Computing Foundation (CNCF) pada tahun 2025 menunjukkan bahwa 82% organisasi telah menggunakan Kubernetes dalam lingkungan produksi (Lawson and Sica, 2026).

5.3. *Continuous Integration/Continuous Deployment (CI/CD)*

Continuous Integration (CI) dan *Continuous Deployment* (CD) adalah praktik pengembangan perangkat lunak yang bertujuan untuk mengotomatisasi dan mempercepat siklus rilis perangkat lunak (Shahin et al., 2017). CI mengacu pada praktik mengintegrasikan perubahan kode dari beberapa kontributor ke dalam repositori bersama secara sering, biasanya beberapa kali sehari, disertai dengan *build* dan pengujian otomatis untuk mendeteksi kesalahan sedini mungkin. CD memperluas CI dengan mengotomatisasi proses *deployment* ke lingkungan produksi atau *staging*. Terdapat dua varian CD: *Continuous Delivery* yang mengotomatisasi proses hingga siap di-*deploy* namun memerlukan persetujuan manual, dan *Continuous Deployment* yang sepenuhnya otomatis hingga produksi (Humble and Farley, 2010).

5.4. *GitHub Actions*

GitHub Actions adalah platform CI/CD yang terintegrasi langsung dengan GitHub, memungkinkan otomatisasi alur kerja pengembangan perangkat lunak (GitHub Inc., 2024). Alur kerja (*workflow*) didefinisikan dalam berkas YAML yang disimpan di direktori `.github/workflows/` dan dapat dipicu oleh berbagai *event* seperti *push*, *pull request*, atau jadwal *cron*. Dalam proyek ini, GitHub Actions digunakan untuk membangun *container image* dari *backend* Laravel dan *frontend* Next.js setiap kali ada perubahan kode yang di-*push* ke cabang repositori

produksi.

5.5. GitOps dengan ArgoCD

GitOps adalah paradigma operasional yang menggunakan Git sebagai sumber kebenaran tunggal (*single source of truth*) untuk infrastruktur deklaratif dan aplikasi. Konsep GitOps merepresentasikan evolusi praktik DevOps yang memanfaatkan sistem kontrol versi sebagai mekanisme sentral untuk manajemen infrastruktur dan *deployment* aplikasi (Beetz and Harrer, 2022). Prinsip utama GitOps meliputi: (1) konfigurasi sistem yang dideskripsikan secara deklaratif; (2) status sistem yang diinginkan disimpan di Git; (3) perubahan yang disetujui diterapkan secara otomatis ke sistem; dan (4) agen perangkat lunak memastikan kebenaran dan melaporkan penyimpangan. Pendekatan GitOps telah menjadi bagian penting dari ekosistem DevOps modern, memberikan visibilitas, auditabilitas, dan konsistensi dalam pengelolaan infrastruktur (Leite et al., 2019). ArgoCD adalah salah satu contoh alat GitOps sumber terbuka untuk Kubernetes yang secara kontinu memantau repositori Git dan menyinkronkan status kluster Kubernetes agar sesuai dengan konfigurasi yang didefinisikan (Argo Project, 2024). ArgoCD menyediakan fitur seperti deteksi *drift*, *rollback* otomatis, serta antarmuka web untuk visualisasi dan manajemen aplikasi.

6. Autentikasi dan Single Sign-On (SSO)

Autentikasi adalah proses verifikasi identitas pengguna atau sistem yang mengakses sumber daya digital (Grassi et al., 2017). Dalam konteks aplikasi web, autentikasi memastikan bahwa pengguna yang mengakses sistem adalah benar-benar pihak yang mengklaim identitasnya. Menurut Ometov et al. (2018),

otentikasi dapat diklasifikasikan berdasarkan faktor yang digunakan: (1) *knowledge-based* seperti kata sandi atau PIN, (2) *possession-based* seperti kartu atau perangkat *hardware token*, dan (3) *inherence-based* seperti sidik jari atau pengenalan wajah. Dalam aplikasi modern, kombinasi beberapa faktor (*multi-factor authentication*) semakin umum digunakan untuk meningkatkan keamanan.

SSO adalah mekanisme autentikasi yang memungkinkan pengguna untuk mengakses berbagai aplikasi dengan menggunakan satu set kredensial tunggal (Pashalidis and Mitchell, 2003). Implementasi SSO memberikan beberapa manfaat signifikan. Pertama, SSO meningkatkan pengalaman pengguna dengan mengeliminasi kebutuhan untuk mengingat dan mengelola kredensial yang berbeda untuk setiap aplikasi. Kedua, SSO mengurangi beban administratif dalam pengelolaan akun pengguna dan pengaturan ulang kata sandi. Ketiga, SSO dapat meningkatkan keamanan dengan memungkinkan penerapan kebijakan autentikasi yang konsisten di seluruh aplikasi (Liu et al., 2018). Dalam konteks organisasi yang memiliki ekosistem aplikasi yang kompleks seperti INFINITE, SSO menjadi solusi yang efektif untuk menyederhanakan manajemen identitas dan akses.

6.1. OAuth 2.0

OAuth 2.0 adalah kerangka kerja otorisasi yang memungkinkan aplikasi pihak ketiga untuk mendapatkan akses terbatas ke suatu layanan atas nama pemilik sumber daya (Hardt, 2012). Berbeda dengan protokol autentikasi, OAuth 2.0 dirancang khusus untuk skenario otorisasi di mana aplikasi memerlukan akses ke sumber daya tanpa perlu mengetahui kredensial pengguna. Menurut Fett et al.

(2016), OAuth 2.0 mendefinisikan empat peran utama: (1) *Resource Owner*, pemilik sumber daya, biasanya pengguna; (2) *Resource Server*, server yang menyimpan sumber daya yang dilindungi; (3) *Client*, aplikasi yang meminta akses; dan (4) *Authorization Server*, server yang mengeluarkan *access token* setelah berhasil mengautentikasi *resource owner*.

OAuth 2.0 mendefinisikan beberapa *grant type* untuk berbagai kasus penggunaan. *Authorization Code Grant* adalah yang paling aman dan umum digunakan untuk aplikasi web, di mana *client* menerima kode otorisasi yang kemudian ditukar dengan *access token*. *Implicit Grant* dirancang untuk aplikasi berbasis peramban namun tidak lagi direkomendasikan karena masalah keamanan. *Resource Owner Password Credentials Grant* memungkinkan *client* menggunakan kredensial pengguna secara langsung, namun hanya cocok untuk aplikasi yang sangat tepercaya. *Client Credentials Grant* digunakan untuk autentikasi *machine-to-machine* tanpa konteks pengguna manusia (Lodderstedt et al., 2020).

6.2. OpenID Connect (OIDC)

OIDC adalah lapisan identitas yang dibangun di atas OAuth 2.0 yang menambahkan kemampuan autentikasi standar (Sakimura et al., 2014). Sementara OAuth 2.0 berfokus pada otorisasi, OIDC menyediakan mekanisme untuk memverifikasi identitas pengguna dan mendapatkan informasi profil dasar. OIDC menambahkan konsep *ID Token*, yang merupakan *JSON Web Token (JWT)* yang berisi klaim tentang autentikasi pengguna dan atribut identitas (Jones et al., 2015). Komponen utama OIDC mencakup: (1) *OpenID Provider (OP)*, yaitu server OAuth 2.0 yang mampu mengautentikasi pengguna dan menyediakan klaim

tentang autentikasi tersebut; (2) *Relying Party* (RP), yaitu aplikasi yang menggunakan OIDC untuk autentikasi; dan (3) *End User*, yaitu pengguna yang identitasnya diverifikasi.

OIDC mendefinisikan beberapa *endpoint* standar yang harus disediakan oleh *OpenID Provider*: (1) *Authorization endpoint* untuk memulai proses autentikasi, (2) *Token endpoint* untuk menukar kode otorisasi dengan token, (3) *UserInfo endpoint* untuk mendapatkan klaim tambahan tentang pengguna, dan (4) *Discovery endpoint* yang menyediakan metadata konfigurasi OIDC secara otomatis. Standardisasi ini memungkinkan interoperabilitas yang tinggi antara berbagai implementasi. Namun, implementasi OIDC harus memperhatikan aspek keamanan secara cermat, termasuk validasi token yang benar, penanganan alur autentikasi yang aman, dan perlindungan terhadap serangan seperti *token substitution* dan *session fixation* (Fett et al., 2014; Mladenov et al., 2016).

OIDC juga mendefinisikan tiga alur autentikasi utama dengan karakteristik yang berbeda. Pertama, *Authorization Code Flow* merupakan alur yang paling aman di mana klien menerima kode otorisasi dari *authorization endpoint*, kemudian menukarkan kode tersebut dengan *ID token* dan *access token* melalui *Token endpoint*. Alur ini cocok untuk aplikasi web tradisional yang memiliki komponen *server-side* karena *client secret* dapat disimpan dengan aman di server. Kedua, *Implicit Flow* dirancang untuk aplikasi yang berjalan sepenuhnya di peramban tanpa komponen *backend*, di mana token langsung dikembalikan dari *Authorization endpoint* tanpa tahap pertukaran kode. Namun, alur ini tidak lagi direkomendasikan karena risiko keamanan yang terkait dengan eksposur token di

URL dan ketidakmampuan untuk mengautentikasi klien secara efektif. Ketiga, *Hybrid Flow* menggabungkan elemen dari kedua alur sebelumnya, memungkinkan beberapa token dikembalikan dari *Authorization endpoint* dan beberapa dari *Token endpoint*, memberikan fleksibilitas untuk skenario yang memerlukan akses langsung ke *ID token* di *frontend* sambil tetap mempertahankan keamanan untuk *access token*. Untuk aplikasi web modern, *Authorization Code Flow* dengan *Proof Key for Code Exchange* (PKCE) direkomendasikan untuk keamanan maksimal, bahkan untuk aplikasi *single-page* dan *mobile* yang sebelumnya menggunakan *Implicit Flow* (Parecki et al., 2025).

6.3. Security Assertion Markup Language (SAML)

SAML adalah standar terbuka berbasis XML untuk pertukaran data autentikasi dan otorisasi antara *identity provider* dan *service provider* (Hughes et al., 2005). SAML telah lama digunakan dalam lingkungan *enterprise* untuk mengimplementasikan SSO, terutama untuk aplikasi berbasis web tradisional. Berbeda dengan OAuth 2.0 dan OIDC yang menggunakan format JSON dan REST, SAML menggunakan format XML dan umumnya dikomunikasikan melalui HTTP POST atau redirect.

Arsitektur SAML terdiri dari tiga komponen utama: (1) *Identity Provider* (IdP) yang mengautentikasi pengguna dan mengeluarkan *assertion* yang berisi informasi autentikasi dan atribut pengguna, (2) *Service Provider* (SP) yang menyediakan layanan dan menerima *assertion* dari IdP untuk membuat keputusan akses, dan (3) *User Agent* (biasanya peramban) yang memfasilitasi komunikasi antara IdP dan SP. SAML mendefinisikan tiga jenis *assertion*: *Authentication*

Assertion untuk membuktikan autentikasi pengguna, *Attribute Assertion* untuk menyampaikan atribut pengguna, dan *Authorization Decision Assertion* untuk keputusan akses (Cantor et al., 2005).

Dalam alur SSO berbasis SAML, proses dimulai ketika pengguna mencoba mengakses *service provider*. SP mengarahkan pengguna ke IdP untuk autentikasi. Setelah pengguna berhasil diautentikasi oleh IdP, IdP mengeluarkan *SAML assertion* yang ditandatangani secara digital dan mengirimkannya kembali ke SP melalui peramban pengguna. SP memvalidasi *assertion* dan memberikan akses kepada pengguna. SAML mendukung dua binding utama: HTTP Redirect Binding dan HTTP POST Binding, yang menentukan bagaimana pesan SAML ditransmisikan (Mainka et al., 2017).

6.4. Perbandingan Protokol SSO

Ketiga protokol SSO memiliki karakteristik dan kasus penggunaan yang berbeda. SAML lebih banyak digunakan dalam lingkungan *enterprise* yang sudah mapan, terutama untuk integrasi dengan sistem *legacy* dan aplikasi berbasis web tradisional. SAML menyediakan fitur yang sangat lengkap dan matang, namun kompleksitas XML dan overhead protokol dapat menjadi hambatan untuk aplikasi modern. OAuth 2.0 dan OIDC lebih populer dalam ekosistem aplikasi modern, terutama untuk aplikasi *mobile*, *Single-Page Application* (SPA), dan API. OAuth 2.0 cocok untuk skenario di mana aplikasi pihak ketiga memerlukan akses terbatas ke sumber daya pengguna tanpa perlu mengetahui kredensial, seperti integrasi dengan layanan media sosial atau aplikasi *cloud*. OIDC melengkapi OAuth 2.0 dengan menambahkan lapisan autentikasi yang terstandarisasi, menjadikannya

pilihan yang ideal untuk implementasi SSO modern yang memerlukan autentikasi pengguna dan informasi profil. Analisis praktik terbaik OAuth/OIDC untuk aplikasi modern oleh Sharif et al. (2022) menunjukkan pentingnya pemilihan alur protokol yang aman sesuai konteks aplikasi. Panduan keamanan terkini dari Lodderstedt et al. (2025) juga menegaskan bahwa OIDC dan OAuth 2.0 telah menjadi standar *de facto* untuk autentikasi dan otorisasi berbasis web maupun *mobile*.

7. Rekayasa Perangkat Lunak

7.1. Perangkat Lunak Berbasis *Website*

Perangkat lunak berbasis *website* atau aplikasi web adalah jenis perangkat lunak yang diakses melalui peramban dan berjalan di atas protokol HTTP/HTTPS, di mana sebagian atau seluruh logika aplikasi dieksekusi di sisi server dan antarmuka pengguna ditampilkan melalui peramban (Taivalsaari et al., 2021). Berbeda dengan aplikasi *desktop* tradisional yang harus diinstal secara lokal di setiap perangkat, aplikasi web dapat diakses dari berbagai perangkat tanpa memerlukan instalasi khusus selain peramban. Menurut Mikkonen et al. (2019), evolusi teknologi web dalam dekade terakhir telah mengubah web dari platform untuk menyajikan dokumen statis menjadi platform pengembangan aplikasi yang matang dan *powerful*, memungkinkan pembuatan aplikasi yang kompleks dengan pengalaman pengguna yang setara atau bahkan melampaui aplikasi *native*.

Keunggulan perangkat lunak berbasis *website* mencakup beberapa aspek signifikan. Pertama, aksesibilitas lintas platform yang memungkinkan pengguna mengakses aplikasi dari berbagai sistem operasi (Windows, macOS, Linux) dan

perangkat (*desktop, tablet, smartphone*) selama memiliki peramban dan koneksi internet. Kedua, kemudahan distribusi dan pembaruan, di mana pengembang dapat melakukan *deployment* pembaruan secara terpusat di server tanpa memerlukan pengguna untuk mengunduh dan menginstal versi baru. Ketiga, biaya pengembangan yang relatif lebih rendah karena hanya perlu mengembangkan satu basis kode untuk berbagai platform. Keempat, manajemen data terpusat yang memudahkan sinkronisasi dan *backup* data pengguna (Charland and Leroux, 2011).

Arsitektur aplikasi web modern dapat diklasifikasikan ke dalam beberapa model berdasarkan pembagian tanggung jawab antara *client* dan *server*. Arsitektur *traditional server-side rendering* merender seluruh halaman HTML di sisi server dan mengirimkannya ke *client*, memberikan keunggulan dalam hal *Search Engine Optimization* (SEO) dan waktu muat awal yang cepat, namun memiliki keterbatasan dalam hal interaktivitas. Arsitektur SPA memuat seluruh aplikasi sekali di awal dan kemudian berkomunikasi dengan server melalui API untuk data, memberikan pengalaman pengguna yang lebih responsif namun dapat menghadapi tantangan dalam SEO dan waktu muat awal. Arsitektur *Progressive Web App* (PWA) menggabungkan fitur aplikasi web dengan kemampuan aplikasi *native* seperti kerja *offline* dan notifikasi *push* (Biørn-Hansen et al., 2018). Dalam perkembangan terkini, arsitektur *JavaScript, APIs, and Markup* (JAMstack) telah muncul sebagai pendekatan modern yang memisahkan *frontend* dari *backend*, memanfaatkan API untuk komunikasi data, dan meningkatkan performa melalui *pre-rendering* dan penggunaan *Content Delivery Network* (CDN). Studi empiris

menunjukkan bahwa JAMstack dapat meningkatkan performa aplikasi web secara signifikan dibandingkan arsitektur tradisional (Markovic et al., 2022).

Dalam penelitian ini, arsitektur *headless* diterapkan sebagai pendekatan untuk memisahkan *backend* dan *frontend* secara total. Arsitektur *headless* merujuk pada sistem di mana lapisan *backend* yang mengelola logika bisnis, basis data, dan API sepenuhnya terpisah dari lapisan *frontend* yang mengelola presentasi dan antarmuka pengguna (Hassan and Eassa, 2024). Pemisahan ini memungkinkan pengembangan, *deployment*, dan penskalaan komponen secara independen. Pendekatan *API-first* dalam arsitektur *headless* memungkinkan desain dan implementasi API sebelum pengembangan antarmuka pengguna, memfasilitasi integrasi yang lebih baik dengan berbagai klien dan meningkatkan fleksibilitas sistem (Velepucha and Flores, 2023). *Backend* Laravel menyediakan RESTful API yang dapat dikonsumsi oleh berbagai klien, sementara *frontend* Next.js bertanggung jawab sepenuhnya untuk rendering dan interaksi pengguna. Keunggulan arsitektur *headless* meliputi: (1) fleksibilitas teknologi, di mana *frontend* dan *backend* dapat dikembangkan dengan teknologi yang berbeda dan diperbarui secara independen; (2) *reusability* API, di mana satu *backend* dapat melayani berbagai klien seperti web, *mobile*, atau IoT; (3) peningkatan performa melalui optimasi dan *caching* yang lebih baik pada setiap lapisan; dan (4) kemudahan kolaborasi tim, di mana tim *frontend* dan *backend* dapat bekerja secara paralel (Strapi, 2026). Namun demikian, arsitektur *headless* juga memiliki tantangan seperti kompleksitas infrastruktur yang lebih tinggi, kebutuhan komunikasi API yang harus dikelola dengan baik, dan kurva pembelajaran yang

lebih curam untuk pengembang yang terbiasa dengan arsitektur monolitik tradisional.

7.2. Model Pengembangan Perangkat Lunak

Model pengembangan perangkat lunak adalah kerangka kerja terstruktur yang menjelaskan proses, kegiatan, dan tahapan yang dilalui dalam membangun sistem perangkat lunak dari konsepsi awal hingga *deployment* dan pemeliharaan (Sommerville, 2016). Model pengembangan menyediakan panduan sistematis untuk mengelola kompleksitas proyek perangkat lunak, memfasilitasi komunikasi antar anggota tim, mengalokasikan sumber daya secara efektif, dan memastikan bahwa produk akhir memenuhi kebutuhan pengguna. Menurut Pressman and Maxim (2014), pemilihan model pengembangan yang tepat sangat bergantung pada karakteristik proyek seperti ukuran, kompleksitas, tingkat kepastian kebutuhan, dan sumber daya yang tersedia.

Dalam penelitian ini, model *Waterfall* digunakan sebagai kerangka pengembangan sistem informasi organisasi INFINITE. Model *Waterfall* pertama kali diperkenalkan oleh Royce (1970) dan merupakan salah satu model pengembangan perangkat lunak tertua yang mengikuti pendekatan sekuensial dan linear. Dalam model ini, setiap tahap pengembangan harus diselesaikan secara penuh sebelum melanjutkan ke tahap berikutnya, dengan aliran progres yang mengalir ke bawah layaknya air terjun. Meskipun model *Waterfall* telah lama digunakan, penelitian terkini menunjukkan bahwa model ini tetap relevan untuk proyek dengan kebutuhan yang stabil dan terdefinisi dengan baik, terutama dalam konteks akademis dan organisasi dengan sumber daya terbatas (Saravanos and

Curinga, 2023). Studi komparatif menunjukkan bahwa model *Waterfall* cocok digunakan untuk proyek dengan kebutuhan yang stabil dan terdefinisi dengan baik sejak awal, seperti halnya sistem informasi organisasi yang memiliki struktur dan proses bisnis yang jelas (Akinsola et al., 2020; Balaji and Murugaiyan, 2012). Analisis metodologi SDLC kontemporer menegaskan bahwa pemilihan model pengembangan harus disesuaikan dengan karakteristik proyek, di mana *Waterfall* memberikan struktur yang jelas untuk proyek dengan kebutuhan yang telah terdokumentasi dengan baik (Jabangwe et al., 2018; Vallon et al., 2018).

Menurut Sommerville (2016), model *Waterfall* terdiri dari lima tahapan utama yang dilaksanakan secara berurutan:

1. ***Requirement Definition***: Tahap ini melibatkan pengumpulan dan dokumentasi kebutuhan sistem secara komprehensif melalui wawancara dengan pemangku kepentingan, observasi proses bisnis yang ada, dan analisis dokumen organisasi. Keluaran dari tahap ini adalah dokumen spesifikasi kebutuhan fungsional dan non-fungsional yang menjadi landasan untuk tahap selanjutnya. Dalam konteks sistem informasi INFINITE, tahap ini mencakup identifikasi kebutuhan pengelolaan data keanggotaan, kepengurusan, perlombaan, dan testimoni (Nuseibeh and Easterbrook, 2000).
2. ***System and Software Design***: Berdasarkan spesifikasi kebutuhan, tahap ini merancang arsitektur sistem secara menyeluruh, termasuk desain basis data, desain antarmuka pengguna, desain API, dan pemilihan teknologi. Perancangan dilakukan pada dua tingkat: *high-level design* yang

mendefinisikan arsitektur sistem secara keseluruhan, dan *detailed design* yang merinci setiap komponen dan modul. Untuk sistem INFINITE, tahap ini menghasilkan diagram *Entity-Relationship Diagram* (ERD), *wireframe* antarmuka, spesifikasi *endpoint* API, dan dokumentasi arsitektur *headless* (Bass et al., 2017).

3. ***Implementation and Unit Testing***: Tahap ini melibatkan penerjemahan desain sistem menjadi kode program yang dapat dieksekusi. Pengembang menulis konfigurasi basis data PostgreSQL, kode sumber untuk komponen *backend* menggunakan Laravel, serta komponen *frontend* menggunakan Next.js . Implementasi dilakukan dengan mengikuti praktik terbaik seperti konvensi penamaan yang konsisten, dokumentasi kode, dan penggunaan sistem kontrol versi Git (Foucault et al., 2019).
4. ***Integration and System Testing***: Setelah implementasi, sistem diuji secara menyeluruh untuk memastikan bahwa ia berfungsi sesuai dengan spesifikasi kebutuhan dan bebas dari kesalahan. Pengujian dilakukan pada berbagai tingkat: *unit testing* untuk menguji komponen individual, *integration testing* untuk menguji interaksi antar komponen, *system testing* untuk menguji sistem secara keseluruhan, dan *user acceptance testing* untuk validasi oleh pengguna akhir. Dalam penelitian ini, pengujian juga mencakup evaluasi kualitas perangkat lunak berdasarkan standar ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use* (Alyahya, 2020).
5. ***Operation and Maintenance***: Tahap akhir melibatkan pengoperasian sistem

di lingkungan produksi, pelatihan pengguna, serta pemeliharaan berkelanjutan untuk memperbaiki bug, melakukan pembaruan, dan menambahkan fitur baru sesuai kebutuhan. Otomatisasi *deployment* melalui GitHub Actions dan ArgoCD dengan GitOps memastikan proses rilis yang konsisten dan dapat diandalkan (Ebert et al., 2016).

Keunggulan model *Waterfall* meliputi: (1) struktur yang jelas dan mudah dipahami, memudahkan manajemen proyek dan estimasi waktu serta biaya; (2) dokumentasi yang komprehensif pada setiap tahap, memfasilitasi transfer pengetahuan dan pemeliharaan jangka panjang; dan (3) cocok untuk proyek dengan kebutuhan yang stabil dan tim yang kurang berpengalaman dengan metodologi *Agile*. Namun demikian, model ini juga memiliki keterbatasan, yaitu: (1) kurang fleksibel terhadap perubahan kebutuhan di tengah proyek; (2) pengguna baru bisa melihat hasil akhir setelah semua tahap selesai; dan (3) risiko menemukan masalah desain atau kebutuhan di tahap akhir yang mahal untuk diperbaiki. Meskipun demikian, untuk konteks sistem informasi organisasi mahasiswa dengan kebutuhan yang relatif terdefinisi dengan baik dan sumber daya yang terbatas, model *Waterfall* menawarkan pendekatan yang terstruktur dan dapat dikelola dengan baik (Akinsola et al., 2020; Petersen et al., 2015).

7.3. Pengujian Kualitas Perangkat Lunak

Pengujian kualitas perangkat lunak adalah proses sistematis untuk mengevaluasi atribut atau kemampuan suatu produk perangkat lunak guna menentukan apakah produk tersebut memenuhi kebutuhan yang ditetapkan dan sesuai dengan standar kualitas yang diharapkan (Alyahya, 2020). Menurut

Pressman and Maxim (2014), pengujian kualitas tidak hanya berfokus pada identifikasi kesalahan atau bug, tetapi juga pada verifikasi bahwa perangkat lunak memenuhi spesifikasi kebutuhan fungsional dan non-fungsional, serta validasi bahwa produk akhir sesuai dengan ekspektasi dan kebutuhan pengguna. Pengujian kualitas perangkat lunak memainkan peran kritis dalam memastikan keandalan, keamanan, dan kepuasan pengguna terhadap sistem yang dikembangkan.

International Organization for Standardization (ISO) adalah badan standardisasi internasional independen yang terdiri dari badan-badan standardisasi nasional dari berbagai negara. ISO mengembangkan dan menerbitkan standar internasional yang mencakup berbagai bidang, termasuk teknologi informasi dan rekayasa perangkat lunak. Standar ISO memberikan spesifikasi teknis, pedoman, dan karakteristik terbaik yang memastikan bahwa produk, layanan, dan sistem aman, dapat diandalkan, dan berkualitas tinggi (International Organization for Standardization, 2026). Dalam konteks kualitas perangkat lunak, ISO bekerja sama dengan International Electrotechnical Commission (IEC) untuk menghasilkan standar ISO/IEC yang diakui secara global.

ISO/IEC 25010:2023 adalah standar internasional yang merupakan bagian dari rangkaian standar *Systems and Software Quality Requirements and Evaluation* (SQuaRE) (International Organization for Standardization, 2023a). Standar ini memperbarui model kualitas produk dengan menambahkan karakteristik *safety*, mengganti *usability* dengan *interaction capability*, serta memisahkan model *quality-in-use* ke dalam standar tersendiri ISO/IEC 25019:2023 (International Organization for Standardization, 2023b). Standar ini menyediakan kerangka kerja

yang lebih komprehensif untuk mengevaluasi kualitas produk teknologi informasi dan komunikasi serta perangkat lunak. Penelitian menunjukkan bahwa penerapan ISO/IEC 25010 memerlukan metodologi pengujian yang sistematis untuk mengukur kecukupan informasi dalam penilaian kualitas perangkat lunak (Hovorushchenko, 2018). Model kualitas dalam ISO/IEC 25010:2023 dan ISO/IEC 25019:2023 menjelaskan dua standar model:

1. **Product Quality** (Kualitas Produk): Model ini mendefinisikan sembilan karakteristik kualitas yang dapat diukur secara internal maupun eksternal dari produk perangkat lunak. Karakteristik-karakteristik tersebut adalah: (1) *functional suitability* yang mengukur kelengkapan, keakuratan, dan kesesuaian fungsi; (2) *performance efficiency* yang mengukur performa dalam hal waktu, sumber daya, dan kapasitas; (3) *compatibility* yang mengukur kemampuan koeksistensi dan interoperabilitas dengan sistem lain; (4) *interaction capability* yang mengukur kemampuan interaksi pengguna dengan sistem untuk menyelesaikan tugas; (5) *reliability* yang mengukur ketersediaan, toleransi kesalahan, dan kemampuan pemulihan; (6) *security* yang mengukur kerahasiaan, integritas, dan akuntabilitas; (7) *maintainability* yang mengukur kemudahan modifikasi dan pengujian; (8) *flexibility* yang mengukur kemampuan adaptasi dan skalabilitas; serta (9) *safety* yang mengukur kemampuan produk untuk menghindari bahaya terhadap nyawa, kesehatan, dan lingkungan. Standar ISO/IEC 25010 telah diterapkan secara luas sebagai panduan untuk seleksi arsitektur perangkat lunak berdasarkan karakteristik kualitas yang diinginkan (Haoues et al., 2017). Penggunaan

metrik kualitas yang tepat sangat penting dalam proses evaluasi, dan penelitian terkini memberikan rekomendasi berbasis riset untuk pemilihan dan implementasi metrik kualitas perangkat lunak (Mladenova, 2020).

2. ***Quality in Use*** (Kualitas dalam Penggunaan): Model ini mengukur efek dan pengaruh pada pemangku kepentingan ketika produk digunakan dalam konteks penggunaan yang spesifik. Model ini terdiri dari tiga karakteristik utama: (1) *beneficialness* yang mencakup *usability* (efektivitas, efisiensi, dan kepuasan), *accessibility*, dan *suitability*; (2) *freedom from risk* yang mengukur mitigasi risiko ekonomi, kesehatan, lingkungan, dan risiko terhadap nyawa manusia; serta (3) *acceptability* yang mengukur keterimaan produk oleh pengguna dan pemangku kepentingan (Wagner et al., 2018).

Dalam konteks pengujian sistem informasi organisasi INFINITE, empat karakteristik dipilih sebagai kriteria utama berdasarkan relevansinya dengan tujuan dan sifat produk yang dikembangkan:

1. ***Functional Suitability*** (Kesesuaian Fungsional): Karakteristik ini mengukur sejauh mana perangkat lunak menyediakan fungsi-fungsi yang memenuhi kebutuhan yang dinyatakan dan tersirat ketika digunakan dalam kondisi tertentu. Karakteristik ini mencakup tiga sub-karakteristik: *functional completeness* yang mengukur kelengkapan fungsi untuk mencapai tujuan pengguna, *functional correctness* yang mengukur keakuratan hasil yang disediakan, dan *functional appropriateness* yang mengukur kesesuaian fungsi untuk tugas dan tujuan tertentu (Abana and Sy, 2022).
2. ***Performance Efficiency*** (Efisiensi Kinerja): Karakteristik ini mengukur

performa sistem dalam hal penggunaan waktu dan sumber daya ketika melakukan fungsi tertentu dalam kondisi yang ditetapkan. Sub-karakteristik meliputi *time behavior* yang mengukur waktu respons dan pemrosesan, *resource utilization* yang mengukur penggunaan CPU, memori, dan bandwidth, serta *capacity* yang mengukur batas maksimum parameter sistem (Kurniasari and Rochimah, 2025).

3. ***Interaction Capability*** (Kemampuan Interaksi): Karakteristik ini mengukur sejauh mana produk atau sistem dapat diinteraksi oleh pengguna tertentu untuk bertukar informasi antara pengguna dan produk atau sistem, melalui antarmuka pengguna, termasuk masukan dari pengguna dan keluaran dari produk atau sistem, untuk tujuan menyelesaikan tugas dalam berbagai konteks penggunaan. Sub-karakteristik meliputi *appropriateness*, *recognizability*, *learnability*, *operability*, *user error protection*, *user engagement*, *inclusivity*, *user assistance*, dan *self-descriptiveness* (Cayola and Macias, 2018).
4. ***Beneficialness*** (Kemanfaatan): Karakteristik ini merupakan bagian dari model *quality-in-use* dalam ISO/IEC 25019:2023, yang mengukur sejauh mana manfaat yang dihasilkan dari penggunaan produk atau sistem bagi pemangku kepentingan dalam konteks penggunaan yang spesifik. Karakteristik ini mencakup sub-karakteristik *usability* yang meliputi efektivitas, efisiensi, dan kepuasan; *accessibility*; serta *suitability*.

B. Hasil Penelitian yang Relevan

Berdasarkan kajian literatur yang telah dilakukan, terdapat beberapa hasil penelitian yang relevan dengan topik penelitian ini, di antaranya:

1. Penelitian oleh Pramitasari and Nurgiyatna (2019) berjudul *Sistem Informasi Unit Kegiatan Mahasiswa Marching Band Universitas Muhammadiyah Surakarta berbasis Web* bertujuan mengembangkan sistem informasi UKM berbasis web untuk mendukung penyebaran informasi, pengelolaan data kegiatan, serta inventaris organisasi. Penelitian tersebut menggunakan metode *Waterfall* dengan implementasi PHP dan MySQL. Hasil pengujian *black box* menunjukkan fitur sistem berjalan sesuai kebutuhan, dan evaluasi pengguna (37 responden) menunjukkan tingkat persetujuan sebesar 91,1% bahwa sistem membantu pengelolaan data dan penyebaran informasi UKM. Relevansi terhadap penelitian ini terletak pada kesamaan konteks organisasi kemahasiswaan dan tujuan digitalisasi proses administrasi organisasi.
2. Penelitian oleh Widjaja and Prasajo (2022) berjudul *Perancangan Sistem Informasi Unit Kegiatan Mahasiswa Universitas Nasional Karangturi Berbasis Web* berfokus pada perancangan sistem informasi UKM untuk mengoptimalkan pengelolaan aktivitas kemahasiswaan. Metode yang digunakan meliputi wawancara dan observasi untuk pengumpulan data, analisis PIECES, perancangan UML, serta pengembangan perangkat lunak dengan model *Waterfall*. Hasil penelitian menunjukkan sistem yang dirancang memberikan manfaat bagi mahasiswa, pengurus UKM, dan admin dalam menjalankan aktivitas organisasi. Penelitian ini relevan karena

menegaskan bahwa pendekatan *Waterfall* dan pemodelan terstruktur efektif untuk kebutuhan sistem informasi UKM di lingkungan perguruan tinggi.

3. Penelitian oleh Tofani and Sukya (2023) berjudul *Sistem Informasi Manajemen Kegiatan UKM English Club PSDKU Polinema di Kediri Berbasis Framework Laravel* mengembangkan sistem manajemen kegiatan UKM yang sebelumnya dikelola secara manual dan tidak terintegrasi. Sistem dibangun menggunakan PHP, Laravel, dan PostgreSQL dengan empat peran pengguna (DPK, ketua UKM, sekretaris UKM, dan admin kegiatan). Fitur utama mencakup pengelolaan program kerja, persetujuan program kerja, pengelolaan proposal kegiatan, serta LPJ kegiatan. Hasil pengujian menunjukkan seluruh fungsi utama berjalan sesuai harapan. Relevansi penelitian ini terhadap penelitian yang dilakukan terletak pada kesamaan penggunaan teknologi Laravel dan PostgreSQL serta fokus pada integrasi proses manajemen kegiatan organisasi mahasiswa. Namun, penelitian tersebut belum membahas pembaruan implementasi menuju *deployment* berbasis kontainer.
4. Penelitian oleh Feriawan et al. (2024) berjudul *Rancang Bangun Sistem Informasi Manajemen Transkrip Aktivitas Kemahasiswaan Primakara University Menggunakan Metode Extreme Programming* membahas pengembangan sistem informasi aktivitas kemahasiswaan berbasis web untuk menggantikan proses manual berbasis email yang tidak efisien. Sistem dikembangkan menggunakan Next.js dan JavaScript dengan metode *Extreme Programming* (XP), melalui tahapan perencanaan, perancangan UML,

implementasi, dan pengujian *black box*. Hasil penelitian menunjukkan sistem memenuhi kebutuhan fungsional dan mendukung pemantauan poin aktivitas secara *real-time*. Relevansi penelitian ini terletak pada kesamaan domain sistem informasi kemahasiswaan dan penggunaan teknologi *frontend* modern, sekaligus memberikan pembandingan metodologis terhadap model *Waterfall* yang digunakan pada penelitian ini. Meskipun demikian, penelitian tersebut belum menekankan orkestrasi layanan dan otomatisasi rilis berbasis kontainer.

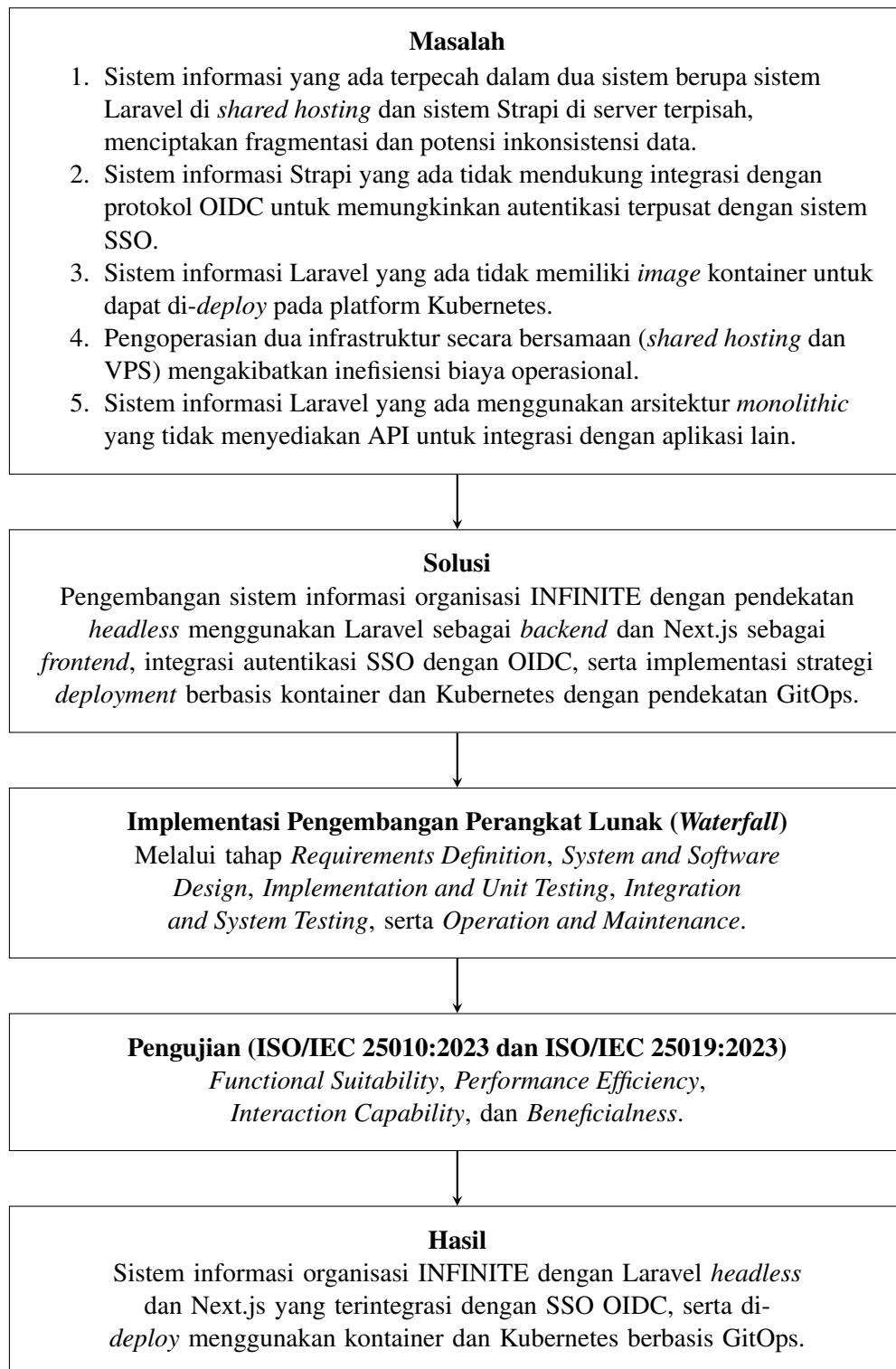
Berdasarkan keempat penelitian tersebut, dapat disimpulkan bahwa pengembangan sistem informasi organisasi kemahasiswaan berbasis web secara konsisten mampu meningkatkan efektivitas pengelolaan data, transparansi proses administrasi, dan kemudahan akses informasi bagi pemangku kepentingan. Namun, penelitian terdahulu umumnya masih berfokus pada pengembangan fitur dan pengujian fungsional, serta belum menempatkan strategi *deployment* modern sebagai fokus utama. Posisi penelitian ini berada pada penguatan integrasi pengelolaan organisasi INFINITE dengan arsitektur *headless* (Laravel sebagai *backend* dan Next.js sebagai *frontend*), pembaruan implementasi melalui kontainer dan Kubernetes berbasis GitOps, serta evaluasi kualitas perangkat lunak berbasis ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use* pada karakteristik yang relevan dengan kebutuhan pengguna.

C. Kerangka Pikir

Berikut ini adalah kerangka pikir yang mendasari penelitian pengembangan sistem informasi organisasi INFINITE dengan pendekatan Laravel *headless*,

Next.js, dan strategi *deployment* berbasis kontainer dan Kubernetes:

Gambar 2.1 Diagram Kerangka Pikir Penelitian



D. Pertanyaan Penelitian

1. Bagaimana tahapan *Requirements Definition* untuk mengembangkan sistem informasi organisasi INFINITE dengan pendekatan *headless* menggunakan Laravel dan Next.js yang terintegrasi autentikasi SSO dengan OIDC?
2. Bagaimana tahapan *System and Software Design* untuk mengembangkan sistem informasi organisasi INFINITE dengan pendekatan *headless* menggunakan Laravel dan Next.js yang terintegrasi autentikasi SSO dengan OIDC?
3. Bagaimana tahapan *Implementation and Unit Testing* untuk mengembangkan sistem informasi organisasi INFINITE dengan pendekatan *headless* menggunakan Laravel dan Next.js yang terintegrasi autentikasi SSO dengan OIDC?
4. Bagaimana tahapan *Integration and System Testing* untuk mengembangkan sistem informasi organisasi INFINITE dengan pendekatan *headless* menggunakan Laravel dan Next.js yang terintegrasi autentikasi SSO dengan OIDC?
5. Bagaimana hasil pengujian kualitas sistem informasi organisasi INFINITE berdasarkan aspek *functional suitability*?
6. Bagaimana hasil pengujian kualitas sistem informasi organisasi INFINITE berdasarkan aspek *performance efficiency*?
7. Bagaimana hasil pengujian kualitas sistem informasi organisasi INFINITE berdasarkan aspek *usability*?
8. Bagaimana hasil pengujian kualitas sistem informasi organisasi INFINITE

berdasarkan aspek *beneficialness*?

BAB III

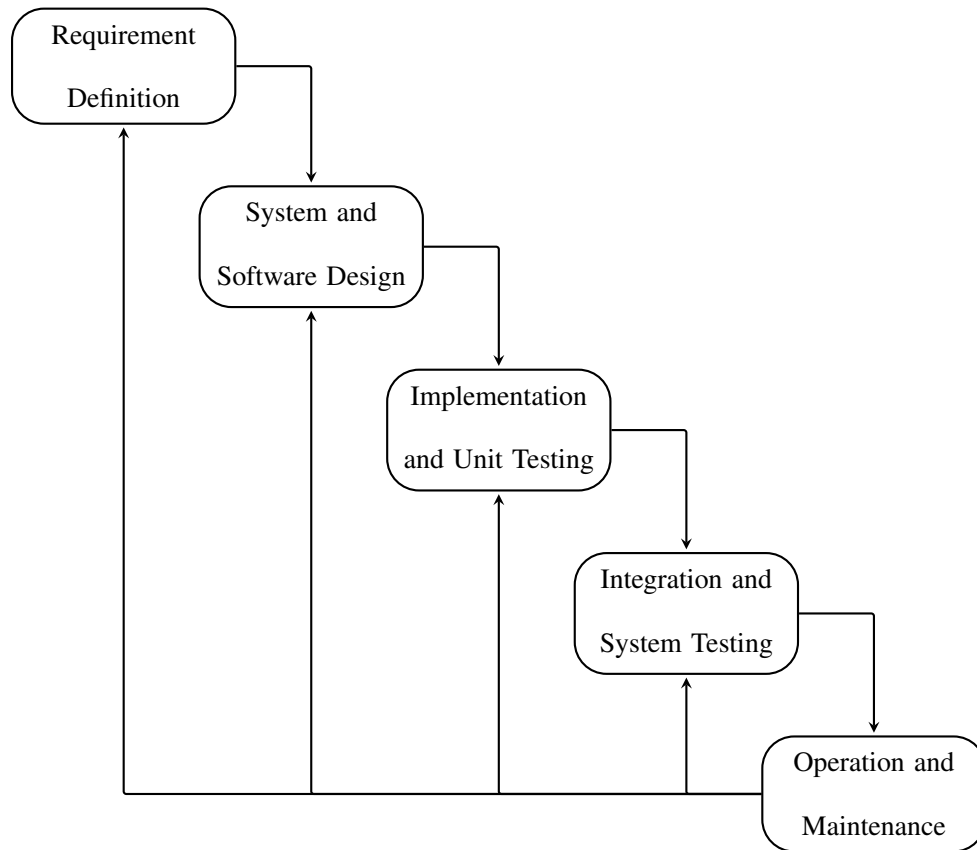
METODE PENELITIAN

A. Jenis Penelitian

Penelitian ini menggunakan metode *Research and Development* (R&D). Menurut Sugiyono (2013), metode R&D adalah suatu metode penelitian yang digunakan untuk menghasilkan produk tertentu dan menguji keefektifan produk tersebut. Metode ini dipilih karena tujuan utama penelitian ini adalah mengembangkan sebuah produk berupa sistem informasi organisasi INFINITE dengan *headless* Laravel, Next.js, dan kontainer. Sistem informasi ini memungkinkan pengelolaan data keanggotaan, kepengurusan, perlombaan, dan testimoni secara terintegrasi dengan arsitektur *headless* yang memisahkan *backend* API Laravel dari *frontend* Next.js.

Proses pengembangan produk dilaksanakan menggunakan model *Waterfall* sebagai siklus hidup pengembangan perangkat lunak, yang diperkenalkan pertama kali oleh Royce (1970) dan dikembangkan lebih lanjut dalam literatur rekayasa perangkat lunak modern (Pressman and Maxim, 2014; Sommerville, 2016). Model *Waterfall* dipilih dibandingkan model pengembangan lainnya seperti *Agile* karena memiliki alur kerja yang sistematis dan bertahap, serta cocok untuk proyek yang sudah didefinisikan dengan baik dan juga stabil (Haniva et al., 2023).

Gambar 3.1 Diagram Model *Waterfall*



Pada model ini, terdapat lima tahap yang dilaksanakan secara berurutan: (1) *Requirements Definition*, (2) *System and Software Design*, (3) *Implementation and Unit Testing*, (4) *Integration and System Testing*, dan (5) *Operation and Maintenance*, sebagaimana yang diilustrasikan pada gambar 3.1. Masing-masing tahap menghasilkan artefak yang menjadi masukan bagi tahap berikutnya, sehingga tidak memungkinkan melewati salah satu tahap langsung ke tahap selanjutnya.

Untuk penelitian ini, fokus utama adalah pada tahap pertama hingga tahap keempat, yaitu dari *Requirements Definition* hingga *Integration and System Testing*. Tahap *Operation and Maintenance* tidak termasuk dalam ruang lingkup penelitian ini dan akan dilakukan secara berkelanjutan oleh pengurus INFINITE

setelah produk sistem informasi organisasi INFINITE selesai dikembangkan dan diujikan.

B. Metode Penelitian

1. *Requirements Definition*

Tahap *requirements definition* bertujuan untuk mengidentifikasi dan mendokumentasikan kebutuhan fungsional maupun non-fungsional dari sistem informasi organisasi INFINITE. Pada tahap ini, digunakan dilakukan wawancara kepada pengurus dan anggota INFINITE untuk mengumpulkan kebutuhan dari perspektif pengguna. Selain itu, dilakukan juga observasi terhadap sistem informasi organisasi yang telah ada sebelumnya untuk memahami kondisi sistem dan tantangan yang dihadapi oleh pengurus dan anggota INFINITE dalam mengelola data organisasi. Hasil dari tahap ini ditujukan sebagai dasar untuk menentukan spesifikasi kebutuhan sistem informasi INFINITE yang akan dikembangkan, serta untuk memastikan bahwa produk yang dikembangkan sesuai dengan kebutuhan dan harapan pengguna.

2. *System and Software Design*

Berdasarkan informasi kebutuhan spesifikasi yang telah diidentifikasi pada tahap sebelumnya, dilakukan tahap *System and Software Design* untuk menerjemahkan kebutuhan tersebut menjadi desain dan spesifikasi teknis sistem informasi INFINITE. Proses desain dilakukan dengan pendekatan *top-down*, dimulai dari perancangan desain sistem keseluruhan hingga desain antarmuka. Aktivitas desain mencakup 3 aspek utama:

- a. Desain arsitektur sistem: Desain arsitektur sistem dilakukan dengan

memanfaatkan *Unified Modeling Language* (UML) yang meliputi *use case diagram*, *class diagram*, dan *sequence diagram*. Desain ini mencakup *backend* Laravel dan *frontend* Next.js.

- b. Desain basis data: Desain skema basis data PostgreSQL dilakukan untuk menentukan struktur dan hubungan antar tabel yang akan digunakan dalam menyimpan data keanggotaan, kepengurusan, perlombaan, dan testimoni. Desain ini meliputi entitas, atribut, dan relasi antar tabel untuk memastikan integritas data dan efisiensi penyimpanan.
- c. Desain antarmuka: Desain antarmuka dilakukan untuk merancang interaksi pengguna dengan sistem informasi INFINITE. Desain ini mencakup desain halaman-halaman *frontend* Next.js. Desain yang baik akan memastikan pengalaman pengguna yang intuitif dan efisien dalam mengelola data organisasi, serta memastikan konsistensi dan estetika dalam tampilan antarmuka pengguna.

3. *Implementation and Unit Testing*

Pada tahap *implementation and unit testing* ini, desain pada tahap sebelumnya direalisasikan menjadi kode program sistem informasi yang fungsional. Implementasi *backend* sistem informasi INFINITE pada penelitian ini dilakukan menggunakan kerangka kerja Laravel dengan bahasa pemrograman PHP dan basis data PostgreSQL. Untuk *frontend*, digunakan kerangka kerja Next.js dengan bahasa pemrograman TypeScript. Implementasi mengikuti prinsip *Clean Architecture* untuk memastikan modularitas dan kemudahan pemeliharaan kode. Aplikasi dikemas dalam bentuk kontainer untuk memudahkan *deployment*.

4. *Integration and System Testing*

Setelah tahap implementasi selesai, dilakukan pengujian sistem informasi INFINITE untuk memastikan bahwa produk yang dikembangkan memenuhi kebutuhan dan harapan pengguna. Pengujian dilakukan dengan beberapa jenis metode untuk menguji setiap aspek sistem untuk memastikan pemenuhan kebutuhan yang telah ditetapkan. Pengujian dilakukan berdasarkan standar kualitas ISO/IEC 25010:2023 untuk aspek *Product Quality* dan ISO/IEC 25019:2023 untuk aspek *Quality in Use*, dengan pemfokusan pada karakteristik *Functional Suitability*, *Interaction Capability*, *Performance Efficiency*, dan *Beneficialness* yang relevan dengan produk yang dikembangkan.

C. *Tempat dan Waktu Penelitian*

Penelitian dilakukan di Fakultas Teknik, Universitas Negeri Yogyakarta serta secara daring. Pengembangan sistem informasi INFINITE dan pengujian dilaksanakan menggunakan infrastruktur komputasi awan dan kluster Kubernetes yang dapat diakses secara jarak jauh. Tahapan penelitian secara keseluruhan dilaksanakan dalam jangka waktu bulan Maret–Mei tahun 2026, dengan jadwal yang dipetakan terhadap tahapan model *Waterfall* sebagaimana yang dapat diamati pada tabel 3.1.

Tabel 3.1 Jadwal Pelaksanaan Penelitian

Kegiatan	Maret				April				Mei			
	1	2	3	4	1	2	3	4	1	2	3	4
<i>Requirements Definition</i>	•	•										
<i>System and Software Design</i>		•	•	•								
<i>Implementation and Unit Testing</i>				•	•	•	•	•				
<i>Integration and System Testing</i>								•	•	•		
Penyusunan Laporan Tugas Akhir	•	•	•	•	•	•	•	•	•	•	•	•

D. Subjek Penelitian

Subjek penelitian ini terdiri dari tenaga ahli yang berperan sebagai validator dan evaluator untuk memastikan kualitas sistem informasi organisasi INFINITE. Selain itu, subjek penelitian juga mencakup pengguna sistem, yaitu pengurus dan anggota INFINITE. Pengujian aspek *Functional Suitability* dan *Interaction Capability* yang termasuk dalam *Product Quality* dilakukan kepada lima tenaga ahli dalam bidang sistem informasi. Sedangkan pengujian aspek *Beneficialness* yang termasuk dalam *Quality in Use* dilakukan terhadap pengguna sistem, yaitu berupa sembilan pengurus dan sembilan anggota INFINITE. Untuk aspek *Performance Efficiency*, akan dilakukan pengujian menggunakan alat pengujian khusus untuk mengukur performa sistem informasi INFINITE dalam berbagai skenario penggunaan.

E. Teknik Pengumpulan Data

Teknik pengumpulan data yang digunakan dalam penelitian pengembangan sistem informasi organisasi INFINITE ini adalah sebagai berikut:

1. Wawancara

Wawancara dilakukan secara mendalam dengan beberapa pengurus INFINITE untuk mendapatkan pemahaman yang lebih komprehensif tentang kebutuhan, tantangan, dan harapan mereka terkait pengembangan sistem informasi INFINITE. Proses wawancara dilakukan secara daring menggunakan platform komunikasi pertemuan video. Wawancara dilakukan terutama dengan Presiden INFINITE serta Ketua Divisi *Research and Development* yang memiliki pengetahuan mendalam tentang kebutuhan organisasi dan bertanggung jawab atas pengembangan dan pengelolaan sistem informasi organisasi.

2. Observasi

Observasi dilakukan untuk memahami sistem informasi organisasi yang telah ada sebelumnya, serta untuk mengidentifikasi kebutuhan dan tantangan yang dihadapi oleh pengurus dan anggota INFINITE dalam mengelola data organisasi. Selain itu observasi juga dilakukan dalam pengambilan data untuk pengujian aspek *performance efficiency* dengan menggunakan alat pengujian khusus.

3. Kuesioner

Kuesioner digunakan untuk mengumpulkan data dari para ahli terkait validasi dan evaluasi setelah pengembangan produk pada aspek *functional suitability*, *interaction capability*, dan *beneficialness*. Kuesioner dirancang dengan menggunakan skala Likert untuk menilai tingkat kesesuaian.

F. Instrumen Penelitian

Instrumen penelitian yang digunakan dalam penelitian ini dirancang untuk mengukur aspek-aspek kualitas sistem informasi organisasi INFINITE berdasarkan standar ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use*. Instrumen ini mencakup:

1. Instrumen Pengujian *Functional Suitability*

Pengujian aspek *Functional Suitability* bertujuan untuk mengukur sejauh mana sistem informasi INFINITE memenuhi kebutuhan fungsional yang telah ditetapkan. Pengujian ini dilakukan dengan menggunakan skala Guttman yang memiliki dua pilihan jawaban, yaitu “Ya” dan “Tidak”. Skala Guttman dipilih karena pengujian fungsional bersifat deterministik, yaitu setiap fungsi dapat dinilai apakah berjalan sesuai spesifikasi atau tidak (Lamada et al., 2020).

Instrumen pengujian *Functional Suitability* disusun berdasarkan spesifikasi kebutuhan fungsional yang telah didefinisikan pada tahap *Requirements Definition*. Setiap butir instrumen merepresentasikan satu fungsi atau fitur sistem yang akan diuji oleh ahli. Instrumen ini akan divalidasi terlebih dahulu oleh ahli sebelum digunakan untuk pengujian. Kisi-kisi instrumen pengujian *Functional Suitability* disajikan pada lampiran 1.

2. Instrumen Pengujian *Interaction Capability*

Pengujian aspek *Interaction Capability* dilakukan untuk mengukur kemudahan penggunaan sistem informasi INFINITE oleh pengguna. Instrumen yang digunakan adalah *USE Questionnaire* yang dikembangkan oleh Lund (2001).

USE Questionnaire dipilih karena merupakan instrumen yang telah tervalidasi dan banyak digunakan dalam penelitian pengembangan sistem informasi untuk mengukur aspek kegunaan, kemudahan penggunaan, kemudahan pembelajaran, dan kepuasan pengguna (Ghaffur, 2017; Jayanto, 2017).

USE Questionnaire terdiri dari empat dimensi utama, yaitu:

- a. *Usefulness*: Mengukur sejauh mana sistem membantu pengguna dalam menyelesaikan tugas dan mencapai tujuan.
- b. *Ease of Use*: Mengukur kemudahan pengguna dalam mengoperasikan sistem.
- c. *Ease of Learning*: Mengukur kemudahan pengguna dalam mempelajari cara menggunakan sistem.
- d. *Satisfaction*: Mengukur tingkat kepuasan pengguna terhadap pengalaman menggunakan sistem.

Instrumen ini menggunakan skala Likert 5 poin dengan rentang 1 (Sangat Tidak Setuju) hingga 5 (Sangat Setuju). Kisi-kisi instrumen pengujian *Interaction Capability* disajikan pada lampiran 1.

3. Instrumen Pengujian *Performance Efficiency*

Pengujian aspek *Performance Efficiency* bertujuan untuk mengukur performa sistem informasi INFINITE dalam merespons permintaan pengguna dan memproses data. Pengujian ini dilakukan dengan menggunakan dua alat yang berbeda untuk mengukur performa *frontend* dan *backend*.

3.1. Pengujian Performa *Frontend*

Pengujian performa *frontend* Next.js dilakukan menggunakan Lighthouse, yaitu alat pengujian otomatis yang dikembangkan oleh Google untuk mengukur

kualitas halaman web (Google, 2026a). Lighthouse mengukur metrik *Core Web Vitals* yang merupakan standar industri untuk mengukur pengalaman pengguna pada halaman web (Google, 2026b). Metrik yang diukur meliputi:

- a. *First Contentful Paint* (FCP): Waktu yang dibutuhkan untuk menampilkan konten pertama pada halaman.
- b. *Speed Index* (SI): Kecepatan visual halaman dalam menampilkan konten.
- c. *Largest Contentful Paint* (LCP): Waktu yang dibutuhkan untuk menampilkan elemen konten terbesar pada halaman.
- d. *Total Blocking Time* (TBT): Total waktu di mana *main thread* terblokir dan tidak dapat merespons input pengguna.
- e. *Cumulative Layout Shift* (CLS): Stabilitas visual halaman selama proses pemuatan.

3.2. Pengujian Performa Backend

Pengujian performa *backend* API Laravel dilakukan menggunakan Grafana k6, yaitu alat pengujian beban (*load testing*) yang dirancang untuk menguji performa API (Grafana Labs, 2026). Pengujian dilakukan dengan tipe *breakpoint test* untuk menemukan titik maksimum performa API, yaitu jumlah *concurrent users* dan *requests per second* maksimal yang dapat ditangani oleh sistem sebelum terjadi degradasi performa atau kegagalan. Metrik yang diukur meliputi:

- a. *Response Time*: Waktu respons API untuk setiap permintaan.
- b. *Throughput*: Jumlah permintaan yang dapat diproses per satuan waktu.
- c. *Error Rate*: Persentase permintaan yang gagal.
- d. *Virtual Users*: Jumlah pengguna virtual simultan yang disimulasikan.

4. Instrumen Pengujian *Beneficialness*

Pengujian aspek *Beneficialness* bertujuan untuk mengukur sejauh mana sistem informasi INFINITE memberikan kemanfaatan bagi pengguna akhir, yang dinilai dari tingkat kepuasan pengguna terhadap sistem. Instrumen yang digunakan adalah *End-User Computing Satisfaction* (EUCS) yang dikembangkan oleh Doll and Torkzadeh (1988). EUCS dipilih karena merupakan instrumen yang telah tervalidasi secara luas dan dirancang khusus untuk mengukur kepuasan pengguna terhadap sistem informasi berbasis komputer (Hidayah et al., 2020).

EUCS terdiri dari lima dimensi utama, yaitu:

- a. *Content*: Mengukur kepuasan pengguna terhadap isi informasi yang disajikan oleh sistem, termasuk kelengkapan, relevansi, dan keakuratan informasi.
- b. *Accuracy*: Mengukur kepuasan pengguna terhadap ketepatan dan kebenaran data yang diproses oleh sistem.
- c. *Format*: Mengukur kepuasan pengguna terhadap tampilan dan penyajian informasi, termasuk tata letak dan desain antarmuka.
- d. *Ease of Use*: Mengukur kepuasan pengguna terhadap kemudahan dalam menggunakan sistem untuk menyelesaikan tugas.
- e. *Timeliness*: Mengukur kepuasan pengguna terhadap kecepatan sistem dalam menyajikan informasi yang dibutuhkan.

Instrumen ini menggunakan skala Likert 5 poin dengan rentang 1 (Sangat Tidak Puas) hingga 5 (Sangat Puas). Kisi-kisi instrumen pengujian *Beneficialness* disajikan pada lampiran 1.

G. Teknik Analisis Data

1. Analisis Pengujian *Functional Suitability*

Analisis data pengujian aspek *Functional Suitability* dilakukan dengan menghitung persentase keberhasilan fungsi sistem berdasarkan hasil pengujian menggunakan skala Guttman. Setiap fungsi yang diuji akan dinilai dengan dua kemungkinan jawaban, yaitu “Ya” (bernilai 1) jika fungsi berjalan sesuai spesifikasi dan “Tidak” (bernilai 0) jika fungsi tidak berjalan sesuai spesifikasi (Lamada et al., 2020).

Perhitungan persentase kelayakan aspek *Functional Suitability* dilakukan dengan menggunakan rumus berikut (Lamada et al., 2020):

$$\text{Persentase Kelayakan} = \frac{\text{Skor yang diperoleh}}{\text{Skor maksimal}} \times 100\% \quad 3.1$$

Hasil perhitungan persentase kelayakan kemudian diinterpretasikan berdasarkan kriteria kelayakan yang disajikan pada tabel 3.2.

Tabel 3.2 Kriteria Interpretasi Kelayakan *Functional Suitability*

Persentase	Kriteria
81% – 100%	Sangat Layak
61% – 80%	Layak
41% – 60%	Cukup Layak
21% – 40%	Kurang Layak
0% – 20%	Tidak Layak

2. Analisis Pengujian *Interaction Capability* dan *Beneficialness*

Analisis data pengujian aspek *Interaction Capability* dan *Beneficialness* dilakukan dengan menghitung skor dari hasil kuesioner yang keduanya menggunakan skala Likert 5 poin. Aspek *Interaction Capability* diukur menggunakan *USE Questionnaire*, sedangkan aspek *Beneficialness* diukur menggunakan *End-User Computing Satisfaction* (EUCS) yang mengukur kepuasan pengguna melalui lima dimensi: *Content*, *Accuracy*, *Format*, *Ease of Use*, dan *Timeliness* (Doll and Torkzadeh, 1988; Hidayah et al., 2020).

Perhitungan persentase kelayakan untuk kedua aspek dilakukan dengan menggunakan rumus berikut (Ghaffur, 2017):

$$\text{Persentase Kelayakan} = \frac{\text{Skor yang diperoleh}}{\text{Skor maksimal}} \times 100\% \quad 3.2$$

Hasil perhitungan persentase kelayakan kemudian diinterpretasikan berdasarkan kriteria yang disajikan pada tabel 3.3.

Tabel 3.3 Kriteria Interpretasi Kelayakan *Interaction Capability* dan *Beneficialness*

Persentase	Kriteria
81% – 100%	Sangat Layak
61% – 80%	Layak
41% – 60%	Cukup Layak
21% – 40%	Kurang Layak
0% – 20%	Tidak Layak

3. Analisis Pengujian *Performance Efficiency*

Analisis data pengujian aspek *Performance Efficiency* dilakukan secara terpisah untuk *frontend* dan *backend* menggunakan metrik dan kriteria yang sesuai dengan masing-masing komponen sistem.

3.1. Analisis Performa *Frontend*

Analisis performa *frontend* Next.js dilakukan berdasarkan hasil pengujian menggunakan Lighthouse. Lighthouse menghasilkan skor performa dengan rentang 0–100 yang dihitung berdasarkan rata-rata tertimbang dari metrik *Core Web Vitals* (Google, 2026a). Bobot masing-masing metrik pada Lighthouse versi 10 adalah sebagai berikut:

- a. *First Contentful Paint* (FCP): 10%
- b. *Speed Index* (SI): 10%
- c. *Largest Contentful Paint* (LCP): 25%
- d. *Total Blocking Time* (TBT): 30%
- e. *Cumulative Layout Shift* (CLS): 25%

Hasil skor performa Lighthouse diinterpretasikan berdasarkan kriteria yang disajikan pada tabel 3.4.

Tabel 3.4 Kriteria Interpretasi Skor Lighthouse

Skor	Kriteria
90 – 100	Baik (<i>Good</i>)
50 – 89	Perlu Perbaikan (<i>Needs Improvement</i>)
0 – 49	Buruk (<i>Poor</i>)

3.2. Analisis Performa *Backend*

Analisis performa *backend* API Laravel dilakukan berdasarkan hasil pengujian menggunakan Grafana k6 dengan tipe *breakpoint test*. *Breakpoint test* bertujuan untuk menemukan batas kapasitas sistem dengan meningkatkan beban secara bertahap hingga sistem mengalami degradasi performa atau kegagalan (Grafana Labs, 2026). Metrik yang dianalisis meliputi:

- a. *Response Time*: Waktu respons rata-rata API untuk setiap permintaan.
- b. *Throughput*: Jumlah permintaan yang dapat diproses per detik (*requests per second*).
- c. *Error Rate*: Persentase permintaan yang gagal.
- d. *Virtual Users*: Jumlah pengguna virtual maksimum yang dapat ditangani sistem secara simultan sebelum terjadi degradasi.

BAB IV

HASIL PENELITIAN DAN PEMBAHASAN

A. Hasil Penelitian

1. Pengembangan Sistem Informasi

Pengembangan sistem informasi organisasi INFINITE dilakukan dengan menggunakan model pengembangan perangkat lunak *Waterfall*, dengan tahapan sebagai berikut:

1.1. *Requirements Definition*

Pada tahap pendefinisian kebutuhan, dilakukan analisis kebutuhan melalui wawancara dengan Presiden INFINITE dan Ketua Divisi *Research and Development*, serta observasi terhadap sistem informasi organisasi yang telah ada sebelumnya. Dari kegiatan tersebut, diperoleh informasi penting sebagai berikut:

1. Sistem informasi organisasi INFINITE sebelumnya terdiri dari dua sistem terpisah, yaitu sistem berbasis Laravel dan sistem berbasis CMS Strapi. Kedua sistem ini tidak terintegrasi, sehingga data tersebar di dua basis data yang berbeda.
2. Sistem sebelumnya tidak memiliki autentikasi terpusat, sehingga pengguna harus melakukan pendaftaran dan masuk secara terpisah di masing-masing sistem.
3. Sistem sebelumnya tidak memiliki manajemen peran dan izin akses yang memadai, sehingga kontrol akses terhadap fitur-fitur sistem tidak konsisten.
4. Sistem berbasis Laravel sebelumnya memiliki fitur pengelolaan data anggota, kompetisi, tim, prestasi, dan pengajuan pendanaan, namun tidak memiliki

fitur manajemen konten seperti testimoni dan galeri proyek, serta manajemen struktur pengurus organisasi.

5. Sistem berbasis CMS Strapi sebelumnya memiliki fitur manajemen konten dan struktur pengurus organisasi, namun tidak memiliki fitur pengelolaan data anggota dan tidak terintegrasi dengan sistem berbasis Laravel.

Hasil analisis kebutuhan kemudian digunakan sebagai dasar penentuan spesifikasi sistem informasi INFINITE.

1. Analisis Kebutuhan Fungsional

Berdasarkan hasil wawancara dan observasi, kebutuhan fungsional sistem informasi organisasi INFINITE mencakup fitur-fitur utama yang diperlukan untuk mendukung pengelolaan data organisasi secara terintegrasi. Sistem informasi terdiri dari dua komponen utama yang saling terintegrasi, yaitu *backend* API Laravel dan *frontend* Next.js. Sistem ini mendukung autentikasi terpusat melalui protokol *OpenID Connect* (OIDC) dengan sistem *Single Sign-On* (SSO) INFINITE. Fitur-fitur yang dikembangkan meliputi:

- a. Pengelolaan data anggota, meliputi data profil personal (nama, NIM, strata, fakultas, program studi), data kontak (telepon, email), serta data keanggotaan (persona, tanggal mulai dan berakhir keanggotaan, status aktif, dan status istimewa).
- b. Pengelolaan data kepengurusan, meliputi pengurus inti organisasi beserta divisi dan anggotanya, serta pengurus komunitas beserta anggotanya.

- c. Pengelolaan data kompetisi, meliputi katalog kompetisi, pelaksanaan kompetisi spesifik, tipe penyelenggara, tingkat kompetisi, peringkat, durasi kompetisi, jenis luaran, serta tipe tim.
- d. Pengelolaan data tim perlombaan, meliputi pembentukan tim oleh anggota, penunjukan ketua tim, serta keanggotaan tim.
- e. Pengelolaan pengajuan pendanaan oleh tim, meliputi pengunggahan dokumen bukti lolos kompetisi dan proposal pendanaan, dengan alur persetujuan.
- f. Pencatatan prestasi yang diraih oleh tim, dengan perhitungan poin berdasarkan pembobotan multi-dimensi dari enam taksonomi kompetisi.
- g. Papan peringkat pencapaian anggota yang dihitung secara agregat per tahun berdasarkan total poin prestasi.
- h. Pengelolaan testimoni dari alumni tentang dampak INFINITE terhadap karier mereka, serta testimoni proyek yang pernah dikerjakan oleh anggota.
- i. Pengelolaan galeri proyek yang mendokumentasikan proyek-proyek yang telah dikerjakan oleh anggota.
- j. Pengelolaan data referensi akademik berupa strata, fakultas, dan program studi.
- k. Manajemen peran dan izin akses untuk mengatur hak akses pengguna terhadap setiap fitur sistem.
- l. Pengelolaan konfigurasi sistem berupa pengaturan kunci-nilai untuk

parameter seperti periode registrasi ulang, tautan media sosial, dan pengaturan lainnya.

- m. Penyediaan API untuk integrasi dan konsumsi oleh *frontend*, *landing page*, maupun sistem eksternal.

2. Analisis Kebutuhan Non-Fungsional

Kebutuhan non-fungsional sistem informasi organisasi INFINITE mencakup kebutuhan lingkungan pengembangan dan spesifikasi sistem sebagai berikut:

a. Kebutuhan Informasi Penunjang Sistem

- Konfigurasi sistem SSO INFINITE berbasis Authentik sebagai *identity provider*.
- Konfigurasi basis data PostgreSQL dan Redis Sentinel sebagai *data store* untuk sistem.

b. Kebutuhan Spesifikasi Sistem

- Sistem dikembangkan dengan arsitektur *headless*, memisahkan *backend* API Laravel dari *frontend* Next.js yang dihubungkan melalui antarmuka RESTful API.
- *Backend* menggunakan bahasa pemrograman PHP dengan kerangka kerja Laravel, basis data PostgreSQL, dan *server* aplikasi Laravel Octane dengan FrankenPHP.
- *Frontend* menggunakan TypeScript dengan kerangka kerja Next.js (*App Router*), Material UI (MUI) sebagai pustaka komponen, Tailwind CSS untuk utilitas gaya, Zustand untuk manajemen *state*,

dan Inversify untuk *dependency injection*.

- Sistem di-*deploy* sebagai aplikasi terkontainerisasi di platform Kubernetes dengan pendekatan GitOps menggunakan ArgoCD.
- Autentikasi menggunakan OIDC melalui sistem SSO INFINITE (Authentik) yang terintegrasi dengan *library* better-auth pada *frontend*.
- Seluruh operasi CRUD tersedia melalui RESTful API berversi (/api/v1/) dengan *cursor-based pagination*, penyaringan (*filtering*), pengurutan (*sorting*), dan pemuatan relasi (*eager-loading*) menggunakan Spatie Query Builder.
- Sistem mendukung penyimpanan berkas (*blob*) dengan pola manifest JSON, mendukung berkas publik dan privat dengan *signed URL*.
- Sinkronisasi data pengguna dengan SSO dilakukan secara asinkron melalui *job queue* dengan dukungan Redis Sentinel untuk *high availability*.

1.2. *System and Software Design*

Berdasarkan kebutuhan yang telah didefinisikan, dilakukan perancangan sistem yang meliputi arsitektur sistem, desain basis data, dan desain antarmuka.

1. Arsitektur Sistem

Perancangan arsitektur sistem dilakukan dengan memanfaatkan *Unified Modeling Language* (UML) yang meliputi diagram-diagram berikut:

a. *Use Case Diagram*

Use Case Diagram menggambarkan interaksi antara aktor pengguna dengan fitur-fitur sistem. Dalam sistem informasi organisasi INFINITE, secara umum tidak ada penentuan aktor berdasarkan peran dalam sistem. Ragam jenis aktor dapat ditambahkan ataupun dikurangi sesuai kebutuhan oleh pengurus organisasi. Setiap aktor dapat memiliki hak akses yang berbeda terhadap fitur-fitur sistem manapun yang diatur secara dinamis melalui mekanisme peran dan hak akses (*Role-Based Access Control*).

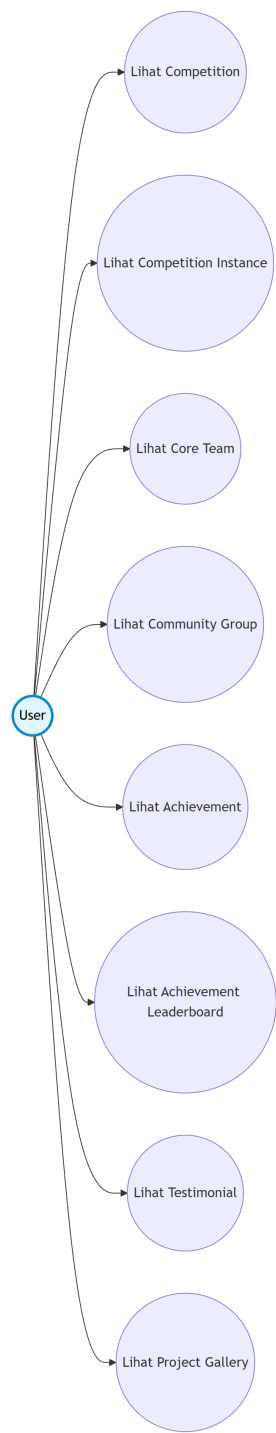
Pembagian jenis aktor yang ada adalah aktor publik dan aktor terautentikasi. Aktor publik dapat melihat data sistem yang memang diperuntukkan untuk informasi publik, seperti yang terlihat pada gambar 4.1. Sedangkan aktor terautentikasi dapat mengakses seluruh fitur sistem sesuai dengan hak akses yang dimilikinya untuk menambahkan, mengubah, atau menghapus data sistem, seperti yang dapat dilihat pada lampiran 5. Deskripsi mengenai fungsi setiap *use case* dapat dilihat pada lampiran 6.

b. *Class Diagram*

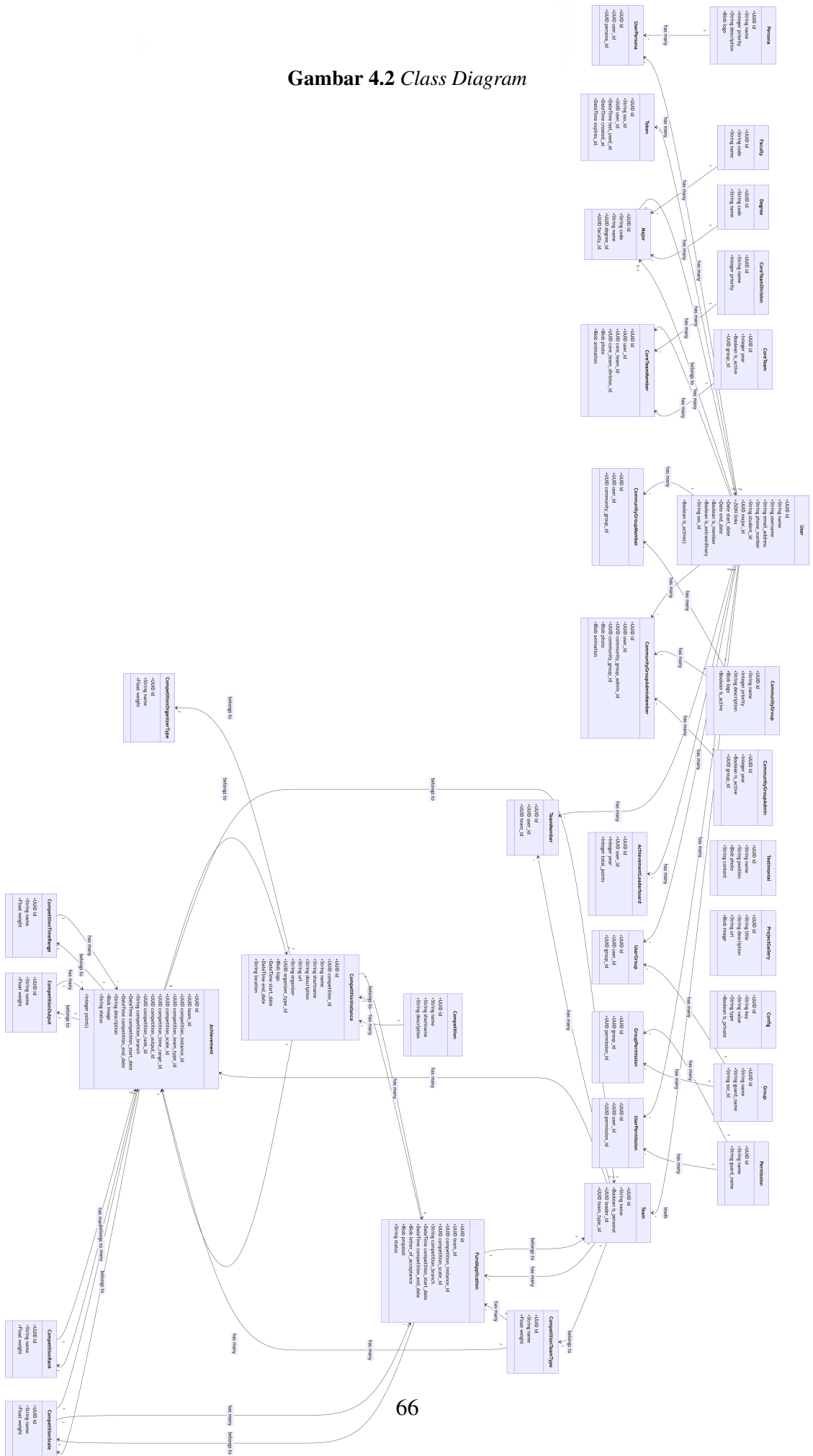
Class Diagram merepresentasikan struktur entitas-entitas dalam sistem beserta atribut, metode, dan relasi antar entitas, sebagaimana yang terlihat pada gambar 4.2. Sistem informasi organisasi INFINITE memiliki 35 entitas, yang dapat dikelompokkan ke dalam beberapa domain utama:

- Domain Otorisasi: Group, Permission, UserGroup,

Gambar 4.1 *Use Case Diagram* Aktor Publik



Gambar 4.2 *Class Diagram*



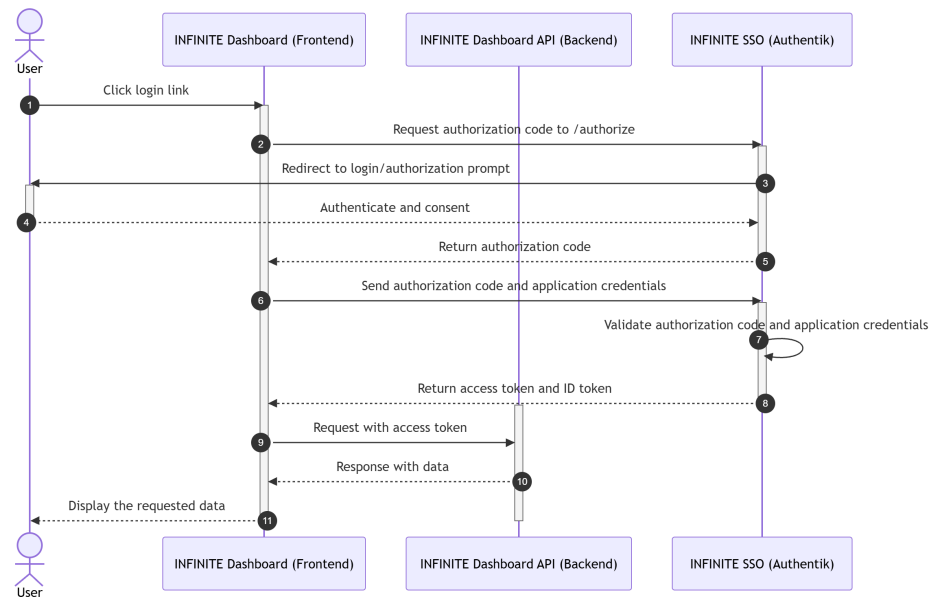
GroupPermission, UserPermission.

- **Domain Pengguna:** User, Persona, UserPersona, Token.
- **Domain Akademik:** Degree, Faculty, Major.
- **Domain Organisasi:** CoreTeam, CoreTeamDivision, CoreTeamMember, CommunityGroup, CommunityGroupAdmin, CommunityGroupMember, CommunityGroupAdminMember.
- **Domain Kompetisi:** Competition, CompetitionInstance, CompetitionOrganizerType, CompetitionScale, CompetitionRank, CompetitionTimeRange, CompetitionOutput, CompetitionTeamType.
- **Domain Pendanaan dan Prestasi Tim:** Team, TeamMember, FundApplication, Achievement, AchievementLeaderboard.
- **Domain Konten:** Testimonial, ProjectGallery.
- **Domain Konfigurasi:** Config.

Entitas User memiliki atribut terkomputasi `is_active` yang didapatkan dari perbandingan tanggal saat ini dengan tanggal `end_date` yang menandai akhir masa aktif anggota. Selain itu, entitas Achievement juga memiliki atribut terkomputasi `point` yang dihitung dari perkalian bobot enam taksonomi kompetisi (`organizer_type × team_type × scale × time_range × output × rank`), yang selanjutnya diakumulasi ke dalam AchievementLeaderboard per pengguna per tahun.

c. *Sequence Diagram*

Gambar 4.3 *Sequence Diagram* Alur Autentikasi OIDC

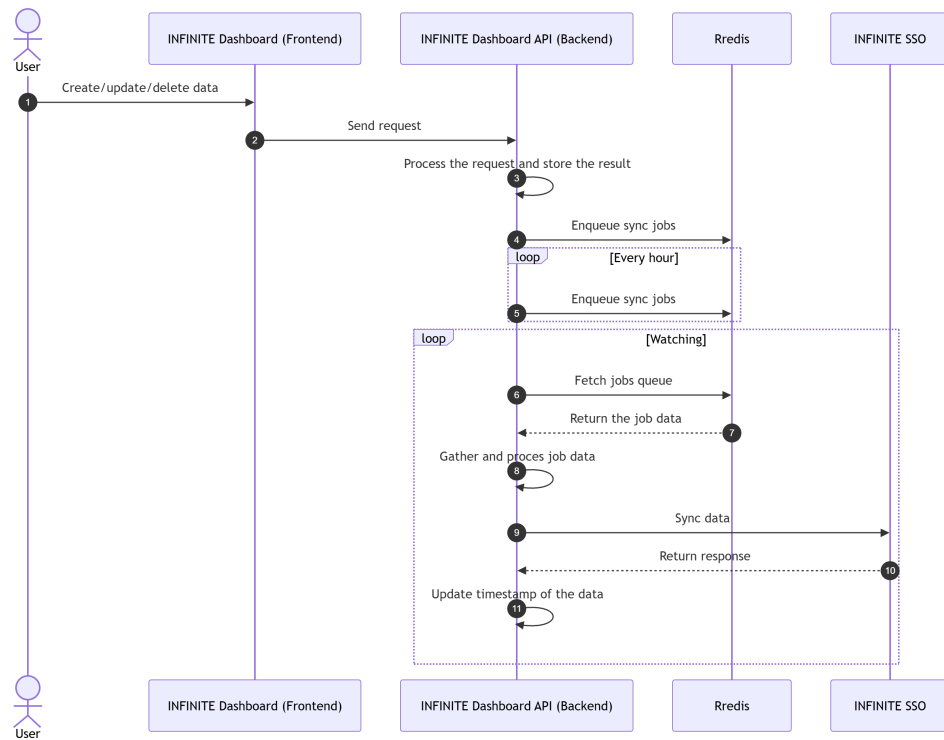


Sequence Diagram menggambarkan alur interaksi antar objek dalam sistem berdasarkan urutan waktu. Diagram ini membantu memahami mekanisme kerja dan proses transaksi data yang terjadi dalam sistem. Beberapa *sequence diagram* utama yang dirancang meliputi: alur autentikasi OIDC (mulai dari *redirect* ke SSO hingga pembuatan sesi pengguna), alur pengelolaan keseluruhan data sistem secara umum (*CRUD*), dan alur sinkronisasi data pengguna dan hak akses/otorisasi ke sistem SSO. *Sequence diagram* yang dirancang disajikan pada gambar 4.3, gambar 4.4, dan gambar 4.5.

2. Desain Basis Data

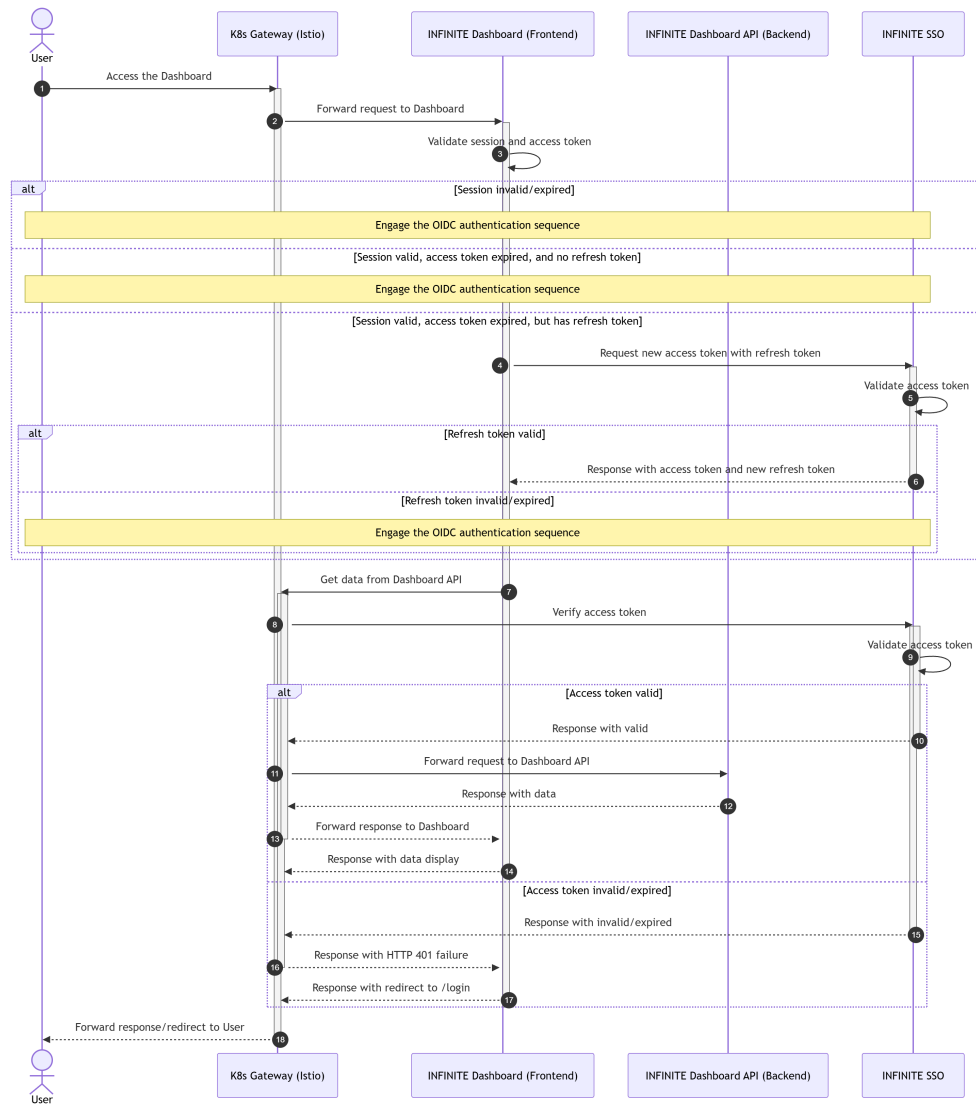
Desain basis data dirancang menggunakan dokumen migrasi dari Laravel dibantu dengan *software* DataGrip untuk menghasilkan 35 tabel basis data. Perancangan basis data mempertimbangkan integritas referensial, efisiensi kueri, dan skalabilitas. Semua tabel menggunakan *primary key*

Gambar 4.4 *Sequence Diagram* Alur Sinkronisasi Data Pengguna dan Hak Akses ke SSO

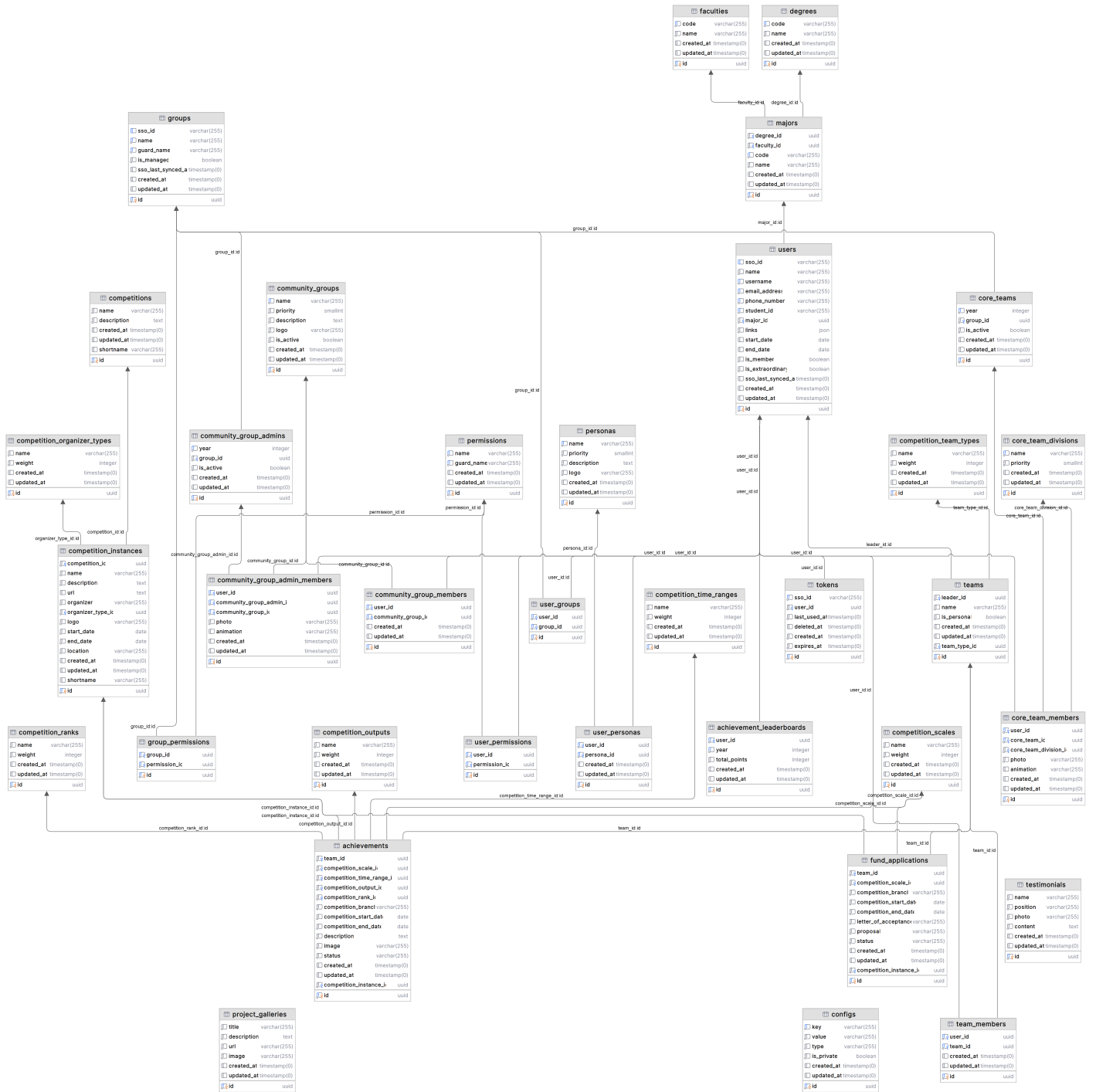


berbasis UUID (*Universally Unique Identifier*) versi 7 untuk menjamin keunikan global dan keamanan namun tetap mendukung pengurutan secara kronologis maupun leksikografis. Relasi antar tabel meliputi relasi satu-ke-banyak (*one-to-many*) seperti User ke Achievement, relasi banyak-ke-banyak (*many-to-many*) seperti User ke CoreTeam melalui CoreTeamMember, serta relasi hierarkis seperti Major ke Degree dan Faculty.

Gambar 4.5 *Sequence Diagram* Alur Pengelolaan Data Sistem (CRUD)



Gambar 4.6 Desain Basis Data



3. Desain Antarmuka

Desain antarmuka *frontend* dirancang menggunakan *software* Figma.

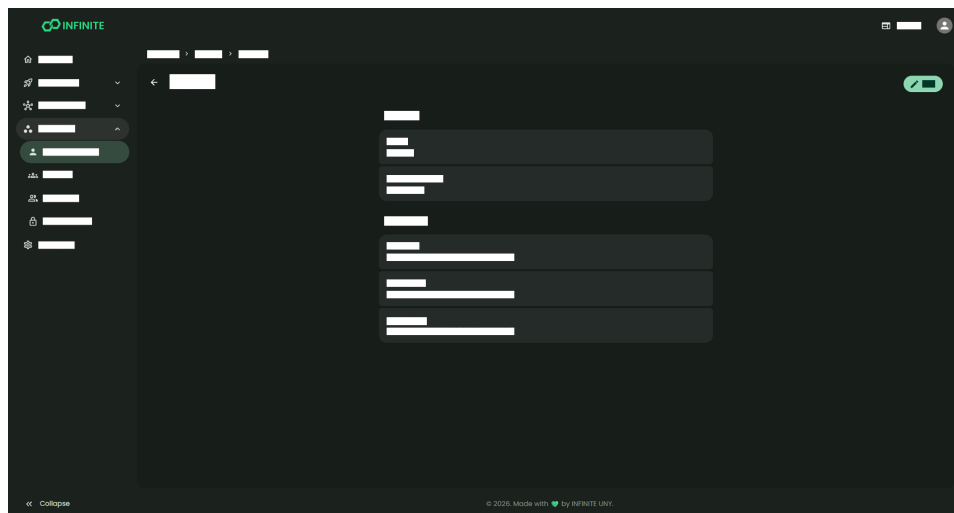
Secara garis besar, terdapat 3 jenis desain antarmuka halaman yang dibuat,

yaitu halaman list data seperti pada gambar 4.7, halaman detail seperti pada gambar 4.8, dan halaman formulir sunting seperti pada gambar 4.9. Antarmuka dirancang mengikuti prinsip *responsive design* dengan navigasi berbasis menu atas (*navbar*) dan menu samping (*sidebar*) yang dapat dilipat.

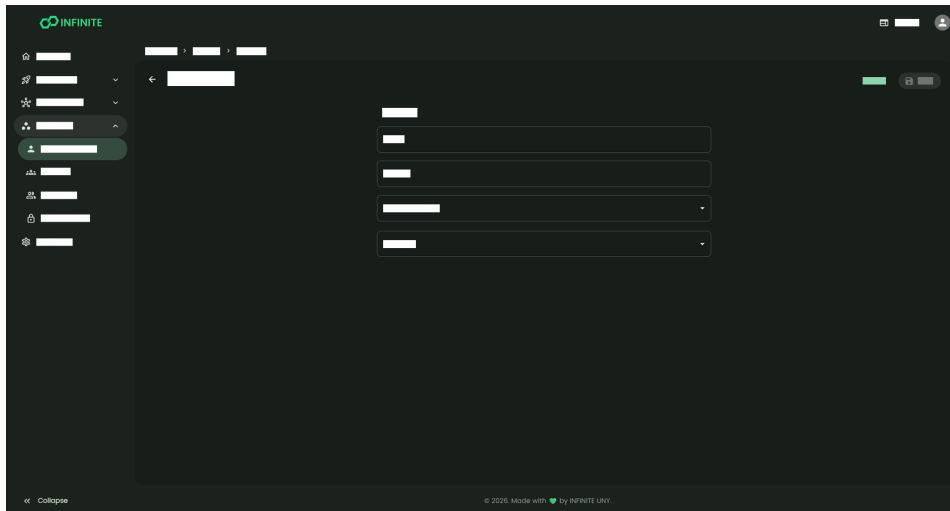
Gambar 4.7 Desain Halaman List Data



Gambar 4.8 Desain Halaman Detail



Gambar 4.9 Desain Halaman Formulir Sunting



1.3. *Implementation and Unit Testing*

Pada tahap ini, desain yang telah dibuat direalisasikan menjadi kode program yang fungsional. Implementasi dilakukan secara bertahap mulai dari basis data, *backend*, hingga *frontend*.

1. Implementasi Basis Data

Implementasi basis data dilakukan pada PostgreSQL melalui mekanisme kode program migrasi dari Laravel. Total terdapat 33 berkas kode program migrasi yang mendefinisikan struktur tabel-tabel beserta relasi, indeks, dan *constraint*. Salah satu contoh kode program migrasi untuk tabel `users` dapat dilihat pada lampiran 7. Setelah berkas migrasi didefinisikan, dilakukan eksekusi migrasi untuk membuat tabel-tabel di basis data PostgreSQL secara otomatis dengan perintah `php artisan migrate`.

Selanjutnya, dilakukan implementasi pengisian data awal (*seed*) untuk tabel-tabel referensi seperti data terkait akademik (Degree, Faculty, dan Major), kompetisi (CompetitionOrganizerType, CompetitionOutput,

CompetitionRank, CompetitionScale, CompetitionTimeRange, dan CompetitionTeamType), organisasi (CoreTeamDivision dan CommunityGroup), profil pengguna (Persona), serta otorisasi (Permission dan Group) melalui berkas kode program *seeder* Laravel. Salah satu contoh kode program *seeder* untuk data groups dapat dilihat pada lampiran 8. Eksekusi *seeder* dilakukan dengan perintah `php artisan db:seed` untuk mengisi data awal ke dalam tabel-tabel basis data yang telah dibuat sebelumnya.

2. Implementasi *Backend*

Backend sistem informasi organisasi INFINITE dikembangkan menggunakan *framework* Laravel dengan bahasa pemrograman PHP. Arsitektur *backend* mengikuti konsep MVC (*Model-View-Controller*) yang diadaptasi untuk pendekatan *headless*, di mana komponen *View* tidak menghasilkan HTML melainkan respons JSON melalui *API Resources*. Penerapan arsitektur pada *backend* adalah sebagai berikut:

a. *Model*

Sebanyak 35 *model* diimplementasikan untuk merepresentasikan tabel basis data. Akses *query* ke basis data tidak dilakukan secara manual, melainkan melalui Eloquent ORM bawaan Laravel. Setiap *model* menggunakan *trait* `HasUuids` untuk *primary key* UUID dan menerapkan metode `serializeDate()` dengan format `DATE_ATOM` (ISO 8601) untuk konsistensi serialisasi tanggal. Selain itu, setiap *model* juga mendeklarasikan fungsi relasi antar *model* seperti

`hasMany()`, `belongsTo()`, dan `belongsToMany()` untuk mempermudah pemanggilan relasi. Contoh kode program dapat dilihat pada lampiran 9

b. *View*

View diimplementasikan menggunakan *API Resource* Laravel yang mengubah *model* menjadi format JSON. Setiap *model* memiliki *resource* dan *resource collection*. Format JSON mengikuti konvensi JSend yang dimodifikasi, dengan struktur `{status, data, message}`. *Resource* dan *resource collection* dibuat secara *custom* dan universal mengikuti format tersebut dengan melakukan ekstensi terhadap `JsonResource` dan `ResourceCollection` bawaan Laravel, seperti pada lampiran 10 dan lampiran 11. Setiap *model* kemudian melakukan ekstensi terhadap *custom resource* dan *custom resource collection* tersebut untuk menghasilkan format JSON yang konsisten. Contoh implementasi *resource* dan *resource collection* untuk *model* *User* dapat dilihat pada lampiran 12 dan lampiran 13.

c. *Controller*

Controller API diorganisasikan dalam direktori `app/Http/Controllers/Api/V1/` dengan versi API konsisten pada prefix `/v1/`. Setiap controller menggunakan `Spatie QueryBuilder` dengan daftar eksplisit untuk kolom yang boleh disertakan (`allowedIncludes`), disaring (`allowedFilters`), dan diurutkan (`allowedSorts`). Seluruh *endpoint* index menggunakan

`cursorPaginate()` untuk paginasi berbasis kursor. Otorisasi diterapkan melalui middleware `can` dan `authorizeResource` yang merujuk konfigurasi *policy* untuk setiap *model*. Contoh kode program dapat dilihat pada lampiran 14

Selain implementasi MVC, *backend* juga mengimplementasikan beberapa pola desain dan mekanisme tambahan untuk mendukung kebutuhan sistem:

a. ***Services dan Repositories***

Lapisan layanan (*service*) dan repositori (*repository*) diimplementasikan dengan pola *interface-implementation*. *Interface* didefinisikan untuk setiap layanan dan repositori sebagai "kontrak" atau panduan, kemudian *implementation* konkrit diregister sebagai *singleton* di `AppServiceProvider`. Layanan dan repositori utama meliputi:

- `OidcService/OidcServiceImpl`: menangani verifikasi token JWT OIDC, introspeksi token, dan pengambilan informasi pengguna dari *identity provider*. Kode program dapat dilihat pada lampiran 15
- `SsoService/SsoServiceImpl`: menangani operasi CRUD terhadap API Authentik SSO untuk sinkronisasi pengguna dan otorisasi. Kode program dapat dilihat pada lampiran 16
- `StorageRepository/StorageRepositoryImpl`: abstraksi penyimpanan berkas dengan pelacakan manifest JSON. Berkas

lama dihapus secara asinkron melalui *job* DeleteBlob. Kode program dapat dilihat pada lampiran 17

b. Autentikasi *Custom*

Sistem autentikasi *backend* mengimplementasikan tiga *guard custom*:

- *OidcTokenGuard*: autentikasi berbasis token Bearer yang memverifikasi tanda tangan JWT melalui OIDC, mencari pengguna berdasarkan *sso_id* dengan *fallback* ke pencocokan email, serta melacak token di basis data. Kode program dapat dilihat pada lampiran 18
- *OidcHeadersGuard*: autentikasi berbasis header untuk mendukung konfigurasi *reverse proxy* yang menangani OIDC di lapisan *proxy*. Kode program dapat dilihat pada lampiran 19
- *DummyGuard*: *guard* khusus pengembangan/pengujian yang mengautentikasi sebagai pengguna *dummy*.

c. *Job* Asinkron

Sebanyak 20 *job* asinkron dengan antrean diimplementasikan untuk kalkulasi poin prestasi, penghapusan berkas lama, serta menjaga sinkronisasi data antara sistem *backend* dan sistem SSO INFENTE, meliputi pembuatan, pembaruan, dan penghapusan pengguna SSO; sinkronisasi massal pengguna; sinkronisasi keanggotaan grup, pengurus, dan administrator. Antrean didukung oleh Redis Sentinel untuk ketersediaan tinggi. Contoh kode program dapat dilihat pada

lampiran 20

d. Penyimpanan Berkas dengan *Object Storage*

Penyimpanan berkas memanfaatkan *object storage* untuk menyimpan berkas secara efisien dan skalabel. Informasi berkas disimpan dalam manifest JSON yang menyimpan informasi disk, visibilitas, jalur, dan nama berkas ke dalam kolom basis data. *Cast Blob* mengubah manifest menjadi URL saat pembacaan. Berkas privat akan menghasilkan *signed URL* dengan masa berlaku terbatas. Kode program *cast* dapat dilihat pada lampiran 9

3. Implementasi *Frontend*

Frontend sistem informasi organisasi INFINITE dikembangkan menggunakan Next.js 16 dengan *App Router* dan TypeScript. Arsitektur *frontend* menerapkan prinsip *Clean Architecture* dengan empat lapisan utama yang dipisahkan secara tegas:

a. *Domain*

Merupakan lapisan paling dalam yang berisi aturan bisnis inti tanpa dependensi ke *framework* eksternal. Lapisan ini terdiri dari:

- Entitas sebanyak 39 kelas yang merepresentasikan objek bisnis seperti *User*, *Team*, *Achievement*, *Competition*, *CoreTeam*, *FundApplication*, dan lainnya. Contoh kode program dapat dilihat pada lampiran 22
- Antarmuka repositori sebanyak 37 berkas yang mendefinisikan kontrak operasi CRUD untuk setiap entitas. Contoh kode program

dapat dilihat pada lampiran 23

- Kelas kesalahan HTTP dengan kelas dasar `HttpError` dan turunan seperti `BadRequestError`, `UnauthorizedError`, `ForbiddenError`, `NotFoundError`, `ValidationError`, dan `InternalServerError`. Contoh kode program dapat dilihat pada lampiran 24

b. *Application*

Berisi *use case* sebanyak 165 berkas, masing-masing merupakan kelas `@injectable()` dengan metode `execute()`. Setiap *use case* hanya bergantung pada antarmuka repositori di lapisan *domain*. Sebagian besar *use case* yang memerlukan autentikasi terlebih dahulu memanggil `AuthRepository.getAccessToken()` sebelum meneruskan token ke repositori yang sesuai. Contoh kode program dapat dilihat pada lampiran 25

c. *Infrastructure*

Berisi implementasi konkret dari antarmuka yang didefinisikan di lapisan *domain*:

- *Data sources* berupa klien Axios untuk komunikasi API *backend*, konfigurasi autentikasi server dan klien dengan *better-auth*, serta penyimpanan sesi dan lokal. Contoh kode program dapat dilihat pada lampiran 26
- *Data Transfer Object* (DTO) dan *mapper* sebanyak 37 berkas yang menyediakan konversi dua arah antara format *snake_case* API dan

format *camelCase domain* menggunakan Luxon untuk penanganan tanggal. Contoh kode program dapat dilihat pada lampiran 27

- Implementasi repositori sebanyak 37 berkas yang menerima `InfinityApiDataSource` melalui injeksi dependensi dan melakukan pemanggilan REST API, memetakan kueri untuk penyaringan, pengurutan, dan paginasi berbasis kursor, serta membungkus kesalahan dengan `handleAxiosError()`. Setiap operasi repositori mengembalikan `Effect.Either<T, Error>` untuk penanganan kesalahan. Contoh kode program dapat dilihat pada lampiran 28

d. *Presentation*

Berisi komponen untuk antarmuka pengguna yang diorganisasikan ke dalam:

- *Components* yang teridisi dari komponen bersama, komponen autentikasi berisi formulir login dan komponen bersama terkait halaman autentikasi, serta komponen internal berisi 72 direktori komponen untuk setiap entitas, terdiri dari tampilan daftar, tampilan detail, formulir, dan bilah alat. Komponen bersama meliputi `Header`, `Footer`, `Sidebar`, `Navbar`, `AlertDialog`, `PDFViewer`, `operator DataGrid`, dan lainnya.
- *Controllers* yang berisi `AuthController` untuk menjembatani `better-auth` ke *route handlers* `Next.js`.
- *Hooks* yang berisi `useInternalStore` sebagai *hook* `Zustand` untuk

Gambar 4.10 Tampilan Halaman List Data

Name	Student ID	Major	Faculty	Start Date	End Date	Member	Active	
Alfin Hendawan Santosa	24050130098	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Dhyan Sigh Dasgupta	24050130074	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Andhika Khudhri Widianto	24050130094	Teknik Industri	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Amelia Anggrani Putri	24050130098	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Nur Kharisya Tama Faradita	24050130092	Teknik Industri	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Danendra Alifan Nurul	24050130098	Teknik Industri	Fakultas Teknik	04/11/2025	30/06/2027	✓	✓	⋮
Dianisari Arikeswari	25050130054	Teknologi Informasi	Fakultas Teknik	04/11/2025	30/06/2027	✓	✓	⋮
Hani Muzakkar	25050130080	Pendidikan Teknik Mekatronika	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Muhammad Jusdikan	24050130080	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Aprilia Candra Pujipta	24050130025	Teknologi Informasi	Fakultas Teknik	04/11/2025	30/06/2027	✓	✓	⋮
Gabariela Krishna Daryal	25050130036	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Nadar Rathi	25050130096	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Laili Singsa	24050130030	Pendidikan Akutansi	Fakultas Ekonomi dan Bisnis	04/11/2025	04/11/2025	✓	✓	⋮
Akmalia Edwan Ghani	24050130036	Teknologi Informasi	Fakultas Teknik	04/11/2025	04/11/2025	✓	✓	⋮
Muhammad Raka Andria Wicak	25050130045	Pendidikan Teknik Mekatronika	Fakultas Teknik	04/11/2025	30/06/2027	✓	✓	⋮

Gambar 4.11 Tampilan Halaman Detail

Overview

Programs

Communities

Directories

Users

Teams

Groups

Permissions

Settings

Overview

Users

Muhammad Naufal Khoirul Imamilhaq Alhifdi

← Muhammad Naufal Khoirul Imamilhaq Alhifdi

Edit

General

Name

Muhammad Naufal Khoirul Imamilhaq Alhifdi

Username

kakak204

Student ID

2205150001

Faculty

Fakultas Teknik

Major

Geometri - Teknologi Informasi

Contacts

Email Address

muhammad7777777777@student.uny.ac.id

Phone Number

6298930489121

LinkedIn

kakak204

Membership

Member

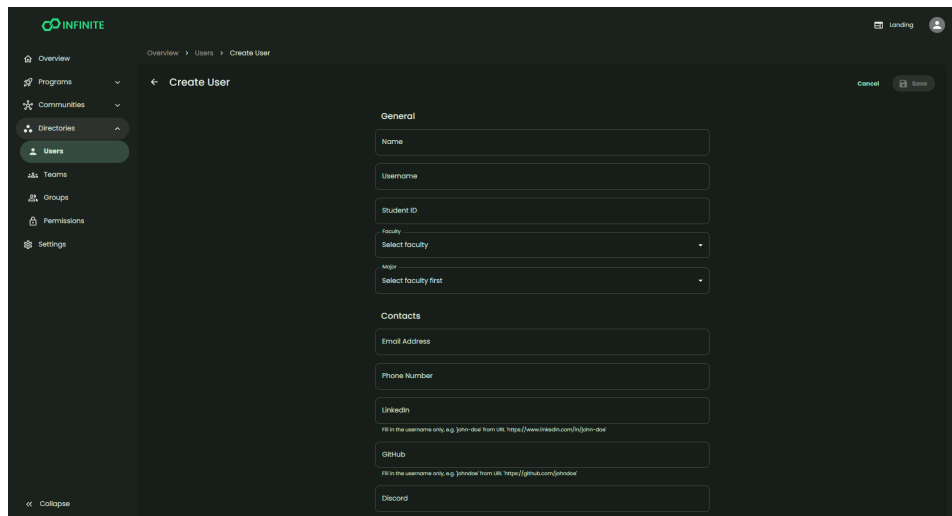
Yes

menyediakan *state* internal sebagai konteks React.

- *Stores* yang berisi manajemen *state* Zustand dengan *middleware* Immer untuk pembaruan *immutable*.

Injeksi dependensi diimplementasikan menggunakan Inversify v8 dengan dekorator `@injectable()` dan injeksi konstruktor `@inject(SYMBOLS.Xxx)`. Terdapat 215 simbol yang didefinisikan di `config/symbols.ts` untuk injeksi dependensi. Dibuat juga 2 kontainer

Gambar 4.12 Tampilan Halaman Formulir Sunting

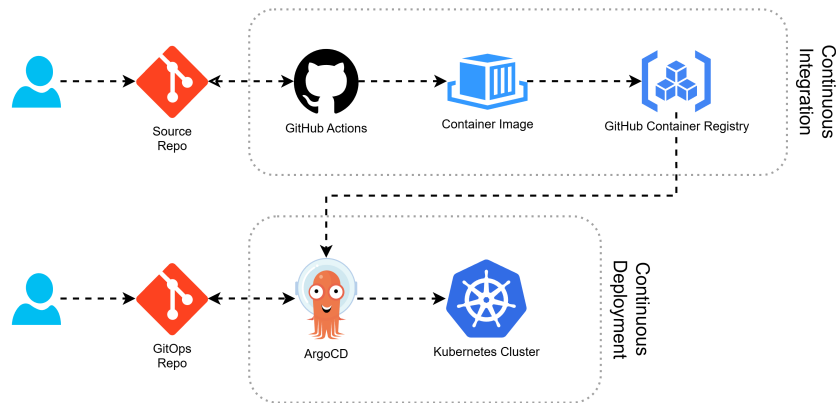
The image shows a web application interface for INFINITE. On the left is a dark sidebar with a navigation menu containing 'Overview', 'Programs', 'Communities', 'Directories', 'Users' (highlighted), 'Teams', 'Groups', 'Permissions', and 'Settings'. The main content area is titled 'Create User' and contains a form with two sections: 'General' and 'Contacts'. The 'General' section includes fields for 'Name', 'Username', 'Student ID', 'Faculty' (a dropdown menu), and 'Major' (a dropdown menu with the instruction 'Select faculty first'). The 'Contacts' section includes fields for 'Email Address', 'Phone Number', 'LinkedIn' (with a hint: 'Fill in the username only, e.g. jhon-doe from url: https://www.linkedin.com/in/jhon-doe/'), 'GitHub' (with a hint: 'Fill in the username only, e.g. jhon-doe from url: https://github.com/jhon-doe/'), and 'Discord'. At the top right of the main area are 'Cancel' and 'Save' buttons. The bottom left of the sidebar has a 'Collapse' button.

untuk melakukan *binding* dependensi, yaitu `src/server-injection.ts` untuk sisi server (menggunakan `AuthServerDataSource` dan `InfinityApiDataSource`), serta `src/client-injection.ts` untuk sisi klien (menggunakan `AuthClientDataSource`, `InfinityApiDataSource`, dan `SessionStorageDataSource`).

Konfigurasi *routing* Next.js membagi rute ke dalam tiga grup: (`auth`) untuk alur autentikasi, (`internal`) untuk dasbor internal yang terautentikasi, dan (`public`) untuk rute publik seperti pemeriksaan kesehatan sistem. Konfigurasi `proxy.ts` dari Next.js diatur untuk memeriksa sesi pada setiap permintaan dan mengarahkan pengguna yang belum terautentikasi ke halaman masuk.

Autentikasi *frontend* menggunakan *library* `better-auth v1.6` dengan plugin `genericOAuth` untuk integrasi dengan SSO INFINITE berbasis protokol OIDC. Sesi disimpan di Redis Sentinel untuk ketersediaan tinggi, dengan masa berlaku sesi 2 jam yang diperbarui setiap 10 menit. Plugin sesi

Gambar 4.13 CI/CD dengan GitHub Actions dan ArgoCD



kustom menambahkan `internalId`, `username`, dan `permissions` ke objek sesi pengguna untuk keperluan kontrol akses di antarmuka.

1.4. *Integration and System Testing*

Setelah seluruh komponen *backend* dan *frontend* diimplementasikan, dilakukan integrasi sistem dan pengujian kualitas secara menyeluruh.

1. Integrasi Sistem dengan Kubernetes Berbasis GitOps

Sistem informasi organisasi INFINITE diintegrasikan dan di-*deploy* sebagai aplikasi terkontainerisasi di platform Kubernetes. Proses integrasi mencakup:

- a. *Packaging backend* dan *frontend* ke dalam *image* kontainer terpisah menggunakan *multi-stage build*. *Image backend* dibangun dengan FrankenPHP serta Supervisor untuk manajemen proses, dengan berbagai mode kontainer seperti *server*, *horizon*, dan *scheduler*. *Image frontend* dibangun dengan konfigurasi mode keluaran *standalone* Next.js dan dijalankan menggunakan *runtime* Bun. Kode program dapat dilihat pada lampiran 29 dan lampiran 30.

- b. *Pipeline* CI menggunakan GitHub Actions yang mencakup tahapan validasi *branch* dan versi, pembuatan dokumentasi API otomatis dengan Scribe, pembuatan *changelog* melalui *conventional commits*, pembangunan *image* kontainer, penandatanganan *image* dengan Cosign, dan rilis ke GitHub Container Registry. Kedua *image* dibangun untuk multi-arsitektur (`linux/amd64` dan `linux/arm64`). Kode program *pipeline* dapat dilihat pada lampiran 31 dan lampiran 32.
- c. *Deployment* di Kubernetes dikelola menggunakan pendekatan GitOps dengan ArgoCD, seperti yang dapat dilihat pada lampiran 33. Repositori Git berfungsi sebagai sumber kebenaran tunggal untuk seluruh konfigurasi infrastruktur deklaratif. ArgoCD secara kontinu memantau repositori Git dan menyinkronkan kluster Kubernetes agar sesuai dengan konfigurasi yang didefinisikan, termasuk deteksi penyimpangan (*drift*) dan *rollback* otomatis.

2. Pengujian Kualitas Sistem

Pengujian kualitas sistem informasi organisasi INFINITE dilakukan berdasarkan standar ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use*, dengan empat karakteristik utama yang dipilih, yaitu *Functional Suitability*, *Interaction Capability*, *Performance Efficiency*, dan *Beneficialness*.

2. Cara Kerja Sistem Informasi

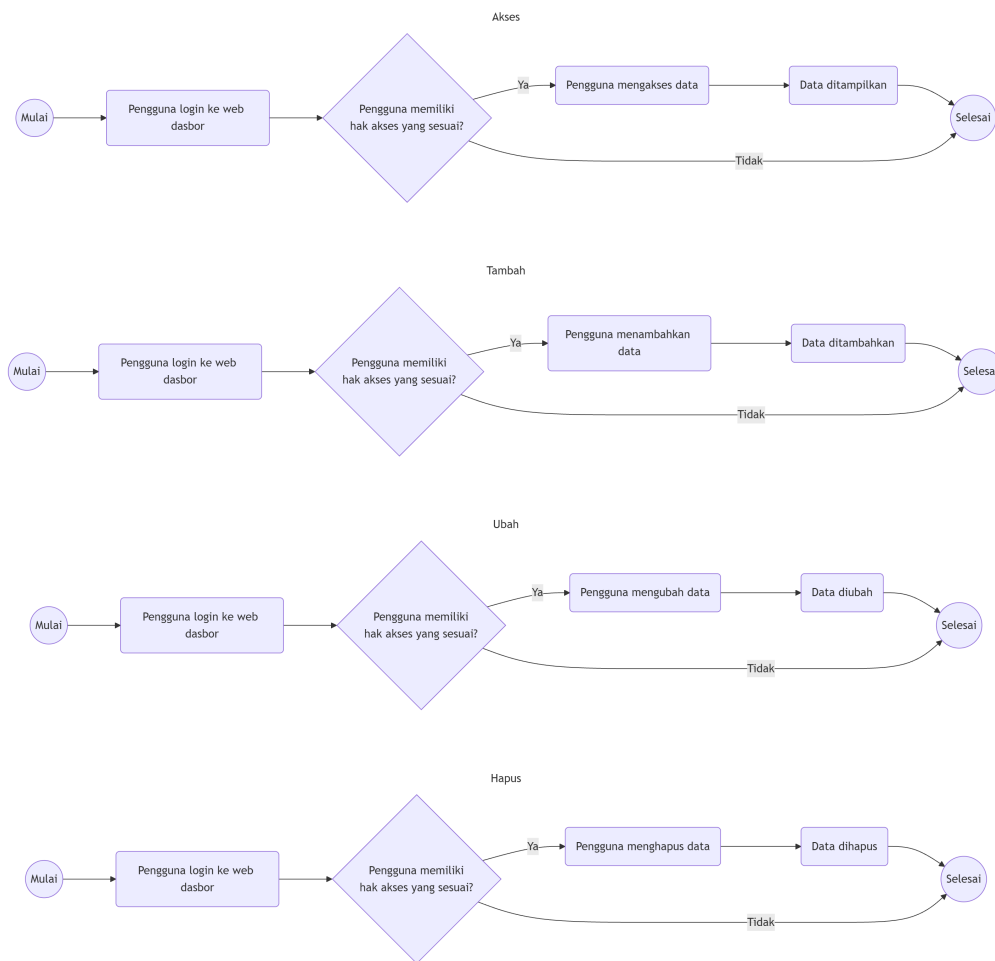
Sistem informasi organisasi INFINITE terdiri dari dua komponen utama yang saling terintegrasi, yaitu *backend* API Laravel dan *frontend* Next.js. Sistem

menyediakan autentikasi terpusat melalui protokol OIDC dengan sistem SSO INFINITE berbasis *software* Authentik. *Backend* menyediakan RESTful API berversi (/api/v1/) yang dikonsumsi oleh *frontend* menggunakan *library* Axios. Seluruh data dikomunikasikan dalam format JSON dengan paginasi berbasis kursor. Sistem dikelola melalui antarmuka dasbor web yang hanya dapat diakses oleh pengguna terautentikasi dengan peran dan hak akses yang sesuai.

2.1. Manajemen Data Umum

Manajemen data untuk setiap entitas secara umum mencakup pengelolaan informasi berupa pengaksesan, penambahan, pembaruan, dan penghapusan data. Pengguna dengan hak akses yang sesuai dengan data tertentu yang hendak dikelola dapat melakukan operasi tersebut melalui antarmuka dasbor web. Alur manajemen data secara umum dapat dilihat seperti pada gambar 4.14.

Gambar 4.14 *Flowchart* Manajemen Data Umum

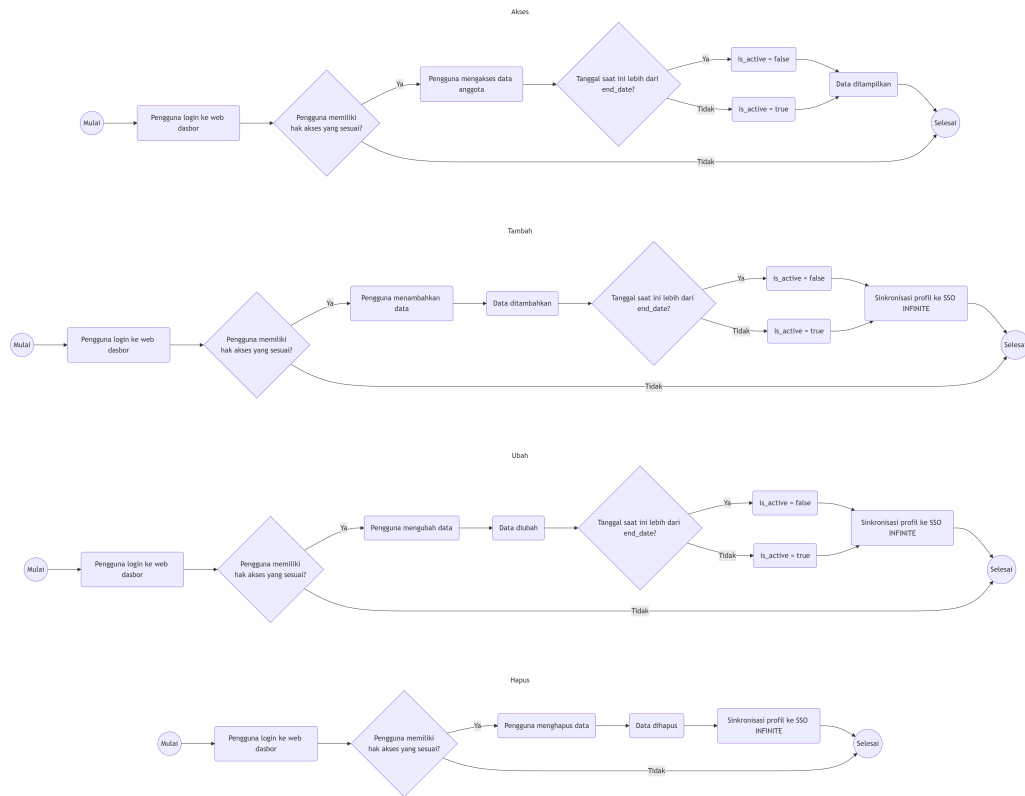


2.2. Manajemen Data Anggota

Manajemen data anggota hampir serupa dengan manajemen data umum, namun memiliki alur tambahan untuk penentuan status keaktifan anggota, serta sinkronisasi profil ke sistem SSO INFINITE. Setiap anggota memiliki atribut `end_date` yang menandai akhir masa aktif keanggotaan. Sistem secara otomatis menentukan status keaktifan anggota dengan membandingkan tanggal saat ini dengan tanggal `end_date`. Jika tanggal saat ini melewati tanggal `end_date`, maka status anggota menjadi tidak aktif (`is_active = false`). Selain itu, setiap perubahan data anggota akan disinkronkan ke sistem SSO INFINITE melalui API

Authentik untuk memastikan konsistensi data pengguna di seluruh sistem. Alur manajemen data anggota dapat dilihat seperti pada gambar 4.15.

Gambar 4.15 *Flowchart* Manajemen Data Anggota



2.3. Manajemen Data Kepengurusan

Manajemen data kepengurusan mencakup pengelolaan struktur *Core Team* dan *Community Group Administrator*. Prosesnya juga hampir serupa dengan manajemen data umum, namun memiliki alur tambahan untuk penentuan kepengurusan yang aktif menjabat saat ini. Setiap waktu, hanya satu kepengurusan yang dapat aktif menjabat, sehingga setiap perubahan kepengurusan akan menonaktifkan kepengurusan sebelumnya. Setelah suatu periode kepengurusan dibuat, pengguna kemudian dapat menetapkan anggota yang mengisi posisi di setiap divisi atau grup.

2.4. Manajemen Data Kompetisi

Manajemen data kompetisi dimulai dari pengelolaan data `Competition` sebagai induk katalog. Untuk setiap tahun atau edisi pelaksanaan kompetisi, dicatatkan sebuah `CompetitionInstance` baru yang merujuk pada induk katalog kompetisi, dilengkapi dengan informasi spesifik pelaksanaan seperti penyelenggara, tanggal, lokasi, dan tautan. Taksonomi kompetisi seperti tipe penyelenggara, tingkat kompetisi, peringkat, durasi, jenis luaran, dan tipe tim didefinisikan sebagai data referensi terpisah yang masing-masing memiliki bobot penilaian.

2.5. Manajemen Data Pendanaan dan Prestasi Tim

Pengguna membentuk `Team` untuk mengikuti kompetisi, dengan menunjuk salah satu pengguna sebagai ketua tim dan menambahkan pengguna lainnya sebagai anggota tim. Pengajuan `FundApplication` diajukan oleh tim yang telah dinyatakan lolos seleksi suatu kompetisi dan membutuhkan pendanaan untuk melanjutkan ke tahap berikutnya. Proses pengajuan dilengkapi dengan pengunggahan dokumen bukti lolos seleksi dan proposal pendanaan yang disimpan dalam bentuk PDF. Pengajuan berada dalam status `Pending` pada awalnya, kemudian dapat diterima (`Accepted`) atau ditolak (`Rejected`) oleh pengurus dengan hak akses yang sesuai. Setiap perubahan status dapat dilihat oleh pemohon.

Setelah kompetisi dilaksanakan, `Achievement` dicatat dengan mencantumkan tim beserta data-data kompetisi dan taksonomi lainnya yang sesuai. Sistem secara otomatis menghitung poin prestasi berdasarkan perkalian bobot keenam taksonomi (`organizer_type × team_type × scale ×`

`time_range × output × rank`). Poin-poin ini kemudian diakumulasi ke dalam `AchievementLeaderboard` yang menampilkan peringkat anggota per tahun berdasarkan total poin.

2.6. Manajemen Data Grup dan Hak Akses

Manajemen data grup dan hak akses dilakukan melalui pengelolaan `Group` dan `Permission`, yang juga hampir serupa dengan manajemen data umum. Setiap grup memiliki daftar hak akses yang ditetapkan, dan setiap pengguna dapat menjadi anggota dari satu atau beberapa grup. Sistem secara otomatis menentukan hak akses pengguna berdasarkan gabungan hak akses dari semua grup yang diberikan. Pengguna dengan hak akses yang sesuai dapat menambahkan atau menghapus anggota. Secara bawaan, setiap pengguna yang merupakan anggota `INFINITE` akan menjadi anggota grup `Member`, sedangkan pengurus akan menjadi anggota grup `Core Team xxxx` atau `CG Admin xxxx` sesuai dengan periode kepengurusan masing-masing. Anggota yang aktif akan diberikan pula grup `Active Member`, sedangkan anggota yang tidak aktif akan diberikan grup `Inactive Member`. Sedangkan pengurus yang aktif menjabat akan diberikan grup `Core Team` atau `CG Admin`, sedangkan pengurus yang tidak aktif menjabat akan diberikan grup `Ex Core Team` atau `Ex CG Admin`.

2.7. Manajemen Konfigurasi

Manajemen konfigurasi dilakukan melalui halaman *Settings* pada web dasbor, dan disimpan dalam tabel `Configs` berupa pasangan kunci-nilai dengan tipe data dan penanda privasi. Konfigurasi publik dapat diakses melalui endpoint API tanpa autentikasi, sedangkan konfigurasi privat hanya tersedia untuk pengguna

terautentikasi. Konfigurasi hanya dapat dikelola oleh pengguna dengan izin yang sesuai.

3. Kualitas Sistem Informasi

Pengujian kualitas sistem informasi dilakukan setelah semua komponen individu saling terintegrasi menjadi satu kesatuan sistem yang utuh. Pengujian kualitas dilakukan mengacu pada standar ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use*, dengan empat karakteristik yang dievaluasi, yaitu *Functional Suitability*, *Interaction Capability*, *Performance Efficiency*, dan *Beneficialness*.

Aspek *Functional Suitability* dan *Interaction Capability* yang termasuk dalam aspek *Product Quality* diuji oleh lima tenaga ahli dalam bidang sistem informasi. Kelima tenaga ahli memiliki latar belakang dan pemahaman yang memadai baik secara profesional maupun terkait INFINITE. Identitas kelima tenaga ahli dapat dilihat pada tabel 4.1.

Tabel 4.1 Identitas Tenaga Ahli Penguji

No.	Nama	Profesi	Instansi
1	Ikhwan Inzaghi	<i>Software Engineer</i>	Universitas Nahdlatul Ulama Yogyakarta
2	Zainal Ma'ruf Abidin	<i>UI/UX Designer</i>	PT Digdaya Inovasi Gemilang
3	Damar Albaribin Syamsu	<i>AI Software Engineer</i>	Pelgo Incorporated
4	Satya Adhiyaksa Ardy	<i>System Software Engineer</i>	Sillicon XPandas
5	Dzul Fadli Rahman, S.Kom., M.Sc.	Dosen	Universitas Negeri Yogyakarta

Aspek *Performance Efficiency* diuji menggunakan Google Lighthouse untuk

frontend dan Grafana k6 untuk *backend*. Sedangkan aspek *Beneficialness* diuji oleh sembilan pengurus dan sembilan anggota INFINITE.

3.1. Hasil Pengujian *Functional Suitability*

Pengujian *Functional Suitability* dilakukan untuk memastikan kesesuaian seluruh fungsi utama sistem dengan spesifikasi yang diharapkan. Hasil pengujian yang telah dilakukan oleh lima tenaga ahli dapat dilihat pada lampiran 2. Berikut adalah ringkasan hasil pengujiannya:

Tabel 4.2 Skor Pengujian *Functional Suitability*

No.	Nama Penguji	Skor
1	Ikhwan Inzaghi	23
2	Zainal Ma'ruf Abidin	23
3	Damar Albaribin Syamsu	23
4	Satya Adhiyaksa Ardy	23
5	Dzul Fadli Rahman, S.Kom., M.Sc.	23
Total		115

Dari total skor tersebut, kemudian dilakukan perhitungan sesuai dengan rumus persentase kelayakan pada persamaan (3.1), sehingga diperoleh:

$$\text{Persentase Kelayakan} = \frac{115}{23 \times 5 \times 1} \times 100\% = \frac{115}{115} \times 100\% = 100\% \quad 4.1$$

Berdasarkan kriteria interpretasi kelayakan pada tabel 3.2, hasil pengujian *Functional Suitability* sebesar 100% termasuk dalam kriteria **Sangat Layak**, yang menunjukkan bahwa seluruh fungsi sistem telah berjalan sesuai dengan spesifikasi yang diharapkan.

3.2. Hasil Pengujian *Interaction Capability*

Pengujian *Interaction Capability* dilakukan menggunakan *USE Questionnaire* yang mencakup empat dimensi, yaitu *Usefulness*, *Ease of Use*, *Ease of Learning*, dan *Satisfaction*. Pengujian ini bertujuan untuk mengukur dan memastikan aspek kegunaan, kemudahan penggunaan, kemudahan pembelajaran, dan kepuasan pengguna. Hasil pengujian yang telah dilakukan oleh lima tenaga ahli dapat dilihat pada lampiran 2. Berikut adalah ringkasan hasil pengujiannya:

Tabel 4.3 Skor Pengujian *Interaction Capability*

No.	Nama Penguji	Skor
1	Ikhwan Inzaghi	122
2	Zainal Ma'ruf Abidin	133
3	Damar Albaribin Syamsu	150
4	Satya Adhiyaksa Ardy	136
5	Dzul Fadli Rahman, S.Kom., M.Sc.	147
Total		688

Dari total skor tersebut, kemudian dilakukan perhitungan sesuai dengan rumus persentase kelayakan pada persamaan (3.2), sehingga diperoleh:

$$\text{Persentase Kelayakan} = \frac{688}{30 \times 5 \times 5} \times 100\% = \frac{688}{750} \times 100\% = 91,73\% \quad 4.2$$

Berdasarkan kriteria interpretasi kelayakan pada tabel 3.3, hasil pengujian *Interaction Capability* sebesar 91,73% termasuk dalam kriteria **Sangat Layak**, yang menunjukkan bahwa sistem informasi memiliki tingkat kegunaan, kemudahan penggunaan, kemudahan pembelajaran, dan kepuasan pengguna yang

sangat baik.

3.3. Hasil Pengujian *Performance Efficiency*

Pengujian *Performance Efficiency* bertujuan untuk mengukur performa sistem informasi INFINITE dalam merespons permintaan pengguna dan memproses data.

Pengujiann ini dilakukan dengan dua pendekatan:

- a. Pengujian *frontend* Next.js dilakukan menggunakan Google Lighthouse pada seluruh halaman web dasbor. Hasil pengujian lengkap dapat dilihat pada lampiran 36. Berdasarkan hasil tersebut, didapatkan rata-rata skor performa *frontend* sebesar 90,42, yang termasuk dalam kategori **Baik** menurut kriteria interpretasi skor performa Lighthouse pada tabel 3.4.
- b. Pengujian *backend* API Laravel dilakukan menggunakan Grafana k6 dengan tipe *breakpoint test*. Pengujian dilakukan pada *endpoint* utama API untuk menemukan batas kapasitas sistem. Hasil pengujian lengkap dapat dilihat pada lampiran 37. Berdasarkan hasil tersebut, didapatkan rata-rata *Response Time* sebesar **1 detik**, rata-rata *Throughput* sebesar **53,6 permintaan per detik**, dan rata-rata *Error Rate* sebesar **0%**, serta maksimum jumlah *Virtual Users* sebanyak **189 pengguna simultan**.

3.4. Hasil Pengujian *Beneficialness*

Pengujian *Beneficialness* dilakukan menggunakan kuesioner *End-User Computing Satisfaction* (EUCS) yang mencakup lima dimensi, yaitu *Content*, *Accuracy*, *Format*, *Ease of Use*, dan *Timeliness*. Pengujian ini bertujuan untuk mengukur dan memastikan kemanfaatan sistem informasi INFINITE bagi pengguna akhir. Berikut adalah ringkasan hasil pengujiannya:

Tabel 4.4 Skor Pengujian *Beneficialness*

No.	Nama	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Total
1	Pengurus 1	4	4	4	4	5	5	2	4	2	4	2	4	44
2	Pengurus 2	4	4	5	5	5	5	5	5	4	4	4	4	54
3	Pengurus 3	5	4	5	5	5	5	5	5	5	3	5	4	56
4	Pengurus 4	4	4	4	4	4	4	4	4	4	4	4	4	48
5	Pengurus 5	4	4	4	4	4	4	4	4	4	4	4	4	48
6	Pengurus 6	4	3	3	4	4	3	3	4	3	4	4	3	42
7	Pengurus 7	5	5	5	5	5	5	5	5	5	5	5	5	60
8	Pengurus 8	3	3	3	3	4	4	4	4	4	4	4	4	44
9	Pengurus 9	4	4	4	4	4	4	4	4	4	4	4	4	48
10	Anggota 1	5	5	5	5	5	5	5	5	5	5	5	5	60
11	Anggota 2	3	4	4	3	4	4	3	3	3	4	3	4	42
12	Anggota 3	4	4	3	4	4	4	4	4	2	2	2	4	41
13	Anggota 4	4	4	4	4	4	4	4	4	4	4	4	4	48
14	Anggota 5	4	4	4	4	4	4	4	3	4	4	3	4	46
15	Anggota 6	5	4	5	5	4	5	4	5	5	4	3	5	54
16	Anggota 7	4	4	3	4	4	4	4	5	4	4	3	4	47
17	Anggota 8	4	4	4	3	5	4	4	4	4	4	4	4	48
18	Anggota 9	4	4	4	4	4	4	4	4	4	4	4	4	48
Total														878

Dari total skor tersebut, kemudian dilakukan perhitungan sesuai dengan rumus persentase kelayakan pada persamaan (3.2), sehingga diperoleh:

$$\text{Persentase Kelayakan} = \frac{878}{12 \times 5 \times 18} \times 100\% = \frac{878}{1080} \times 100\% = 81,30\% \quad 4.3$$

Berdasarkan kriteria interpretasi kelayakan pada tabel 3.3, hasil pengujian *Beneficialness* sebesar 81,30% termasuk dalam kriteria **Sangat Layak**, yang menunjukkan bahwa sistem informasi memberikan kemanfaatan yang sangat baik bagi pengguna akhir.

B. Pembahasan

Sistem informasi organisasi INFINITE dikembangkan untuk mengatasi permasalahan-permasalahan yang dihadapi sistem yang telah ada sebelumnya

sebagaimana diidentifikasi pada latar belakang penelitian, yaitu: (1) sistem informasi yang terpecah dalam dua sistem berupa sistem Laravel di *shared hosting* dan sistem Strapi di server terpisah, menciptakan fragmentasi dan potensi inkonsistensi data; (2) sistem Strapi yang tidak mendukung integrasi dengan protokol OIDC untuk memungkinkan autentikasi terpusat dengan sistem SSO; (3) sistem Laravel yang tidak memiliki *image* kontainer untuk dapat di-*deploy* pada platform Kubernetes; (4) pengoperasian dua infrastruktur secara bersamaan (*shared hosting* dan VPS) yang mengakibatkan inefisiensi biaya operasional; serta (5) sistem Laravel yang menggunakan arsitektur *monolithic* sehingga tidak menyediakan API untuk integrasi dengan aplikasi lain. Sistem baru ini dikembangkan menggunakan model *Waterfall* yang terdiri dari empat tahapan pelaksanaan, yaitu *Requirements Definition*, *System and Software Design*, *Implementation and Unit Testing*, serta *Integration and System Testing*. Tahap pertama dimulai dengan analisis kebutuhan melalui wawancara dengan pengurus INFINITE dan observasi terhadap sistem informasi yang telah ada sebelumnya. Hasil analisis dijadikan sebagai acuan untuk merancang arsitektur sistem, antarmuka pengguna, serta basis data. Rancangan kemudian diimplementasikan menggunakan Laravel sebagai *backend* dengan arsitektur *headless*, dan Next.js sebagai *frontend*. Implementasi menghasilkan sistem informasi organisasi INFINITE yang terintegrasi dengan autentikasi OIDC SSO INFINITE dan di-*deploy* sebagai aplikasi terkontainerisasi di Kubernetes.

Sistem informasi yang dihasilkan menyederhakan proses administrasi dan pengelolaan informasi organisasi INFINITE. Data organisasi yang sebelumnya

terfragmentasi dalam sistem monolitik di *shared hosting* dan Strapi di dua server terpisah kini menjadi satu platform terintegrasi yang terkontainerisasi dan ter-deploy di Kubernetes. Konsolidasi infrastruktur ini sekaligus mengatasi inefisiensi biaya operasional karena kebutuhan *shared hosting* tersendiri tidak lagi diperlukan. Selain itu, integrasi autentikasi OIDC dengan sistem SSO INFINITE juga menyelesaikan tantangan fragmentasi autentikasi yang sebelumnya dihadapi. Seluruh aplikasi dalam ekosistem INFINITE kini dapat menggunakan satu set kredensial tunggal, meningkatkan pengalaman pengguna dan mengurangi beban administratif dalam pengelolaan akun.

Arsitektur *headless* yang diterapkan pada sistem informasi INFINITE memberikan beberapa keunggulan signifikan dibandingkan sistem monolitik sebelumnya. Pertama, pemisahan total antara *backend* dan *frontend* memungkinkan kedua komponen dikembangkan, di-deploy, dan diskalakan secara independen. Hal ini sejalan dengan kebutuhan INFINITE yang beroperasi di lingkungan Kubernetes, di mana setiap komponen berjalan dalam kontainer terpisah yang dapat diskalakan sesuai beban. Kedua, pendekatan *API-first* memungkinkan *backend* untuk tidak hanya melayani *frontend* utama, tetapi juga menyediakan API yang dapat dikonsumsi oleh aplikasi-aplikasi lain dalam ekosistem INFINITE. Ketiga, penggunaan *Clean Architecture* pada *frontend* Next.js meningkatkan modularitas dan memudahkan pemeliharaan, dengan pemisahan tegas antara domain, aplikasi, infrastruktur, dan presentasi.

Penggunaan model *deployment* dengan kontainer dan lingkungan Kubernetes juga memberikan fleksibilitas, skalabilitas, dan konsistensi yang lebih baik.

Pipeline CI/CD berbasis GitHub Actions memastikan bahwa setiap perubahan kode pada *backend* dan *frontend* melalui proses validasi, pembangunan *image* kontainer, penandatanganan digital, dan rilis ke *registry* yang terotomasi. Pengelolaan Kubernetes melalui ArgoCD secara deklaratif dengan GitOps sebagai sumber kebenaran tunggal memastikan bahwa seluruh konfigurasi infrastruktur dapat direproduksi, dimonitor, disinkronkan, dideteksi dan dipulihkan dari penyimpangan (*drift*), serta dilakukan *rollback* secara otomatis.

Pengujian kualitas sistem berdasarkan standar ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use* dilakukan untuk memvalidasi bahwa sistem yang dikembangkan memenuhi kebutuhan pengguna dan standar kualitas perangkat lunak. Pengujian dilakukan terhadap empat karakteristik, yaitu *Functional Suitability*, *Interaction Capability*, *Performance Efficiency*, dan *Beneficialness*, dengan hasil sebagai berikut:

- a. Pengujian *Functional Suitability* dilakukan oleh lima tenaga ahli dan memperoleh persentase kelayakan sebesar 100% yang termasuk dalam kategori **Sangat Layak**. Hasil ini membuktikan bahwa seluruh fungsi sistem telah berjalan sesuai dengan spesifikasi yang diharapkan.
- b. Pengujian *Interaction Capability* menggunakan *USE Questionnaire* dilakukan oleh lima tenaga ahli dan memperoleh persentase kelayakan sebesar 91,73% yang termasuk dalam kategori **Sangat Layak**. Hasil ini membuktikan bahwa sistem informasi memiliki tingkat kegunaan, kemudahan penggunaan, kemudahan pembelajaran, dan kepuasan pengguna yang sangat baik.

c. Pengujian *Performance Efficiency* dilakukan dengan dua pendekatan.

Pengujian *frontend* menggunakan Google Lighthouse memperoleh rata-rata skor performa sebesar 90,42 yang termasuk dalam kategori **Baik**. Hasil ini menunjukkan bahwa halaman-halaman dasbor *frontend* dengan Next.js memiliki waktu pemuatan yang efisien dan memenuhi standar responsivitas aplikasi web modern. Pengujian *backend* menggunakan Grafana k6 dengan tipe *breakpoint test* memperoleh rata-rata *Response Time* sebesar **1 detik**, rata-rata *Throughput* sebesar **53,6 permintaan per detik**, rata-rata *Error Rate* sebesar **0%**, serta maksimum jumlah *Virtual Users* sebanyak **189 pengguna simultan**. Hasil ini memvalidasi bahwa *backend* yang dijalankan menggunakan Laravel Octane dengan FrankenPHP mampu menangani beban yang diantisipasi dengan respons yang cepat dan tanpa kesalahan.

d. Pengujian *Beneficialness* menggunakan kuesioner *End-User Computing Satisfaction* (EUCS) dilakukan oleh sembilan pengurus dan sembilan anggota INFINITE serta memperoleh persentase kelayakan sebesar 81,30% yang termasuk dalam kategori **Sangat Layak**. Hasil ini membuktikan bahwa sistem yang dikembangkan memberikan kemanfaatan yang sangat baik bagi pengguna, baik pengurus maupun anggota INFINITE.

Berdasarkan hasil pengujian keempat karakteristik tersebut, dapat disimpulkan bahwa sistem informasi organisasi INFINITE yang dikembangkan berjalan dengan baik secara fungsional dan teknis, serta diterima dengan baik oleh pengguna. Sistem tergolong layak digunakan sebagai solusi dalam mendukung pengelolaan data organisasi INFINITE secara terintegrasi dan berkelanjutan.

BAB V

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil penelitian dan pengembangan sistem informasi organisasi INFINITE yang telah diuraikan pada bab-bab sebelumnya, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Sistem informasi organisasi INFINITE berhasil dikembangkan dengan model *Waterfall* yang terdiri dari *Requirements Definition*, *System and Software Design*, *Implementation and Unit Testing*, serta *Integration and System Testing*. Sistem yang dihasilkan berupa *backend* Laravel dengan arsitektur *headless* yang menyediakan API, *frontend* Next.js, serta integrasi autentikasi terpusat berstandar protokol OIDC melalui sistem SSO INFINITE. Sistem berhasil di-*deploy* sebagai aplikasi terkontainerisasi di platform Kubernetes dengan pendekatan GitOps menggunakan ArgoCD, memungkinkan pengelolaan konfigurasi infrastruktur secara deklaratif melalui repositori Git. *Pipeline* CI/CD berbasis GitHub Actions memastikan setiap perubahan kode melalui proses validasi, pembangunan *image* kontainer, penandatanganan digital dengan Cosign, serta rilis ke *registry* secara otomatis.
2. Sistem informasi organisasi INFINITE telah diuji berdasarkan standar kualitas ISO/IEC 25010:2023 untuk *Product Quality* dan ISO/IEC 25019:2023 untuk *Quality in Use*. Pengujian karakteristik *Functional Suitability* memperoleh persentase kelayakan sebesar 100% (Sangat Layak).

Pengujian karakteristik *Interaction Capability* memperoleh persentase kelayakan sebesar 91,73% (Sangat Layak). Pengujian karakteristik *Performance Efficiency* pada *frontend* menghasilkan rata-rata skor performa sebesar 90,42 (Baik), sedangkan pada *backend* menghasilkan rata-rata *Response Time* sebesar 1 detik, *Throughput* sebesar 53,6 permintaan per detik, *Error Rate* sebesar 0%, serta maksimum 189 *Virtual Users* simultan. Pengujian karakteristik *Beneficialness* memperoleh persentase kelayakan sebesar 81,30% (Sangat Layak). Berdasarkan hasil pengujian tersebut, sistem informasi organisasi INFINITE tergolong layak digunakan sebagai solusi pengelolaan data organisasi secara terintegrasi dan berkelanjutan.

B. Keterbatasan Produk

Produk sistem informasi organisasi INFINITE yang dihasilkan masih memiliki sejumlah keterbatasan, antara lain sebagai berikut:

- a. Sistem belum diintegrasikan dengan halaman *landing* organisasi yang ditujukan untuk publik dalam melihat data seperti papan peringkat prestasi atau galeri proyek, sehingga konsumsi data publik masih terbatas melalui API.
- b. Pengujian kualitas karakteristik *Performance Efficiency* pada *backend* dilakukan dengan tipe *breakpoint test* yang berfokus pada batas kapasitas sistem, sehingga belum mencakup skenario pengujian beban jangka panjang (*soak test*) maupun pengujian lonjakan beban (*spike test*).
- c. Infrastruktur Kubernetes belum dilengkapi dengan mekanisme pemantauan metrik aplikasi, sehingga selama pengujian tidak tersedia observabilitas

terhadap penggunaan sumber daya, pelacakan kesalahan, maupun metrik operasional lainnya.

- d. Faktor eksternal pada sisi infrastruktur seperti *deployment* basis data, *gateway* API, dan konfigurasi komponen infrastruktur terkait lainnya belum *di-tune* secara optimal, sehingga dampak terhadap performa sistem belum dapat dianalisis secara komprehensif.

C. Saran

Berdasarkan kesimpulan dan keterbatasan produk yang telah diuraikan, terdapat sejumlah saran yang dapat dijadikan acuan untuk pengembangan lebih lanjut, yaitu sebagai berikut:

1. Mengembangkan halaman *landing* organisasi yang memungkinkan publik untuk melihat data seperti papan peringkat prestasi dan galeri proyek.
2. Melakukan pengujian *Performance Efficiency* pada *backend* dengan skenario *soak test* dan *spike test* untuk memastikan ketahanan sistem terhadap beban jangka panjang maupun lonjakan beban mendadak.
3. Mengimplementasikan mekanisme pemantauan metrik aplikasi menggunakan VictoriaMetrics yang ringan dan Grafana untuk menyediakan observabilitas terhadap penggunaan sumber daya, pelacakan kesalahan, dan metrik operasional lainnya pada infrastruktur Kubernetes.
4. Melakukan *tuning* dan optimasi terhadap faktor eksternal pada sisi infrastruktur seperti konfigurasi basis data, *gateway* API, dan komponen terkait, serta melakukan pengujian performa terintegrasi untuk menganalisis dampak secara komprehensif.

DAFTAR PUSTAKA

- Abana, R. and Sy, R. (2022). Iso/iec 25010 based evaluation of rice seed analyzer: a machine vision application using image processing technique. *Indonesian Journal of Electrical Engineering and Computer Science*, 28(2):994–1001.
- Akinsola, J. E. T., Ogunbanwo, A. S., Okesola, O. J., Odun-Ayo, I. J., Ayegbusi, F. D., and Adebisi, A. A. (2020). Comparative analysis of software development life cycle models (sdlc). In *Computer Science On-line Conference*, pages 310–322. Springer.
- Alyahya, S. (2020). Crowdsourced software testing: A systematic literature review. *Information and Software Technology*, 127:106363.
- Argo Project (2024). Argo cd - declarative gitops cd for kubernetes. <https://argo-cd.readthedocs.io/en/stable/>.
- Arikunto, S. (2013). *Prosedur Penelitian: Suatu Pendekatan Praktik*. Rineka Cipta, Jakarta, 15th edition.
- Balaji, S. and Murugaiyan, M. S. (2012). Waterfall vs v-model vs agile: A comparative study on sdlc. *International Journal of Information Technology and Business Management*, 2(1):26–30.
- Bass, L., Clements, P., and Kazman, R. (2017). *Software Architecture in Practice*. Addison-Wesley Professional, Boston, MA, 3rd edition.
- Beetz, F. and Harrer, S. (2022). Gitops: The evolution of devops? *IEEE Software*, 39(2):63–69.
- Bierman, G., Abadi, M., and Torgersen, M. (2014). Understanding typescript. *European Conference on Object-Oriented Programming*, pages 257–281.
- Biørn-Hansen, A., Majchrzak, T. A., and Grønli, T.-M. (2018). Progressive web apps: The definite approach to cross-platform development? In *Proceedings of the 51st Hawaii International Conference on System Sciences*, pages 5735–5744. University of Hawai’i at Manoa.
- Burns, B., Beda, J., and Hightower, K. (2019). *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O’Reilly Media, Incorporated, 2nd edition.
- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, omega, and kubernetes. In *Communications of the ACM*, volume 59, pages 50–57.
- Cantor, S., Hirsch, F., Kemp, J., Philpott, R., and Maler, E. (2005). Bindings for the oasis security assertion markup language (saml) v2.0. OASIS Standard.
- Cayola, L. and Macias, J. A. (2018). Systematic guidance on usability methods in user-centered software development. *Information and Software Technology*, 97:163–175.

- Charland, A. and Leroux, B. (2011). Mobile application development: Web vs. native. *Communications of the ACM*, 54(5):49–53.
- Doll, W. J. and Torkzadeh, G. (1988). The measurement of end-user computing satisfaction. *MIS Quarterly*, 12(2):259–274.
- Ebert, C., Gallardo, G., Hernantes, J., and Serrano, N. (2016). Devops. *IEEE Software*, 33(3):94–100.
- Feriawan, I. P. S., Paramitha, A. A. I. I., and Dewi, E. G. A. (2024). Rancang bangun sistem informasi manajemen transkrip aktivitas kemahasiswaan primakara university menggunakan metode extreme programming. *JATI (Jurnal Mahasiswa Teknik Informatika)*, 8(5):10162–10169.
- Ferrari, L. and Pirozzi, E. (2023). *Learn PostgreSQL*. Packt Publishing, Birmingham, UK.
- Fett, D., Küsters, R., and Schmitz, G. (2014). An expressive model for the web infrastructure: Definition and application to the browser id sso system. In *2014 IEEE Symposium on Security and Privacy*, pages 673–688. IEEE.
- Fett, D., Küsters, R., and Schmitz, G. (2016). A comprehensive formal security analysis of oauth 2.0. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 1204–1215. ACM.
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine.
- Foucault, M., Blanc, X., Falleri, J.-R., and Storey, M.-A. (2019). Fostering good coding practices through individual feedback and gamification: an industrial case study. *Empirical Software Engineering*, 24(6):3731–3754.
- Ghaffur, T. A. (2017). Analisis kualitas sistem informasi kegiatan sekolah berbasis mobile web di smk negeri 2 yogyakarta. *Elinvo (Electronics, Informatics, and Vocational Education)*, 2(1):94–101.
- GitHub Inc. (2024). Github actions documentation. <https://docs.github.com/en/actions>.
- Google (2026a). Lighthouse. <https://developer.chrome.com/docs/lighthouse>. Diakses: 2026-03-02.
- Google (2026b). Web vitals. <https://web.dev/articles/vitals>. Diakses: 2026-03-02.
- Grafana Labs (2026). Grafana k6. <https://grafana.com/docs/k6/latest/>. Diakses: 2026-03-02.
- Grassi, P. A., Garcia, M. E., and Fenton, J. L. (2017). Digital identity guidelines: Authentication and lifecycle management. Technical Report NIST SP 800-63B, National Institute of Standards and Technology.

- Haniva, D. T., Ramadhan, J. A., and Suharso, A. (2023). Systematic literature review penggunaan metodologi pengembangan sistem informasi waterfall, agile, dan hybrid. *Journal of Information Engineering and Educational Technology*.
- Haoues, M., Sellami, A., and Ben-Abdallah, H. (2017). A guideline for software architecture selection based on iso 25010 quality related characteristics. *International Journal of System Assurance Engineering and Management*, 8:886–909.
- Hardt, D. (2012). The oauth 2.0 authorization framework. RFC 6749, Internet Engineering Task Force.
- Hassan, S. A. Z. and Eassa, A. (2024). Comparing performance between headless cms and traditional cms. *International Journal of Business Information Systems*.
- Hidayah, N. A., Fetrina, E., and Taufan, A. Z. (2020). Model satisfaction users measurement of academic information system using end-user computing satisfaction (eucs) method. *Applied Information System and Management (AISM)*, 3(2):119–123.
- Hovorushchenko, T. (2018). Methodology of evaluating the sufficiency of information for software quality assessment according to iso 25010. *Journal of Information and Organizational Sciences*, 42(1):15–34.
- Hughes, J., Cantor, S., Hodges, J., Hirsch, F., Mishra, P., Philpott, R., and Maler, E. (2005). Profiles for the oasis security assertion markup language (saml) v2.0. OASIS Standard.
- Humble, J. and Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, Boston, MA.
- INFINITE UNY (2026). Hello. <https://www.infiniteuny.id/>. Diakses: 2026-03-02.
- International Organization for Standardization (2023a). Iso/iec 25010:2023 – systems and software engineering – systems and software quality requirements and evaluation (square) – product quality model. International Standard. ISO/IEC 25010:2023.
- International Organization for Standardization (2023b). Iso/iec 25019:2023 – systems and software engineering – systems and software quality requirements and evaluation (square) – quality-in-use model. International Standard. ISO/IEC 25019:2023.
- International Organization for Standardization (2026). About iso. <https://www.iso.org/about>. Diakses: 2026-03-02.
- Jabangwe, R., Edison, H., and Duc, A. (2018). Software engineering process models for mobile app development: A systematic literature review. *Journal of Systems and Software*, 145:98–111.

- Jayanto, R. D. (2017). Evaluasi kualitas aplikasi mobile kamus istilah jaringan pada platform android dengan standar iso/iec 25010. *Elinvo (Electronics, Informatics, and Vocational Education)*, 2(2):178–182.
- Jones, M. B., Bradley, J., and Sakimura, N. (2015). Json web token (jwt). RFC 7519, Internet Engineering Task Force.
- Kaushalya, T. and Perera, I. (2021). Framework to migrate angularjs based legacy web application to react component architecture. In *2021 Moratuwa Engineering Research Conference (MERCon)*, pages 357–362. IEEE.
- Krasner, G. E. and Pope, S. T. (1988). A cookbook for using the model-view-controller user interface paradigm in smalltalk-80. *Journal of Object-Oriented Programming*, 1(3):26–49.
- Kurniasari, M. R. and Rochimah, S. (2025). An evaluation model of website testing framework based on iso 25010 performance efficiency. *Indonesian Journal of Electrical Engineering and Computer Science*, 37(2):1130–1139.
- Laaziri, M., Benmoussa, K., Khouilji, S., and Kerkeb, M. L. (2019). A comparative study of php frameworks performance. In *Procedia Manufacturing*, volume 32, pages 864–871. Elsevier.
- Lamada, M. S., Miru, A. S., and Amalia, R. (2020). Pengujian aplikasi sistem monitoring perkuliahan menggunakan standar iso 25010. *Jurnal MediaTIK*, 3(3).
- Laravel LLC (2026). Laravel documentation. <https://laravel.com/docs/>. Diakses: 2026-03-02.
- Laudon, K. C. and Laudon, J. P. (2020). *Management Information Systems: Managing the Digital Firm*. Pearson Education, Harlow, UK, 16th edition.
- Lawson, A. and Sica, J. (2026). Cncf annual cloud native survey: The infrastructure of ai's future.
- Leite, L., Rocha, C., Kon, F., Milojevic, D., and Meirelles, P. (2019). A survey of devops concepts and challenges. In *ACM Computing Surveys*, volume 52, pages 1–35. ACM.
- Limoncelli, T. A. (2018). Gitops: A path to more self-service it. In *Communications of the ACM*, volume 61, pages 38–42. ACM.
- Liu, X., Liu, J., Wang, W., and Zhu, S. (2018). Android single sign-on security: Issues, taxonomy and directions. *Future Generation Computer Systems*, 89:402–420.
- Lodderstedt, T., Bradley, J., Labunets, A., and Fett, D. (2020). Oauth 2.0 security best current practice. Internet-Draft, Internet Engineering Task Force. draft-ietf-oauth-security-topics-16.

- Lodderstedt, T., Bradley, J., Labunets, A., and Fett, D. (2025). Best current practice for oauth 2.0 security. RFC 9700, Internet Engineering Task Force.
- Lund, A. M. (2001). Measuring usability with the use questionnaire. *Usability Interface*, 8(2):3–6.
- Mainka, C., Mladenov, V., Schwenk, J., and Wich, T. (2017). Sok: Single sign-on security – an evaluation of openid connect. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 251–266. IEEE.
- Markovic, D., Scekic, M., Bucaioni, A., and Cicchetti, A. (2022). Could jamstack be the future of web applications architecture? an empirical study. In *Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing*, pages 1872–1881. ACM.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, Boston, MA.
- Mell, P. and Grance, T. (2011). The nist definition of cloud computing. Technical Report SP 800-145, National Institute of Standards and Technology.
- Meta Platforms Inc. (2026). React reference overview. <https://react.dev/reference/react>. Diakses: 2026-03-02.
- Microsoft (2026). Typescript documentation. <https://www.typescriptlang.org/docs/>. Diakses: 2026-03-02.
- Mikkonen, T., Pautasso, C., Systa, K., and Taivalsaari, A. (2019). On the web platform cornucopia. In *Web Engineering*, pages 287–296. Springer.
- Mladenov, V., Mainka, C., and Schwenk, J. (2016). On the security of modern single sign-on protocols: Second-order vulnerabilities in openid connect.
- Mladenova, T. (2020). Software quality metrics – research, analysis and recommendation. In *2020 International Conference Automatics and Informatics (ICAI)*, pages 1–6. IEEE.
- Nuseibeh, B. and Easterbrook, S. (2000). Requirements engineering: A roadmap. *Proceedings of the Conference on the Future of Software Engineering*, pages 35–46.
- O’Brien, J. A. and Marakas, G. M. (2012). *Introduction to Information Systems*. McGraw-Hill Education, New York, NY, 16th edition.
- Ometov, A., Bezzateev, S., Mäkitalo, N., Andreev, S., Mikkonen, T., and Koucheryavy, Y. (2018). Multi-factor authentication: A survey. *Cryptography*, 2(1):1.
- Parecki, A., Ryck, P. D., and Waite, D. (2025). OAuth 2.0 for Browser-Based Applications. Internet-Draft draft-ietf-oauth-browser-based-apps-26, Internet Engineering Task Force. Work in Progress.

- Pashalidis, A. and Mitchell, C. J. (2003). A taxonomy of single sign-on systems. In *Information Security and Privacy*, pages 249–264. Springer.
- Pertiwi, A. D., Septian, R. N., Ashifa, R., and Prihantini, P. (2021). Peran organisasi kemahasiswaan dalam membangun karakter: Urgensi organisasi kemahasiswaan pada generasi digital. *Aulad: Journal on Early Childhood*.
- Petersen, K., Vakkalanka, S., and Kuzniarz, L. (2015). Guidelines for conducting systematic mapping studies in software engineering: An update. *Information and Software Technology*, 64:1–18.
- Pramitasari, B. and Nurgiyatna, N. (2019). Sistem informasi unit kegiatan mahasiswa marching band universitas muhammadiyah surakarta berbasis web. *Emitor: Jurnal Teknik Elektro*, 19(2):59–65.
- Pressman, R. S. and Maxim, B. R. (2014). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, NY, 8th edition.
- Richardson, L., Amundsen, M., and Ruby, S. (2013). *RESTful Web APIs: Services for a Changing World*. O'Reilly Media, Sebastopol, CA, 1st edition.
- Royce, W. W. (1970). *Managing the Development of Large Software Systems*. IEEE.
- Sakimura, N., Bradley, J., Jones, M. B., de Medeiros, B., and Mortimore, C. (2014). Openid connect core 1.0. https://openid.net/specs/openid-connect-core-1_0.html. OpenID Foundation.
- Saravanos, A. and Curinga, M. X. (2023). Simulating the software development lifecycle: The waterfall model. *Applied System Innovation*, 6(6):108.
- Sastypratiwi, H. and Yulianti, Y. (2019). Web application development using mvc-component-based approach. In *2019 International Conference on Data and Software Engineering (ICoDSE)*. IEEE.
- Shahin, M., Babar, M. A., and Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access*, 5:3909–3943.
- Sharif, A., Carbone, R., Sciarretta, G., and Ranise, S. (2022). Best current practices for oauth/oidc native apps. *Journal of Information Security and Applications*, 65:103097.
- Silva, W. O. d. and Farah, P. R. (2018). Characteristics and performance assessment of approaches pre-rendering and isomorphic javascript as a complement to spa architecture. In *Proceedings of the VII Brazilian Symposium on Software Components, Architectures, and Reuse*, pages 31–40. ACM.
- Sommerville, I. (2016). *Software Engineering*. Pearson Education, Harlow, UK, 10th edition.

- Strapi (2026). What is a headless cms? <https://strapi.io/what-is-headless-cms>. Diakses: 2026-03-02.
- Sugiyono (2013). *Metode Penelitian Kuantitatif, Kualitatif, dan R&D*. Alfabeta, Bandung.
- Taivalsaari, A., Mikkonen, T., Pautasso, C., and Systä, K. (2021). Full stack is not what it used to be. *Web Engineering*, pages 363–371.
- The PHP Documentation Group (2026). History of php. <https://www.php.net/manual/en/history.php.php>. Diakses: 2026-03-02.
- The PostgreSQL Global Development Group (2026). About. <https://www.postgresql.org/about/>. Diakses: 2026-03-02.
- Tofani, A. H. and Sukya, F. (2023). Sistem informasi manajemen kegiatan ukm english club psdku polinema di kediri berbasis framework laravel. *Jurnal Informatika dan Multimedia*, 14(2):15–22.
- Vallon, R., da Silva Estacio, B. J., Prikladnicki, R., and Grechenig, T. (2018). Systematic literature review on agile practices in global software development. *Information and Software Technology*, 96:161–180.
- Velepucha, V. and Flores, P. (2023). A survey on microservices architecture: Principles, patterns and migration challenges. *IEEE Access*, 11:91926–91963.
- Vercel Inc. (2026). Next.js docs. <https://nextjs.org/docs>. Diakses: 2026-03-02.
- Wagner, S., Méndez Fernández, D., Kalinowski, M., and Felderer, M. (2018). Status quo in requirements engineering: A theory and a global family of surveys. *ACM Transactions on Software Engineering and Methodology*, 28(2):1–48.
- Widjaja, S. and Prasojo, N. D. (2022). Perancangan sistem informasi unit kegiatan mahasiswa universitas nasional karangturi berbasis web. *Science Technology and Management Journal*, 2(1):31–37.
- Yadav, N., Rajpoot, D. S., and Dhakad, S. K. (2019). Laravel: A php framework for e-commerce website. In *International Conference on Intelligent Information Processing*.
- Younis, R., Iqbal, M., Munir, K., Javed, M. A., Haris, M. J., and Alahmari, S. A. (2024). A comprehensive analysis of cloud service models: Iaas, paas, and saas in the context of emerging technologies and trend. In *2024 International Conference on Electrical, Communication and Computer Engineering (ICECCE)*.

LAMPIRAN

Lampiran 1. Kisi-kisi Instrumen

Karakteristik	Sub-karakteristik	Indikator
<i>Functional Suitability</i>	<i>Functional Completeness</i>	1. Sistem dapat melakukan autentikasi melalui SSO INFINITE 2. Sistem dapat mengelola data akademik (strata, fakultas, program studi) 3. Sistem dapat mengelola data anggota (<i>User</i>) 4. Sistem dapat mengelola data kategori anggota (<i>Persona</i>) 5. Sistem dapat mengelola data kepengurusan (<i>Core Team</i>) 6. Sistem dapat mengelola data anggota kepengurusan (<i>Core Team Member</i>) berupa divisi dan foto 7. Sistem dapat mengelola data kelompok komunitas (<i>Community Group</i>) 8. Sistem dapat mengelola data anggota kelompok komunitas (<i>Community Group Member</i>) 9. Sistem dapat mengelola data administrator kelompok komunitas (<i>Community Group Admin</i>) 10. Sistem dapat mengelola data anggota administrator kelompok komunitas (<i>Community Group Admin Member</i>) 11. Sistem dapat mengelola data seri kompetisi (<i>Competition</i>) dan edisi pelaksanaan kompetisi (<i>Competition Instance</i>) 12. Sistem dapat mengelola data tim (<i>Team</i>) dan anggota tim (<i>Team Member</i>) untuk keperluan kompetisi 13. Sistem dapat mengelola data prestasi (<i>Achievement</i>) beserta status persetujuan 14. Sistem dapat mengelola data pengajuan dana (<i>Fund Application</i>) beserta status persetujuan

Karakteristik	Sub-karakteristik	Indikator
		15. Sistem dapat mengelola data testimoni (<i>Testimonial</i>) dan galeri proyek (<i>Project Gallery</i>) 16. Sistem dapat mengelola hak akses pengguna melalui grup (<i>Group</i>) dan izin (<i>Permission</i>) 17. Sistem dapat mengelola konfigurasi sistem (<i>Config</i>) termasuk pengaturan daftar ulang keanggotaan
	Functional Correctness	1. Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan 2. Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (<i>Pending/Accepted/Rejected</i>) 3. Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar
	Functional Appropriateness	1. Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data 2. Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar 3. Sistem menampilkan halaman pengaturan (<i>Settings</i>) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi
Interaction Capability	Appropriateness recognizability menggunakan Usefulness	1. <i>It helps me be more effective</i> 2. <i>It helps me be more productive</i> 3. <i>It is useful</i> 4. <i>It gives me more control over the activities in my life</i> 5. <i>It makes the things I want to accomplish easier to get done</i> 6. <i>It saves me time when I use it</i> 7. <i>It meets my needs</i> 8. <i>It does everything I would expect it to do</i>
	Learnability dan Self-descriptiveness menggunakan Ease of Learning	1. <i>I learned to use it quickly</i> 2. <i>I easily remember how to use it</i> 3. <i>It is easy to learn to use it</i> 4. <i>I quickly became skillful with it</i>

Karakteristik	Sub-karakteristik	Indikator
	Operability , User error protection , dan User assistance menggunakan Ease of Use	1. <i>It is easy to use</i> 2. <i>It is simple to use</i> 3. <i>It is user friendly</i> 4. <i>It requires the fewest steps possible to accomplish what I want to do with it</i> 5. <i>It is flexible</i> 6. <i>I can use it without written instructions</i> 7. <i>I don't notice any inconsistencies as I use it</i> 8. <i>Both occasional and regular users would like it</i> 9. <i>I can recover from mistakes quickly and easily</i> 10. <i>I can use it successfully every time</i>
	User engagement menggunakan Satisfaction	1. <i>I am satisfied with it</i> 2. <i>I would recommend it to a friend</i> 3. <i>It is fun to use</i> 4. <i>It works the way I want it to work</i> 5. <i>It is wonderful</i> 6. <i>I feel I need to have it</i> 7. <i>It is pleasant to use</i>
Performance Efficiency	Time Behavior	Frontend 1. <i>First Contentful Paint (FCP)</i>, kecepatan konten pertama tampil 2. <i>Speed Index (SI)</i>, kecepatan visual halaman menampilkan konten 3. <i>Largest Contentful Paint (LCP)</i>, waktu elemen konten terbesar tampil 4. <i>Total Blocking Time (TBT)</i>, total waktu main thread terblokir 5. <i>Cumulative Layout Shift (CLS)</i>, stabilitas visual halaman saat dimuat Backend <i>Response Time</i> , waktu respons rata-rata API
	Capacity	Backend 1. <i>Throughput</i>, jumlah permintaan yang diproses per detik (<i>requests per second</i>) 2. <i>Error Rate</i>, persentase permintaan gagal 3. <i>Virtual Users</i>, jumlah pengguna virtual maksimum sebelum degradasi

Karakteristik	Sub-karakteristik	Indikator
<i>Beneficialness</i>	<i>Usability → Satisfaction</i> menggunakan <i>Content</i>	1. <i>Does the system provide the precise information you need?</i> 2. <i>Does the information content meet your needs?</i> 3. <i>Does the system provide reports that seem to be just about exactly what you need?</i> 4. <i>Does the system provide sufficient information?</i>
	<i>Usability → Satisfaction</i> menggunakan <i>Accuracy</i>	1. <i>Is the system accurate?</i> 2. <i>Are you satisfied with the accuracy of the system?</i>
	<i>Usability → Satisfaction</i> menggunakan <i>Format</i>	1. <i>Do you think the output is presented in a useful format?</i> 2. <i>Is the information clear?</i>
	<i>Usability → Satisfaction</i> menggunakan <i>Ease of Use</i>	1. <i>Is the system user friendly?</i> 2. <i>Is the system easy to use?</i>
	<i>Usability → Satisfaction</i> menggunakan <i>Timeliness</i>	1. <i>Do you get the information you need in time?</i> 2. <i>Does the system provide up-to-date information?</i>

Lampiran 2. Instrumen *Product Quality*

No	Pernyataan	Penilaian	
		Ya	Tidak
1	Sistem dapat melakukan autentikasi melalui SSO INFINITE		
2	Sistem dapat mengelola data akademik (strata, fakultas, program studi)		
3	Sistem dapat mengelola data anggota (User)		
4	Sistem dapat mengelola data kategori anggota (Persona)		
5	Sistem dapat mengelola data kepengurusan (<i>Core Team</i>)		
6	Sistem dapat mengelola data anggota kepengurusan (<i>Core Team Member</i>) berupa divisi dan foto		
7	Sistem dapat mengelola data kelompok komunitas (<i>Community Group</i>)		
8	Sistem dapat mengelola data anggota kelompok komunitas (<i>Community Group Member</i>)		
9	Sistem dapat mengelola data administrator kelompok komunitas (<i>Community Group Admin</i>)		
10	Sistem dapat mengelola data anggota administrator kelompok komunitas (<i>Community Group Admin Member</i>) berupa kelompok dan foto		
11	Sistem dapat mengelola data seri kompetisi (<i>Competition</i>) dan edisi pelaksanaan kompetisi (<i>Competition Instance</i>)		
12	Sistem dapat mengelola data tim (<i>Team</i>) dan anggota tim (<i>Team Member</i>) untuk keperluan kompetisi		
13	Sistem dapat mengelola data prestasi (<i>Achievement</i>) beserta status persetujuan		
14	Sistem dapat mengelola data pengajuan dana (<i>Fund Application</i>) beserta status persetujuan		
15	Sistem dapat mengelola data testimoni (<i>Testimonial</i>) dan galeri proyek (<i>Project Gallery</i>)		
16	Sistem dapat mengelola hak akses pengguna melalui grup (<i>Group</i>) dan izin (<i>Permission</i>)		
17	Sistem dapat mengelola konfigurasi sistem (<i>Config</i>) termasuk pengaturan daftar ulang keanggotaan		
18	Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan		
19	Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (<i>Pending/Accepted/Rejected</i>)		

No	Pernyataan	Penilaian	
		Ya	Tidak
20	Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar		
21	Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data		
22	Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar		
23	Sistem menampilkan halaman pengaturan (<i>Settings</i>) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi		

No	Pernyataan	Penilaian				
		STS	TS	N	S	SS
1	Sistem membantu saya menjadi lebih efektif					
2	Sistem membantu saya menjadi lebih produktif					
3	Sistem ini berguna					
4	Sistem memberi saya lebih banyak kendali atas aktivitas saya					
5	Sistem mempermudah penyelesaian hal-hal yang ingin saya selesaikan					
6	Sistem menghemat waktu saya saat menggunakannya					
7	Sistem ini memenuhi kebutuhan saya					
8	Sistem melakukan semua hal yang saya harapkan					
9	Sistem ini mudah digunakan					
10	Sistem ini simpel untuk digunakan					
11	Sistem ini ramah pengguna (<i>user friendly</i>)					
12	Sistem hanya membutuhkan sedikit langkah untuk menyelesaikan apa yang ingin saya lakukan					
13	Sistem ini fleksibel					
14	Menggunakan sistem ini tidak membutuhkan banyak usaha					
15	Saya dapat menggunakan sistem ini tanpa petunjuk tertulis					
16	Saya tidak menemukan adanya inkonsistensi saat menggunakan sistem ini					

No	Pernyataan	Penilaian				
		STS	TS	N	S	SS
17	Pengguna awam (<i>occasional</i>) maupun pengguna rutin (<i>regular</i>) akan menyukai sistem ini					
18	Saya dapat memulihkan (memperbaiki) kesalahan dengan cepat dan mudah					
19	Saya dapat menggunakan sistem ini dengan sukses setiap saat					
20	Saya belajar menggunakan sistem ini dengan cepat					
21	Saya mudah mengingat cara menggunakan sistem ini					
22	Sistem ini mudah dipelajari cara penggunaannya					
23	Saya dengan cepat menjadi mahir menggunakan sistem ini					
24	Saya merasa puas dengan sistem ini					
25	Saya akan merekomendasikan sistem ini kepada teman					
26	Sistem ini menyenangkan untuk digunakan					
27	Sistem bekerja sesuai dengan yang saya inginkan					
28	Sistem ini luar biasa					
29	Saya merasa perlu memiliki sistem ini					
30	Sistem ini menyenangkan/nyaman digunakan					

Keterangan:

- STS : Sangat Tidak Setuju
- TS : Tidak Setuju
- N : Netral
- S : Setuju
- SS : Sangat Setuju

Lampiran 3. Instrumen *Quality in Use*

No	Pernyataan	Penilaian				
		STS	TS	N	S	SS
1	Sistem memberikan informasi yang tepat seperti yang saya butuhkan					
2	Konten informasi pada sistem memenuhi kebutuhan saya					
3	Sistem menyediakan laporan (data keluaran) yang hampir persis seperti yang saya butuhkan					
4	Sistem memberikan informasi yang memadai					
5	Sistem ini menghasilkan data yang akurat					
6	Saya puas dengan keakuratan sistem ini					
7	Keluaran (<i>output</i>) dari sistem disajikan dalam format yang berguna					
8	Informasi pada sistem disajikan dengan jelas					
9	Sistem ini ramah pengguna (<i>user friendly</i>)					
10	Sistem ini mudah digunakan					
11	Saya mendapatkan informasi yang saya butuhkan secara tepat waktu					
12	Sistem menyediakan informasi yang mutakhir atau terbaru					

Keterangan:

- STS : Sangat Tidak Setuju
- TS : Tidak Setuju
- N : Netral
- S : Setuju
- SS : Sangat Setuju

Lampiran 4. Surat Pernyataan Validasi Instrumen

SURAT PERNYATAAN VALIDASI INSTRUMEN PENELITIAN TUGAS AKHIR

Saya yang bertanda tangan di bawah ini:

Nama : Muhammad Resa Arif Yudianto, M.Kom.
NIP : 199603022024061001
Jabatan : Asisten Ahli (III/b)
Departemen : Departemen Pendidikan Teknik Elektronika dan Informatika

menyatakan bahwa instrumen penelitian TA atas nama mahasiswa:

Nama : Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM : 22051130001
Program Studi : Teknologi Informasi
Judul TA : Pengembangan Sistem Informasi Organisasi INFINITE dengan
Laravel *Headless*, Next.js, dan Kontainer

Setelah dilakukan kajian atas instrumen penelitian TA tersebut dapat dinyatakan:

- ☒ Layak digunakan untuk penelitian
- ☐ Layak digunakan dengan revisi
- ☐ Tidak layak digunakan untuk penelitian

dengan catatan dan saran/perbaikan sebagaimana terlampir.

Demikian agar dapat digunakan sebagaimana mestinya.

Yogyakarta, 31 Mei 2026

Validator,

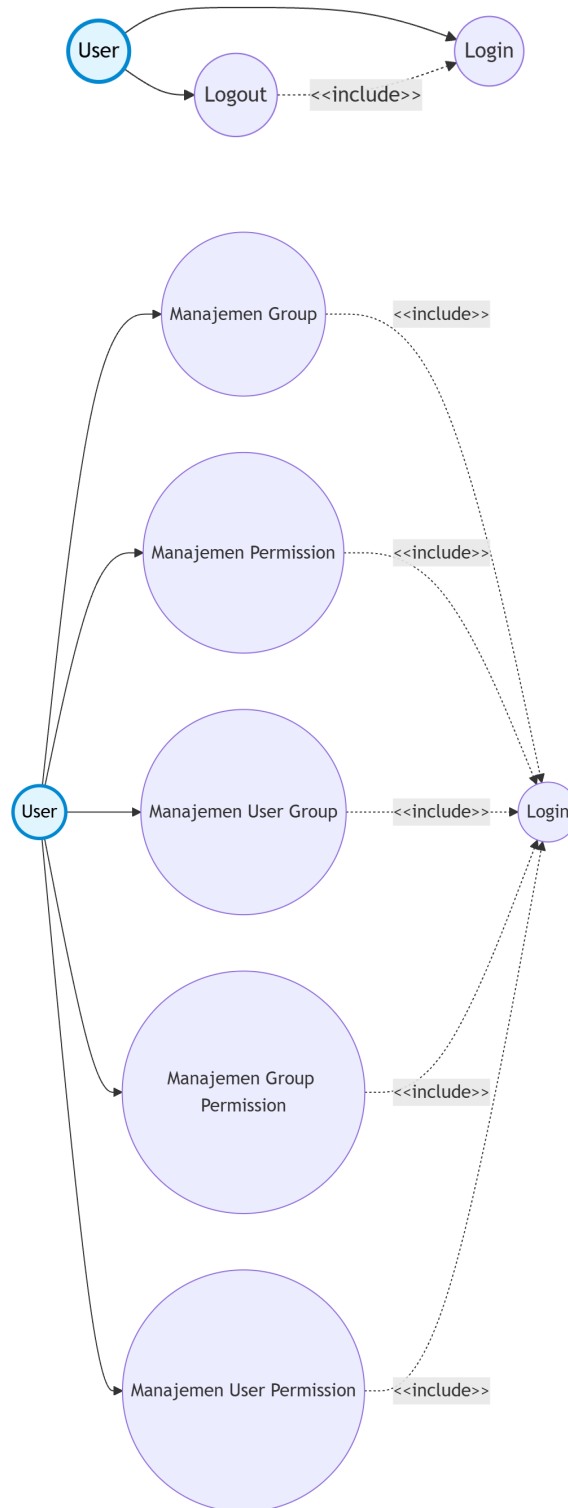


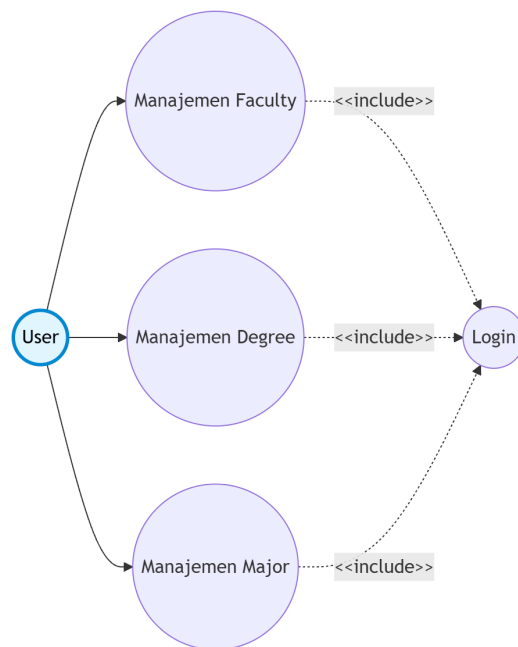
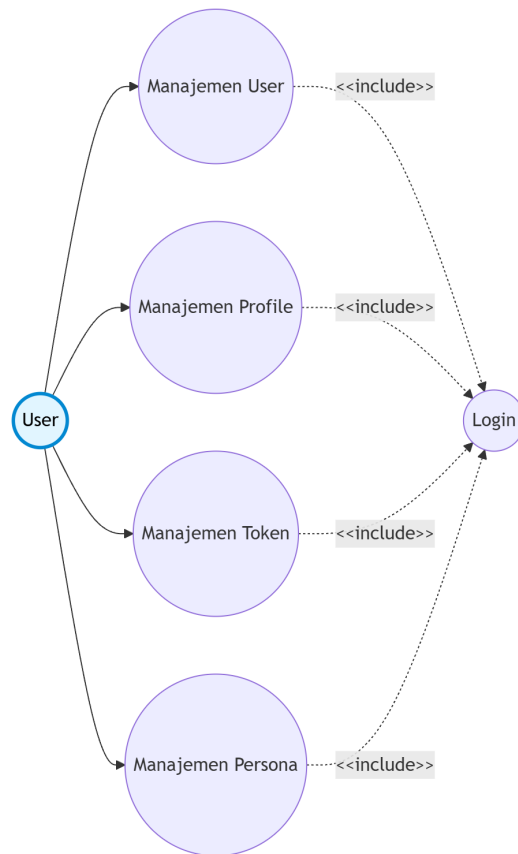
Muhammad Resa Arif Yudianto, M.Kom.

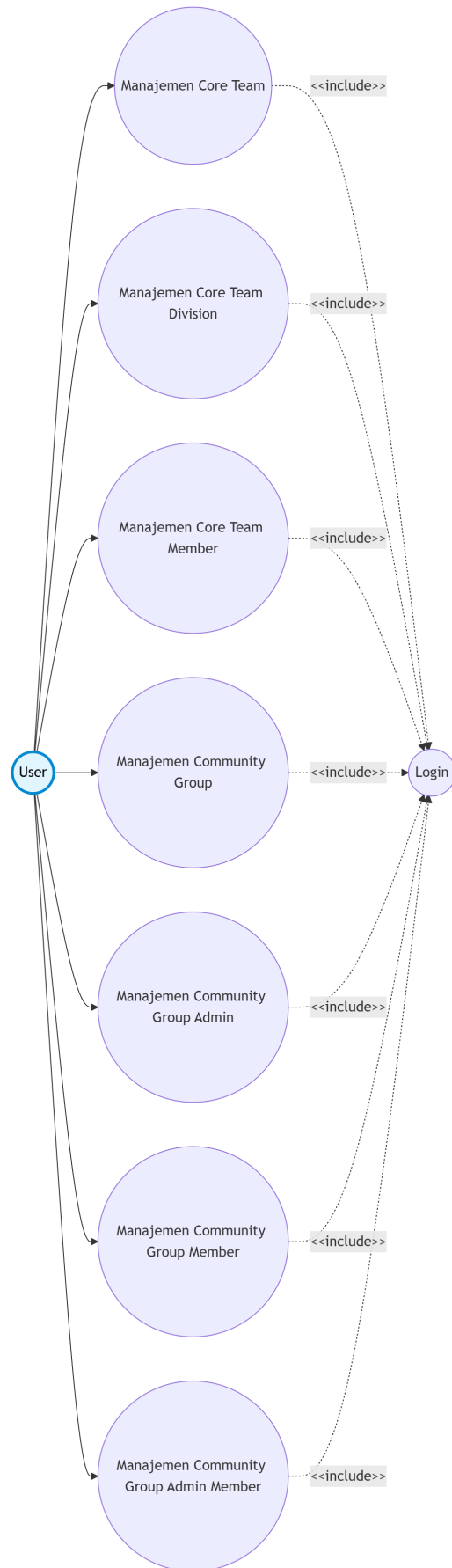
NIP. 199603022024061001

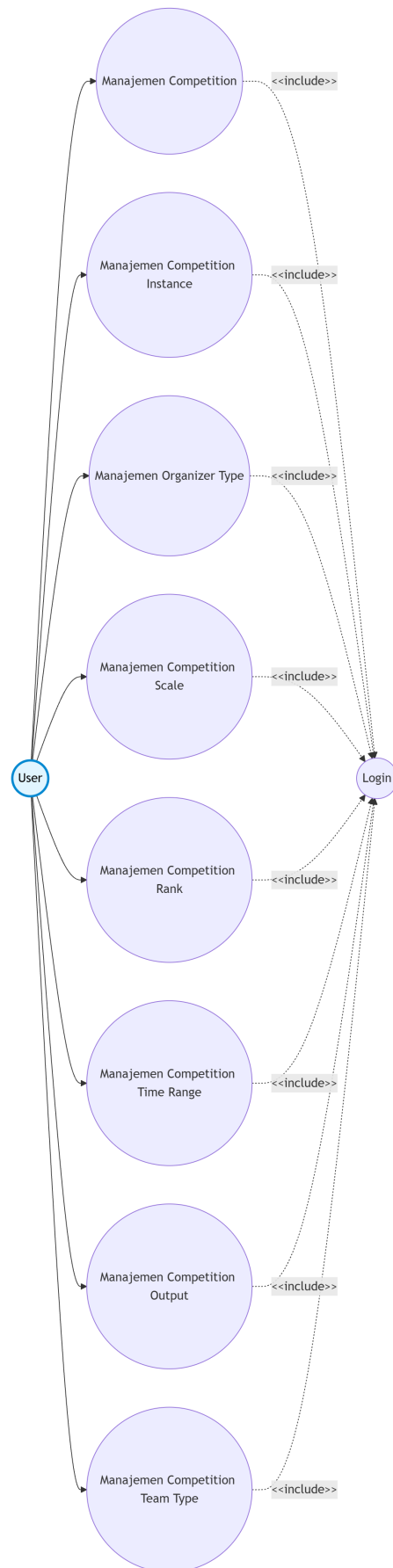
Lampiran 5. Use Case Diagram Aktor Terautentikasi

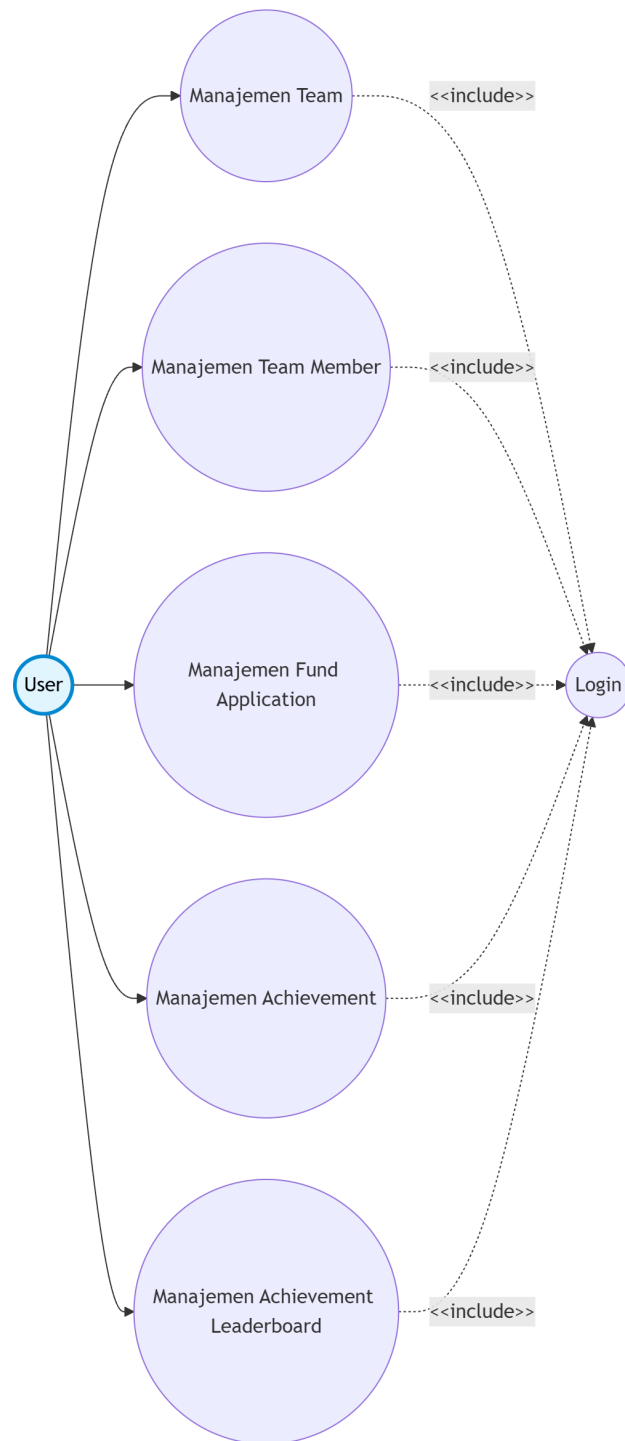
Use case diagram dipecah dalam beberapa diagram untuk memudahkan pembacaan, karena diagram keseluruhan akan menjadi terlalu besar. Setiap diagram menampilkan aktor dan fitur sistem yang saling berinteraksi sesuai dengan domain fitur yang ada.

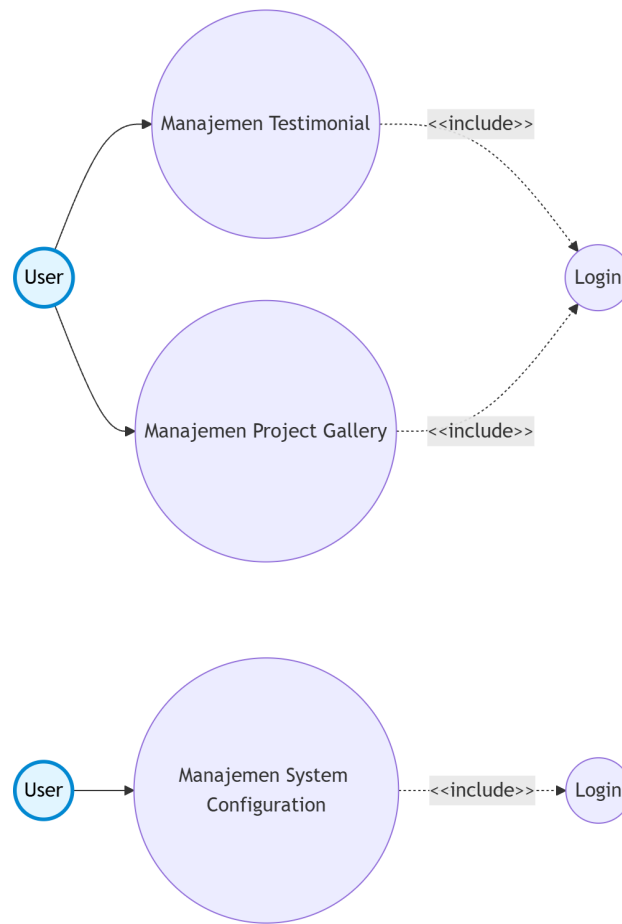












Lampiran 6. Use Case Description

No	Use Case	Deskripsi
1	Login	Berfungsi untuk masuk ke dalam sistem menggunakan kredensial SSO INFINITE yang valid.
2	Logout	Berfungsi untuk keluar dari sistem dan mengakhiri sesi autentikasi pengguna.
3	Manajemen Group	Berfungsi untuk pengelolaan data grup pengguna untuk pengelompokan hak akses.
4	Manajemen Permission	Berfungsi untuk pengelolaan data izin akses yang dapat diberikan kepada grup atau pengguna.
5	Manajemen User Group	Berfungsi untuk pengelolaan keanggotaan grup dari setiap pengguna dalam sistem.
6	Manajemen Group Permission	Berfungsi untuk pengelolaan izin akses yang diberikan kepada setiap grup.
7	Manajemen User Permission	Berfungsi untuk pengelolaan izin akses khusus yang diberikan langsung kepada pengguna tertentu.
8	Manajemen Faculty	Berfungsi untuk pengelolaan data fakultas yang tersedia dalam sistem.
9	Manajemen Degree	Berfungsi untuk pengelolaan data strata pendidikan (S1, S2, S3, D3, D4).
10	Manajemen Major	Berfungsi untuk pengelolaan data program studi yang tersedia dalam sistem.
11	Manajemen User	Berfungsi untuk pengelolaan data anggota organisasi termasuk informasi profil dan status keanggotaan.
12	Manajemen Profile	Berfungsi untuk pengelolaan data profil pribadi termasuk informasi kontak dan preferensi.
13	Manajemen Token	Berfungsi untuk pengelolaan token akses API untuk integrasi dengan sistem eksternal.
14	Manajemen Persona	Berfungsi untuk pengelolaan kategori anggota (persona) untuk klasifikasi dan segmentasi pengguna.
15	Manajemen Core Team	Berfungsi untuk pengelolaan data kepengurusan inti organisasi untuk periode tertentu.
16	Manajemen Core Team Division	Berfungsi untuk pengelolaan divisi atau departemen dalam struktur kepengurusan inti.
17	Manajemen Core Team Member	Berfungsi untuk pengelolaan anggota kepengurusan inti termasuk penempatan divisi dan foto.

No	Use Case	Deskripsi
18	Manajemen Community Group	Berfungsi untuk pengelolaan kelompok komunitas yang merupakan subdivisi dari organisasi utama.
19	Manajemen Community Group Admin	Berfungsi untuk pengelolaan administrator untuk setiap kelompok komunitas.
20	Manajemen Community Group Member	Berfungsi untuk pengelolaan keanggotaan dalam setiap kelompok komunitas.
21	Manajemen Community Group Admin Member	Berfungsi untuk pengelolaan anggota administrator dalam setiap kelompok komunitas.
22	Manajemen Competition	Berfungsi untuk pengelolaan data seri kompetisi yang diselenggarakan oleh organisasi.
23	Manajemen Competition Instance	Berfungsi untuk pengelolaan edisi pelaksanaan kompetisi dari suatu seri kompetisi.
24	Manajemen Organizer Type	Berfungsi untuk pengelolaan jenis penyelenggara kompetisi (internal, eksternal, kolaborasi).
25	Manajemen Competition Scale	Berfungsi untuk pengelolaan skala kompetisi (lokal, regional, nasional, internasional).
26	Manajemen Competition Rank	Berfungsi untuk pengelolaan peringkat atau pencapaian dalam kompetisi.
27	Manajemen Competition Time Range	Berfungsi untuk pengelolaan rentang waktu pelaksanaan kompetisi.
28	Manajemen Competition Output	Berfungsi untuk pengelolaan luaran atau hasil yang dihasilkan dari kompetisi.
29	Manajemen Competition Team Type	Berfungsi untuk pengelolaan jenis tim yang dapat berpartisipasi dalam kompetisi (individu, tim).
30	Manajemen Team	Berfungsi untuk pengelolaan data tim yang terdaftar dalam sistem untuk keperluan kompetisi.
31	Manajemen Team Member	Berfungsi untuk pengelolaan keanggotaan tim termasuk peran dan kontribusi anggota.
32	Manajemen Achievement	Berfungsi untuk pengelolaan data prestasi anggota beserta status persetujuan.
33	Manajemen Achievement Leaderboard	Berfungsi untuk pengelolaan papan peringkat prestasi untuk menampilkan pencapaian terbaik.
34	Manajemen Fund Application	Berfungsi untuk pengelolaan pengajuan dana dari anggota beserta status persetujuan.

No	<i>Use Case</i>	Deskripsi
35	Manajemen Testimonial	Berfungsi untuk pengelolaan testimoni dari anggota tentang pengalaman mereka dengan organisasi.
36	Manajemen Project Gallery	Berfungsi untuk pengelolaan galeri proyek yang menampilkan hasil karya anggota organisasi.
37	Manajemen System Configuration	Berfungsi untuk pengelolaan konfigurasi sistem termasuk pengaturan daftar ulang keanggotaan.
38	Lihat Competition	Berfungsi untuk menampilkan daftar seri kompetisi yang tersedia tanpa perlu autentikasi.
39	Lihat Competition Instance	Berfungsi untuk menampilkan detail edisi pelaksanaan kompetisi tanpa perlu autentikasi.
40	Lihat Core Team	Berfungsi untuk menampilkan struktur kepengurusan inti organisasi tanpa perlu autentikasi.
41	Lihat Community Group	Berfungsi untuk menampilkan daftar kelompok komunitas tanpa perlu autentikasi.
42	Lihat Achievement	Berfungsi untuk menampilkan daftar prestasi anggota tanpa perlu autentikasi.
43	Lihat Achievement Leaderboard	Berfungsi untuk menampilkan papan peringkat prestasi tanpa perlu autentikasi.
44	Lihat Testimonial	Berfungsi untuk menampilkan testimoni anggota tanpa perlu autentikasi.
45	Lihat Project Gallery	Berfungsi untuk menampilkan galeri proyek tanpa perlu autentikasi.

Lampiran 7. Kode Program Migrasi Tabel *users*

```
1 <?php
2
3 return new class extends Migration
4 {
5     /**
6      * Run the migrations.
7      */
8     public function up(): void
9     {
10         Schema::create('users', function (Blueprint $table) {
11             $table->uuid('id')->primary();
12             $table->string('sso_id')->unique()->nullable();
13             $table->string('name')->fulltext();
14             $table->string('username')->unique();
15             $table->string('email_address')->unique();
16             $table->string('phone_number')->unique();
17             $table->string('student_id')->unique();
18             $table->foreignUuid('major_id')->constrained()->restrictOnUpdate
19             ()->restrictOnDelete();
20             $table->json('links');
21             $table->date('start_date')->nullable();
22             $table->date('end_date')->nullable();
23             $table->boolean('is_member');
24             $table->boolean('is_extraordinary')->default(false);
25             $table->timestamp('sso_last_synced_at')->nullable();
26             $table->timestamps();
27         });
28         Schema::create('tokens', function (Blueprint $table) {
29             $table->uuid('id')->primary();
30             $table->string('sso_id')->unique();
31             $table->foreignUuid('user_id')->constrained()->restrictOnUpdate()
32             ->cascadeOnDelete();
33             $table->timestamp('last_used_at');
34             $table->softDeletes('deleted_at');
35             $table->timestamp('created_at');
36             $table->timestamp('expires_at');
37         });
38     }
39
40     /**
41      * Reverse the migrations.
42      */
43     public function down(): void
44     {
45         Schema::dropIfExists('users');
46         Schema::dropIfExists('tokens');
```

Lampiran 8. Kode Program *seeder* untuk Data *groups*

```
1 <?php
2
3 class GroupSeeder extends Seeder
4 {
5     use WithoutModelEvents;
6
7     /**
8      * Run the database seeds.
9      */
```

```

10 public function run(): void
11 {
12     $groups = [
13         [
14             'name' => 'Administrator',
15             'is_managed' => true,
16         ],
17         [
18             'name' => 'Member',
19             'is_managed' => true,
20         ],
21         [
22             'name' => 'Active Member',
23             'is_managed' => true,
24         ],
25         [
26             'name' => 'Inactive Member',
27             'is_managed' => true,
28         ],
29     ];
30
31     foreach ($groups as $group) {
32         Group::firstOrCreate([
33             ...$group,
34             'guard_name' => Guard::getDefaultName(User::class),
35         ]);
36     }
37 }
38 }

```

Lampiran 9. Kode Program *Model User*

```

1 <?php
2
3 class User extends Model implements AuthenticatableContract,
4     AuthorizableContract, MustVerifyEmailContract
5 {
6     use Authenticatable, Authorizable, MustVerifyEmail;
7     use HasFactory, HasGroups, HasUuids, Notifiable;
8
9     /**
10      * The attributes that are mass assignable.
11      *
12      * @var array<int, string>
13      */
14     protected $fillable = [
15         'name',
16         'username',
17         'email_address',
18         'phone_number',
19         'student_id',
20         'major_id',
21         'links',
22         'start_date',
23         'end_date',
24         'is_member',
25         'is_extraordinary',
26     ];
27
28     /**
29      * The accessors to append to the model's array form.
30      */

```

```

30     * @var array
31     */
32     protected $appends = [
33         'is_active',
34     ];
35
36     protected function isActive(): Attribute
37     {
38         return new Attribute(
39             get: fn () => $this->is_member && $this->start_date <= Carbon::
now() &&
40                 $this->end_date >= Carbon::now(),
41         );
42     }
43
44     /**
45     * Prepare a date for array / JSON serialization.
46     */
47     protected function serializeDate(DateTimeInterface $date): string
48     {
49         return $date->format(DATE_ATOM);
50     }
51
52     public function major(): BelongsTo
53     {
54         return $this->belongsTo(Major::class);
55     }
56
57     public function personas(): BelongsToMany
58     {
59         return $this->belongsToMany(
60             Persona::class,
61             'user_personas',
62             'user_id',
63             'persona_id',
64         )
65             ->using(UserPersona::class)
66             ->as('entitlement')
67             ->withPivot([
68                 'id',
69             ])
70             ->withTimestamps();
71     }
72
73     public function teams(): BelongsToMany
74     {
75         return $this->belongsToMany(
76             Team::class,
77             'team_members',
78             'user_id',
79             'team_id',
80         )
81             ->using(TeamMember::class)
82             ->as('membership')
83             ->withPivot([
84                 'id',
85             ])
86             ->withTimestamps();
87     }
88
89     public function coreTeams(): BelongsToMany

```

```

90     {
91         return $this->belongsToMany(
92             CoreTeam::class,
93             'core_team_members',
94             'user_id',
95             'core_team_id',
96         )
97         ->using(CoreTeamMember::class)
98         ->as('membership')
99         ->withPivot([
100             'id',
101             'core_team_division_id',
102             'photo',
103             'animation',
104         ])
105         ->withTimestamps();
106     }
107
108     public function communityGroups(): BelongsToMany
109     {
110         return $this->belongsToMany(
111             CommunityGroup::class,
112             'community_group_members',
113             'user_id',
114             'community_group_id',
115         )
116         ->using(CommunityGroupMember::class)
117         ->as('membership')
118         ->withPivot([
119             'id',
120         ])
121         ->withTimestamps();
122     }
123
124     public function communityGroupAdmins(): BelongsToMany
125     {
126         return $this->belongsToMany(
127             CommunityGroupAdmin::class,
128             'community_group_admin_members',
129             'user_id',
130             'community_group_admin_id',
131         )
132         ->using(CommunityGroupAdminMember::class)
133         ->as('membership')
134         ->withPivot([
135             'id',
136             'community_group_id',
137             'photo',
138             'animation',
139         ])
140         ->withTimestamps();
141     }
142
143     public function ledTeams(): HasMany
144     {
145         return $this->hasMany(Team::class, 'leader_id');
146     }
147 }

```

Lampiran 10. Kode Program *Custom Resource*

1 <?php

```

2
3 class Resource extends JsonResource
4 {
5     protected $resourceName = 'resource';
6
7     /**
8      * Transform the resource into an array.
9      *
10     * @return array<string, mixed>
11     */
12     public function toArray(Request $request): array
13     {
14         return ResponseFormatter::singleton(
15             $this->resourceName,
16             $this->toBaseArray($request),
17         );
18     }
19
20     /**
21     * Transform the resource into an array.
22     *
23     * @return array<string, mixed>
24     */
25     public function toBaseArray(Request $request): array
26     {
27         return $this->resource->toArray();
28     }
29 }

```

Lampiran 11. Kode Program *Custom Resource Collection*

```

1 <?php
2
3 class Collection extends ResourceCollection
4 {
5     protected $collectionName = 'resources';
6
7     /**
8      * Transform the resource collection into an array.
9      *
10     * @return array<int|string, mixed>
11     */
12     public function toArray(Request $request): array
13     {
14         return ResponseFormatter::collection(
15             $this->collectionName,
16             $this->collection->map->toBaseArray($request)->all(),
17         );
18     }
19
20     /**
21     * Customize the pagination information for the resource.
22     */
23     public function paginationInformation(Request $request, array $paginated,
24         array $default): array
25     {
26         $default = [
27             'data' => [
28                 'meta' => [
29                     'per_page' => $paginated['per_page'],
30                     'prev_cursor' => $paginated['prev_cursor'],
31                     'next_cursor' => $paginated['next_cursor'],

```



```

31         ],
32     ],
33 ];
34
35     return $default;
36 }
37 }

```

Lampiran 12. Kode Program *Resource* untuk *Model User*

```

1 <?php
2
3 class UserResource extends Resource
4 {
5     protected $resourceName = 'user';
6
7     /**
8      * Transform the resource into an array.
9      *
10     * @return array<string, mixed>
11     */
12     public function toBaseArray(Request $request): array
13     {
14         $user = $this->resource->toArray();
15
16         if (! Auth::guard(config('auth.defaults.semi_public_guard'))->user())
17         {
18             $classifiedFields = [
19                 'email_address',
20                 'phone_number',
21                 'student_id',
22             ];
23
24             foreach ($classifiedFields as $field) {
25                 if (isset($user[$field])) {
26                     $user[$field] = 'REDACTED';
27                 }
28             }
29
30             return $user;
31         }
32     }

```

Lampiran 13. Kode Program *Resource Collection* untuk *Model User*

```

1 <?php
2
3 class UserCollection extends Collection
4 {
5     protected $collectionName = 'users';
6 }

```

Lampiran 14. Kode Program *Controller* *UserController*

```

1 <?php
2
3 /**
4  * @group Users
5  * Manage users.
6  */
7 class UserController extends Controller
8 {

```

```

9 public function __construct()
10 {
11     $this->middleware('can:create,.User::class')->only('store');
12     $this->middleware('can:update,user')->only('update');
13     $this->middleware('can:delete,user')->only('destroy');
14 }
15
16 /**
17  * List all users
18  *
19  * @unauthenticated
20  *
21  * @apiResourceCollection App\Http\Resources\User\UserCollection
22  *
23  * @apiResourceModel App\Models\User paginate=10,cursor
24  */
25 public function index(Request $request)
26 {
27     $users = QueryBuilder::for(User::class)
28         ->allowedFields(
29             'id',
30             'sso_id',
31             'name',
32             'username',
33             'email_address',
34             'phone_number',
35             'student_id',
36             'major_id',
37             'links',
38             'start_date',
39             'end_date',
40             'is_member',
41             'is_extraordinary',
42             'created_at',
43             'updated_at',
44         )
45         ->allowedIncludes(
46             'major',
47             'major.degree',
48             'major.faculty',
49             'personas',
50             'groups',
51             'permissions',
52         )
53         ->allowedFilters(
54             AllowedFilter::exact('sso_id'),
55             'name',
56             'username',
57             'email_address',
58             'phone_number',
59             'student_id',
60             AllowedFilter::exact('major_id'),
61             AllowedFilter::operator('start_date', FilterOperator::DYNAMIC)
62         ,
63             AllowedFilter::operator('end_date', FilterOperator::DYNAMIC),
64             AllowedFilter::exact('is_member'),
65             AllowedFilter::exact('is_extraordinary'),
66             AllowedFilter::operator('created_at', FilterOperator::DYNAMIC)
67         ,
68             AllowedFilter::operator('updated_at', FilterOperator::DYNAMIC)
69         ,

```

```

67         )
68         ->allowedSorts(
69             'id',
70             'name',
71             'username',
72             'email_address',
73             'phone_number',
74             'student_id',
75             'major_id',
76             'start_date',
77             'end_date',
78             'is_member',
79             'is_extraordinary',
80             'created_at',
81             'updated_at',
82         )
83         ->defaultSorts('-id')
84         ->cursorPaginate($request->query('per_page', 10));
85
86         return new UserCollection($users);
87     }
88
89     /**
90      * Create a user
91      *
92      * @apiResource App\Http\Resources\User\UserResource status=201
93      *
94      * @apiResourceModel App\Models\User
95      *
96      * @bodyParam links object required Example: {"github": "https://github.
com/infiniteuny", "linkedin": "https://linkedin.com/company/infiniteuny/",
"website": "https://infiniteuny.id"}
97      * @bodyParam links.* string Example: https://example.com
98      */
99     public function store(StoreUserRequest $request)
100     {
101         $data = $request->validated();
102         $data['links'] = $data['links'] ?? [];
103         $data['is_member'] ??= false;
104         $data['is_extraordinary'] ??= false;
105
106         $user = User::create($data);
107
108         CreateSsoUser::dispatch($user);
109
110         return new UserResource($user);
111     }
112
113     /**
114      * Retrieve a user
115      *
116      * @unauthenticated
117      *
118      * @apiResource App\Http\Resources\User\UserResource
119      *
120      * @apiResourceModel App\Models\User
121      */
122     public function show(User $user)
123     {
124         $user = QueryBuilder::for(User::where('id', $user->id))
125             ->allowedFields(

```

```

126         'id',
127         'sso_id',
128         'name',
129         'username',
130         'email_address',
131         'phone_number',
132         'student_id',
133         'major_id',
134         'links',
135         'start_date',
136         'end_date',
137         'is_member',
138         'is_extraordinary',
139         'created_at',
140         'updated_at',
141     )
142     ->allowedIncludes(
143         'major',
144         'major.degree',
145         'major.faculty',
146         'personas',
147         'groups',
148         'permissions',
149     )
150     ->firstOrFail();
151
152     return new UserResource($user);
153 }
154
155 /**
156  * Update a user
157  *
158  * @apiResource App\Http\Resources\User\UserResource
159  *
160  * @apiResourceModel App\Models\User
161  *
162  * @bodyParam links object Example: {"github": "https://github.com/
infiniteuny", "linkedin": "https://linkedin.com/company/infiniteuny/", "
website": "https://infiniteuny.id"}
163  * @bodyParam links.* string Example: https://example.com
164  */
165 public function update(UpdateUserRequest $request, User $user)
166 {
167     $user->update($request->validated());
168
169     if ($user->sso_id) {
170         UpdateSsoUser::dispatch($user);
171     } else {
172         CreateSsoUser::dispatch($user);
173     }
174
175     return new UserResource($user);
176 }
177
178 /**
179  * Delete a user
180  *
181  * @apiResource App\Http\Resources\User\UserResource
182  *
183  * @apiResourceModel App\Models\User
184  */

```

```

185     public function destroy(User $user)
186     {
187         $user->delete();
188
189         if ($user->sso_id) {
190             DeleteSsoUser::dispatch($user->sso_id);
191         }
192
193         return new UserResource($user);
194     }
195 }

```

Lampiran 15. Kode Program *Service OidcService*

```

1 <?php
2
3 class OidcServiceImpl implements OidcService
4 {
5     protected IssuerInterface $issuer;
6
7     protected ClientInterface $client;
8
9     protected TokenVerifierInterface $accessTokenVerifier;
10
11     protected IntrospectionService $introspectionService;
12
13     protected UserInfoService $userInfoService;
14
15     public function __construct(
16         private PsrCacheRepository $cacheRepository,
17     ) {
18         $this->getIssuer();
19         $this->getClient();
20         $this->getAccessTokenVerifier();
21         $this->getIntrospectionService();
22         $this->getUserInfoService();
23     }
24
25     protected function getMetadataProviderBuilder(): MetadataProviderBuilder
26     {
27         $builder = (new MetadataProviderBuilder)
28             ->setCache($this->cacheRepository)
29             // Cache metadata for 30 days
30             ->setCacheTtl(2592000);
31
32         return $builder;
33     }
34
35     protected function getJwksProviderBuilder(): JwksProviderBuilder
36     {
37         $builder = (new JwksProviderBuilder)
38             ->withCache($this->cacheRepository)
39             // Cache JWKS for 1 day
40             ->withCacheTtl(86400);
41
42         return $builder;
43     }
44
45     protected function getIssuerBuilder(): IssuerBuilder
46     {
47         $builder = (new IssuerBuilder)
48             ->setMetadataProviderBuilder($this->getMetadataProviderBuilder())

```

```

49         ->setJwksProviderBuilder($this->getJwksProviderBuilder());
50
51     return $builder;
52 }
53
54 protected function getClientBuilder(): ClientBuilder
55 {
56     $builder = (new ClientBuilder)
57         ->setIssuer($this->getIssuer())
58         ->setClientMetadata(ClientMetadata::fromArray([
59             'client_id' => config('oidc.client_id'),
60             'client_secret' => config('oidc.client_secret'),
61         ]));
62
63     return $builder;
64 }
65
66 protected function getAccessTokenVerifierBuilder():
67     AccessTokenVerifierBuilder
68 {
69     return new AccessTokenVerifierBuilder;
70 }
71
72 protected function getIntrospectionServiceBuilder():
73     IntrospectionServiceBuilder
74 {
75     return new IntrospectionServiceBuilder;
76 }
77
78 protected function getUserInfoServiceBuilder(): UserInfoServiceBuilder
79 {
80     return new UserInfoServiceBuilder;
81 }
82
83 protected function getIssuerMetadata(): IssuerMetadataInterface
84 {
85     return $this->getIssuer()->getMetadata();
86 }
87
88 protected function getClientMetadata(): ClientMetadataInterface
89 {
90     return $this->getClient()->getMetadata();
91 }
92
93 protected function getIssuer(): IssuerInterface
94 {
95     if (! empty($this->issuer)) {
96         return $this->issuer;
97     }
98
99     return $this->issuer = $this->getIssuerBuilder()
100         ->build(config('oidc.configurations_uri'));
101 }
102
103 protected function getClient(): ClientInterface
104 {
105     if (! empty($this->client)) {
106         return $this->client;
107     }
108
109     return $this->client = $this->getClientBuilder()

```

```

108         ->build();
109     }
110
111     protected function getAccessTokenVerifier(): TokenVerifierInterface
112     {
113         if (! empty($this->accessTokenVerifier)) {
114             return $this->accessTokenVerifier;
115         }
116
117         return $this->accessTokenVerifier = $this->
118         getAccessTokenVerifierBuilder()
119         ->build($this->getClient());
120     }
121
122     protected function getIntrospectionService(): IntrospectionService
123     {
124         if (! empty($this->introspectionService)) {
125             return $this->introspectionService;
126         }
127
128         return $this->introspectionService = $this->
129         getIntrospectionServiceBuilder()
130         ->build();
131     }
132
133     protected function getUserInfoService(): UserInfoService
134     {
135         if (! empty($this->userInfoService)) {
136             return $this->userInfoService;
137         }
138
139         return $this->userInfoService = $this->getUserInfoServiceBuilder()
140         ->build();
141     }
142
143     /**
144      * @throws InvalidTokenException
145      */
146     public function verify(string $token): array
147     {
148         $verifier = $this->getAccessTokenVerifier();
149
150         return $verifier->verify($token);
151     }
152
153     public function introspect(string $token): array
154     {
155         $service = $this->getIntrospectionService();
156
157         return $service->introspect($this->client, $token);
158     }
159
160     public function getUserInfo(string $token): array
161     {
162         $service = $this->getUserInfoService();
163
164         return $service->getUserInfo($this->client, TokenSet::fromParams(['
165         access_token' => $token]));
166     }
167 }

```

Lampiran 16. Kode Program *Service SsoService*

```
1 <?php
2
3 class SsoServiceImpl implements SsoService
4 {
5     public function listUsers(array $filters = []): Collection
6     {
7         $response = Http::authentik()
8             ->get('/api/v3/core/users/', $filters);
9
10        $users = SsoUserData::collect($response->json('results', []),
11            Collection::class);
12
13        return $users;
14    }
15
16    public function createUser(SsoUserData $ssoUserData): SsoUserData
17    {
18        $response = Http::authentik()
19            ->post('/api/v3/core/users/', $ssoUserData->toArray());
20
21        return SsoUserData::from($response->json());
22    }
23
24    public function getUser(string $userId): SsoUserData
25    {
26        $response = Http::authentik()
27            ->get("/api/v3/core/users/$userId/");
28
29        return SsoUserData::from($response->json());
30    }
31
32    public function updateUser(string $userId, SsoUserData $ssoUserData):
33        SsoUserData
34    {
35        $response = Http::authentik()
36            ->patch("/api/v3/core/users/$userId/", $ssoUserData->toArray());
37
38        return SsoUserData::from($response->json());
39    }
40
41    public function deleteUser(string $userId): bool
42    {
43        $response = Http::authentik()
44            ->delete("/api/v3/core/users/$userId/");
45
46        return $response->successful();
47    }
48
49    public function listGroups(array $filters = []): Collection
50    {
51        $response = Http::authentik()
52            ->get('/api/v3/core/groups/', $filters);
53
54        $groups = SsoGroupData::collect($response->json('results', []),
55            Collection::class);
56
57        return $groups;
58    }
59 }
```



```

57 public function createGroup(SsoGroupData $ssoGroupData): SsoGroupData
58 {
59     $response = Http::authentik()
60         ->post('/api/v3/core/groups/', $ssoGroupData->toArray());
61
62     return SsoGroupData::from($response->json());
63 }
64
65 public function getGroup(string $groupId): SsoGroupData
66 {
67     $response = Http::authentik()
68         ->get("/api/v3/core/groups/$groupId/");
69
70     return SsoGroupData::from($response->json());
71 }
72
73 public function updateGroup(string $groupId, SsoGroupData $ssoGroupData):
74     SsoGroupData
75 {
76     $response = Http::authentik()
77         ->patch("/api/v3/core/groups/$groupId/", $ssoGroupData->toArray()
78 );
79
80     return SsoGroupData::from($response->json());
81 }
82
83 public function deleteGroup(string $groupId): bool
84 {
85     $response = Http::authentik()
86         ->delete("/api/v3/core/groups/$groupId/");
87
88     return $response->successful();
89 }
90 }

```

Lampiran 17. Kode Program *Repository StorageRepository*

```

1 <?php
2
3 class StorageRepositoryImpl implements StorageRepository
4 {
5     /**
6      * @throws Exception
7      */
8     public function store(
9         UploadedFile $file,
10         string $path,
11         StorageVisibility $visibility = StorageVisibility::PRIVATE,
12         ?string $disk = null,
13         ?string $name = null,
14     ): string {
15         if (is_null($name)) {
16             $name = Uuid::v7().'.'.$file->extension();
17         } else {
18             $name = Str::ascii($name);
19         }
20
21         if (is_null($disk)) {
22             $disk = config('filesystems.default');
23         }
24
25         $visibility = $visibility->value;

```

```

26     $path = trim($visibility.'/'.$path, '/');
27
28     $result = Storage::disk($disk)->putFileAs($path, $file, $name,
29     $visibility);
30
31     if ($result) {
32         return json_encode([
33             'disk' => $disk,
34             'visibility' => $visibility,
35             'path' => $path,
36             'name' => $name,
37         ]);
38     } else {
39         throw new Exception('Failed to store file');
40     }
41
42     public function url(string $manifest): string
43     {
44         $params = json_decode($manifest);
45         $disk = $params->disk;
46         $visibility = $params->visibility;
47         $path = $params->path;
48         $name = $params->name;
49
50         if ($visibility === StorageVisibility::PRIVATE->value) {
51             // For private files, we need to generate a temporary URL
52             // with a specific expiration time
53             $expiration = Carbon::now()->addHours(6);
54
55             return Storage::disk($disk)->temporaryUrl($path.'/'.$name,
56             $expiration);
57         } else {
58             return Storage::disk($disk)->url($path.'/'.$name);
59         }
60
61     public function delete(string $manifest): bool
62     {
63         $params = json_decode($manifest);
64         $disk = $params->disk;
65         $path = $params->path;
66         $name = $params->name;
67
68         if (Storage::disk($disk)->exists($path.'/'.$name)) {
69             return Storage::disk($disk)->delete($path.'/'.$name);
70         } else {
71             throw new NotFoundException('File not found.');
```

Lampiran 18. Kode Program *Guard* OidcTokenGuard

```

1 <?php
2
3 class OidcTokenGuard implements Guard
4 {
5     use GuardHelpers;
6
7     protected Request $request;
8
```

```

9     protected OidcService $oidcService;
10
11     /**
12      * Create a new authentication guard.
13      *
14      * @return void
15      */
16     public function __construct(
17         UserProvider $provider,
18         OidcService $oidcService,
19         Request $request
20     ) {
21         $this->request = $request;
22         $this->provider = $provider;
23         $this->oidcService = $oidcService;
24         $this->user = null;
25     }
26
27     /**
28      * Get the currently authenticated user.
29      */
30     public function user(): ?Authenticatable
31     {
32         if (! is_null($this->user)) {
33             return $this->user;
34         }
35
36         if ($this->validate()) {
37             return $this->user;
38         }
39
40         return $this->user;
41     }
42
43     /**
44      * Validate a user's credentials.
45      */
46     public function validate(array $credentials = []): bool
47     {
48         if (empty($credentials['token'])) {
49             if (! $credentials['token'] = $this->getAccessToken()) {
50                 return false;
51             }
52         }
53
54         // Verify the token signature
55         try {
56             $tokenPayload = $this->oidcService->verify($credentials['token'])
57         ;
58         } catch (Throwable $e) {
59             return false;
60         }
61
62         // Check if the token subject exists in the users database
63         $user = $this->provider->retrieveByCredentials(['sso_id' =>
64             $tokenPayload['sub']]);
65
66         // If the user is not found, try to get the user info from the OIDC
67         server
68         if (is_null($user)) {
69             try {

```

```

67         $userInfo = $this->oidcService->getUserInfo($credentials['
token']);
68
69         /** @var User $user */
70         $user = $this->provider->retrieveByCredentials(['
email_address' => $userInfo['email']]);
71
72         if (! is_null($user)) {
73             $user->sso_id = $tokenPayload['sub'];
74             $user->save();
75         } else {
76             return false;
77         }
78     } catch (Throwable $e) {
79         return false;
80     }
81 }
82
83 // Check if the token is already used to authenticate before
84 $token = Token::withTrashed()->where('sso_id', $tokenPayload['uid'])
->first();
85
86 // If the token is not found in the database, check if it is still
active
87 if (is_null($token)) {
88     try {
89         $tokenInfo = $this->oidcService->introspect($credentials['
token']);
90
91         if ($tokenInfo['active']) {
92             $token = new Token([
93                 'sso_id' => $tokenPayload['uid'],
94                 'user_id' => $user->id,
95                 'created_at' => Carbon::createFromTimestamp(
$tokenInfo['iat']),
96                 'expires_at' => Carbon::createFromTimestamp(
$tokenInfo['exp']),
97             ]);
98         } else {
99             return false;
100         }
101     } catch (Throwable $e) {
102         return false;
103     }
104 } else {
105     if ($token->trashed()) {
106         return false;
107     }
108 }
109
110 $token->last_used_at = Carbon::now();
111 $token->save();
112 $this->setUser($user);
113
114 return true;
115 }
116
117 /**
118  * Get the access token from the current request
119  */
120 private function getAccessToken(): ?string

```

```

121     {
122         return $this->request->bearerToken();
123     }
124 }

```

Lampiran 19. Kode Program *Guard* *OidcHeadersGuard*

```

1 <?php
2
3 namespace App\Guards;
4
5 use App\Models\Token;
6 use App\Models\User;
7 use App\Services\OidcService;
8 use Illuminate\Auth\GuardHelpers;
9 use Illuminate\Contracts\Auth\Authenticatable;
10 use Illuminate\Contracts\Auth\Guard;
11 use Illuminate\Contracts\Auth\UserProvider;
12 use Illuminate\Http\Request;
13 use Illuminate\Support\Carbon;
14 use Throwable;
15
16 class OidcHeadersGuard implements Guard
17 {
18     use GuardHelpers;
19
20     protected Request $request;
21
22     protected OidcService $oidcService;
23
24     /**
25      * Create a new authentication guard.
26      *
27      * @return void
28      */
29     public function __construct(
30         UserProvider $provider,
31         OidcService $oidcService,
32         Request $request
33     ) {
34         $this->request = $request;
35         $this->provider = $provider;
36         $this->oidcService = $oidcService;
37         $this->user = null;
38     }
39
40     /**
41      * Get the currently authenticated user.
42      */
43     public function user(): ?Authenticatable
44     {
45         if (! is_null($this->user)) {
46             return $this->user;
47         }
48
49         if ($this->validate()) {
50             return $this->user;
51         }
52
53         return $this->user;
54     }
55 }

```

```

56  /**
57   * Validate a user's credentials.
58   */
59  public function validate(array $credentials = []): bool
60  {
61      if (empty($credentials['headers'])) {
62          if (! $credentials['headers'] = $this->getCredentialHeaders()) {
63              return false;
64          }
65      }
66
67      // Check if the token subject exists in the users database
68      $user = $this->provider->retrieveByCredentials(['sso_id' =>
69          $credentials['headers']['user_id']]);
70
71      // If the user is not found, try to get the user info from the OIDC
72      server
73      if (is_null($user)) {
74          try {
75              /** @var User $user */
76              $user = $this->provider->retrieveByCredentials(['
77                  email_address' => $credentials['headers']['user_email']]);
78
79              if (! is_null($user)) {
80                  $user->sso_id = $credentials['headers']['user_id'];
81                  $user->save();
82              } else {
83                  return false;
84              }
85          } catch (Throwable $e) {
86              return false;
87          }
88      }
89
90      // Check if the token is already used to authenticate before
91      $token = Token::withTrashed()->where('sso_id', $credentials['headers'
92          ]['token_id']->first();
93
94      // If the token is not found in the database, check if it is still
95      active
96      if (is_null($token)) {
97          try {
98              $tokenInfo = $this->oidcService->introspect($credentials['
99                  headers']['token']);
100
101              if ($tokenInfo['active']) {
102                  $token = new Token([
103                      'sso_id' => $credentials['headers']['token_id'],
104                      'user_id' => $user->id,
105                      'created_at' => Carbon::createFromTimestamp(
106                          $tokenInfo['iat']),
107                      'expires_at' => Carbon::createFromTimestamp(
108                          $tokenInfo['exp']),
109                  ]);
110              } else {
111                  return false;
112              }
113          } catch (Throwable $e) {
114              return false;
115          }
116      } else {
117

```

```

109         if ($token->trashed()) {
110             return false;
111         }
112     }
113
114     $token->last_used_at = Carbon::now();
115     $token->save();
116     $this->setUser($user);
117
118     return true;
119 }
120
121 /**
122  * Get the credential headers from the current request
123  */
124 private function getCredentialHeaders(): ?array
125 {
126     $userId = $this->request->header('X-User-Id');
127     $userEmail = $this->request->header('X-User-Email');
128     $tokenId = $this->request->header('X-Token-Id');
129     $tokenCreatedAt = $this->request->header('X-Token-Created-At');
130     $tokenExpiresAt = $this->request->header('X-Token-Expires-At');
131     $token = $this->request->bearerToken();
132
133     if ($userId && $userEmail && $tokenId && $tokenCreatedAt &&
134         $tokenExpiresAt && $token) {
135         return [
136             'user_id' => $userId,
137             'user_email' => $userEmail,
138             'token_id' => $tokenId,
139             'token_created_at' => $tokenCreatedAt,
140             'token_expires_at' => $tokenExpiresAt,
141             'token' => $token,
142         ];
143     } else {
144         return null;
145     }
146 }

```

Lampiran 20. Kode Program *Job SyncAchievementLeaderboards*

```

1 <?php
2
3 class SyncAchievementLeaderboards implements ShouldBeUnique, ShouldQueue
4 {
5     use Queueable;
6
7     /**
8      * Create a new job instance.
9      */
10    public function __construct(
11        public int $year,
12    ) {}
13
14    /**
15     * The number of times the job may be attempted.
16     *
17     * @var int
18     */
19    public $tries = 5;
20

```

```

21  /**
22   * Get the unique ID for the job.
23   */
24  public function uniqueId(): string
25  {
26      return $this->year;
27  }
28
29  /**
30   * Execute the job.
31   */
32  public function handle(): void
33  {
34      $achievementLeaderboards = DB::table('achievements AS a')
35          ->select([
36              'users.id AS user_id',
37              DB::raw('EXTRACT(YEAR FROM a.competition_start_date) AS year')
38              ,
39              DB::raw('SUM(COALESCE(cot.weight,0) * COALESCE(ctt.weight,0) *
40                  COALESCE(cs.weight,0) * COALESCE(ctr.weight,0) * COALESCE(co.weight,0) *
41                  COALESCE(cr.weight,0)) AS total_points'),
42              ])
43          ->join('competition_instances AS ci', 'a.competition_instance_id',
44              '=', 'ci.id')
45          ->join('teams AS t', 'a.team_id', '=', 't.id')
46          ->join('team_members AS tm', 't.id', '=', 'tm.team_id')
47          ->join('users', 'tm.user_id', '=', 'users.id')
48          ->leftJoin('competition_organizer_types AS cot', 'ci.
49              organizer_type_id', '=', 'cot.id')
50          ->leftJoin('competition_team_types AS ctt', 't.team_type_id', '=',
51              'ctt.id')
52          ->leftJoin('competition_scales AS cs', 'a.competition_scale_id', '
53              =', 'cs.id')
54          ->leftJoin('competition_time_ranges AS ctr', 'a.
55              competition_time_range_id', '=', 'ctr.id')
56          ->leftJoin('competition_outputs AS co', 'a.competition_output_id',
57              '=', 'co.id')
58          ->leftJoin('competition_ranks AS cr', 'a.competition_rank_id', '='
59              , 'cr.id')
60          ->groupBy('users.id', DB::raw('EXTRACT(YEAR FROM a.
61              competition_start_date)'))
62          ->whereYear('a.competition_start_date', $this->year)
63          ->get();
64
65      AchievementLeaderboard::where('year', $this->year)->whereNotIn('
66          user_id', $achievementLeaderboards->pluck('user_id'))
67          ->delete();
68
69      foreach ($achievementLeaderboards as $row) {
70          AchievementLeaderboard::updateOrCreate(
71              [
72                  'user_id' => $row->user_id,
73                  'year' => (int) $row->year,
74              ],
75              [
76                  'total_points' => $row->total_points,
77              ]
78          );
79      }
80  }
81  }

```


Lampiran 21. Kode Program *Cast Blob*

```
1 <?php
2
3 class Blob implements CastsAttributes
4 {
5     /**
6      * Cast the given value.
7      *
8      * @param array<string, mixed> $attributes
9      */
10    public function get(Model $model, string $key, mixed $value, array
        $attributes): mixed
11    {
12        return $value ? Storage::url($value) : null;
13    }
14
15    /**
16     * Prepare the given value for storage.
17     *
18     * @param array<string, mixed> $attributes
19     */
20    public function set(Model $model, string $key, mixed $value, array
        $attributes): mixed
21    {
22        return $value;
23    }
24 }
```

Lampiran 22. Kode Program Entitas *User*

```
1 export type UserIncludeOptions = (
2     | 'major'
3     | 'major.degree'
4     | 'major.faculty'
5     | 'personas'
6     | 'groups'
7     | 'permissions'
8 )[];
9
10 export interface UserFilterOptions {
11     ssoId?: string;
12     name?: string;
13     username?: string;
14     emailAddress?: string;
15     phoneNumber?: string;
16     studentId?: string;
17     majorId?: string;
18     startDateOperator?: FilterOperator;
19     startDate?: Date | null;
20     endDateOperator?: FilterOperator;
21     endDate?: Date | null;
22     isMember?: boolean;
23     isExtraordinary?: boolean;
24     createdAtOperator?: FilterOperator;
25     createdAt?: Date;
26     updatedAtOperator?: FilterOperator;
27     updatedAt?: Date;
28 }
29
30 export interface UserSortOptions {
31     id?: 'ASC' | 'DESC';
```

```

32 name?: 'ASC' | 'DESC';
33 username?: 'ASC' | 'DESC';
34 emailAddress?: 'ASC' | 'DESC';
35 phoneNumber?: 'ASC' | 'DESC';
36 studentId?: 'ASC' | 'DESC';
37 majorId?: 'ASC' | 'DESC';
38 startDate?: 'ASC' | 'DESC';
39 endDate?: 'ASC' | 'DESC';
40 isMember?: 'ASC' | 'DESC';
41 isExtraordinary?: 'ASC' | 'DESC';
42 createdAt?: 'ASC' | 'DESC';
43 updatedAt?: 'ASC' | 'DESC';
44 }
45
46 export class User {
47   public id: string;
48   public name: string;
49   public username: string;
50   public emailAddress: string;
51   public phoneNumber: string;
52   public studentId: string;
53   public majorId: string;
54   public links: Record<string, string | undefined>;
55   public startDate: Date | null;
56   public endDate: Date | null;
57   public isMember: boolean;
58   public isExtraordinary: boolean;
59   public isActive: boolean;
60   public createdAt: Date;
61   public updatedAt: Date;
62   public major?: Major;
63
64   public constructor(
65     id: string,
66     name: string,
67     username: string,
68     emailAddress: string,
69     phoneNumber: string,
70     studentId: string,
71     majorId: string,
72     links: Record<string, string | undefined>,
73     startDate: Date | null,
74     endDate: Date | null,
75     isMember: boolean,
76     isExtraordinary: boolean,
77     isActive: boolean,
78     createdAt: Date,
79     updatedAt: Date,
80     major?: Major,
81   ) {
82     this.id = id;
83     this.name = name;
84     this.username = username;
85     this.emailAddress = emailAddress;
86     this.phoneNumber = phoneNumber;
87     this.studentId = studentId;
88     this.majorId = majorId;
89     this.links = links;
90     this.startDate = startDate;
91     this.endDate = endDate;
92     this.isMember = isMember;

```

```

93     this.isExtraordinary = isExtraordinary;
94     this.isActive = isActive;
95     this.createdAt = createdAt;
96     this.updatedAt = updatedAt;
97     this.major = major;
98 }
99 }

```

Lampiran 23. Kode Program Antarmuka Repository UserRepository

```

1  export interface UserRepository {
2      getUsers(
3          includeOptions?: UserIncludeOptions,
4          filterOptions?: UserFilterOptions,
5          sortOptions?: UserSortOptions,
6          paginationOptions?: PaginationOptions,
7          abortSignal?: AbortSignal,
8          token?: string,
9      ): Promise<Either<User[], PaginationOptions>, Error>>;
10
11     getUser(
12         id: string,
13         includeOptions?: UserIncludeOptions,
14         abortSignal?: AbortSignal,
15         token?: string,
16     ): Promise<Either<User, Error>>;
17
18     createUser(
19         user: PartialBy<
20             Omit<User, 'id' | 'isActive' | 'createdAt' | 'updatedAt'>,
21             'startDate' | 'endDate'
22         >,
23         abortSignal?: AbortSignal,
24         token?: string,
25     ): Promise<Either<User, Error>>;
26
27     updateUser(
28         id: string,
29         user: Partial<Omit<User, 'id' | 'isActive' | 'createdAt' | 'updatedAt'>>,
30         abortSignal?: AbortSignal,
31         token?: string,
32     ): Promise<Either<User, Error>>;
33
34     deleteUser(id: string, abortSignal?: AbortSignal, token?: string): Promise<
35         Either<User, Error>>;
36
37     extendUserMembership(
38         id: string,
39         abortSignal?: AbortSignal,
40         token?: string,
41     ): Promise<Either<User, Error>>;
42 }

```

Lampiran 24. Kode Program Kelas Kesalahan HttpError

```

1  export class HttpError extends Error {
2      public statusCode: number;
3      public data?: unknown;
4
5      public constructor(
6          message: string = 'Unknown HTTP error',
7          statusCode: number = 500,

```

```

8     data?: unknown,
9     options?: ErrorOptions,
10  ) {
11      super(message, options);
12
13      this.statusCode = statusCode;
14      this.data = data;
15  }
16 }
17
18 export class BadRequestError extends HttpError {
19     public constructor(message: string = 'Bad request', data?: unknown, options
20         ? ErrorOptions) {
21         super(message, 400, data, options);
22     }
23 }
24 export class UnauthorizedError extends HttpError {
25     public constructor(message: string = 'Unauthorized', data?: unknown, options
26         ? ErrorOptions) {
27         super(message, 401, data, options);
28     }
29 }
30 export class ForbiddenError extends HttpError {
31     public constructor(message: string = 'Forbidden', data?: unknown, options?:
32         ErrorOptions) {
33         super(message, 403, data, options);
34     }
35 }
36 export class NotFoundError extends HttpError {
37     public constructor(message: string = 'Not found', data?: unknown, options?:
38         ErrorOptions) {
39         super(message, 404, data, options);
40     }
41 }
42 export class UnprocessableContentError extends HttpError {
43     public constructor(
44         message: string = 'Unprocessable content',
45         data?: unknown,
46         options?: ErrorOptions,
47     ) {
48         super(message, 422, data, options);
49     }
50 }
51
52 export class ValidationError extends UnprocessableContentError {
53     public data?: Record<string, string[]>;
54
55     public constructor(
56         message: string = 'Validation error',
57         data?: Record<string, string[]>,
58         options?: ErrorOptions,
59     ) {
60         super(message, data, options);
61         this.data = data;
62     }
63 }
64

```

```

65 export class InternalServerError extends HttpError {
66   public constructor(
67     message: string = 'Internal server error',
68     data?: unknown,
69     options?: ErrorOptions,
70   ) {
71     super(message, 500, data, options);
72   }
73 }

```

Lampiran 25. Kode Program *Use Case* GetUsers

```

1  export type GetUsersParams = [
2    includeOptions?: UserIncludeOptions,
3    filterOptions?: UserFilterOptions,
4    sortOptions?: UserSortOptions,
5    paginationOptions?: PaginationOptions,
6    abortSignal?: AbortSignal,
7    authenticate?: boolean,
8  ];
9
10 @injectable()
11 export class GetUsers implements UseCase<
12   Promise<Either<User[], PaginationOptions>, Error>>,
13   GetUsersParams
14 > {
15   private readonly userRepository: UserRepository;
16   private readonly authRepository: AuthRepository;
17
18   public constructor(
19     @inject(SYMBOLS.UserRepository)
20     userRepository: UserRepository,
21     @inject(SYMBOLS.AuthRepository)
22     authRepository: AuthRepository,
23   ) {
24     this.userRepository = userRepository;
25     this.authRepository = authRepository;
26   }
27
28   public async execute(
29     includeOptions?: UserIncludeOptions,
30     filterOptions?: UserFilterOptions,
31     sortOptions?: UserSortOptions,
32     paginationOptions?: PaginationOptions,
33     abortSignal?: AbortSignal,
34     authenticate: boolean = true,
35   ): Promise<Either<User[], PaginationOptions>, Error>> {
36     let accessToken: string | undefined;
37
38     if (authenticate) {
39       const accessTokenResult = await this.authRepository.getAccessToken();
40
41       if (isRight(accessTokenResult)) {
42         accessToken = accessTokenResult.right;
43       } else {
44         return left(accessTokenResult.left);
45       }
46     }
47
48     return await this.userRepository.getUsers(
49       includeOptions,
50       filterOptions,

```

```

51     sortOptions,
52     paginationOptions,
53     abortSignal,
54     accessToken,
55   );
56 }
57 }

```

Lampiran 26. Kode Program Data Source InfinityApiDataSource

```

1 export interface InfinityApiDataSource extends AxiosInstance {}
2
3 export const infinityApiDataSourceImpl: InfinityApiDataSource = axios.create({
4   baseURL: process.env.NEXT_PUBLIC_INFINITY_API_URL,
5   timeout:
6     process.env.NEXT_PUBLIC_INFINITY_API_TIMEOUT != null
7     ? Number(process.env.NEXT_PUBLIC_INFINITY_API_TIMEOUT)
8     : 10000,
9 });

```

Lampiran 27. Kode Program DTO UserDto

```

1 export interface UserDto {
2   id: string;
3   name: string;
4   username: string;
5   email_address: string;
6   phone_number: string;
7   student_id: string;
8   major_id: string;
9   links: Record<string, string | undefined>;
10  start_date: string | null;
11  end_date: string | null;
12  is_member: boolean;
13  is_extraordinary: boolean;
14  is_active: boolean;
15  created_at: string;
16  updated_at: string;
17  major?: MajorDto;
18 }
19
20 export class UserMapper {
21   public static fromDomainToDto(user: Partial<User>): Partial<UserDto> {
22     return {
23       id: user.id,
24       name: user.name,
25       username: user.username,
26       email_address: user.emailAddress,
27       phone_number: user.phoneNumber,
28       student_id: user.studentId,
29       major_id: user.majorId,
30       links: user.links,
31       start_date: user.startDate ? user.startDate.toISOString().split('T')[0]
32       : null,
33       end_date: user.endDate ? user.endDate.toISOString().split('T')[0] : null
34     },
35     {
36       is_member: user.isMember,
37       is_extraordinary: user.isExtraordinary,
38       is_active: user.isActive,
39       created_at: user.createdAt?.toISOString(),
40       updated_at: user.updatedAt?.toISOString(),
41       major: user.major ? (MajorMapper.fromDomainToDto(user.major) as MajorDto)
42       : undefined,
43     }
44   }
45 }

```

```

39     };
40 }
41
42 public static fromDtoToDomain(dto: UserDto): User {
43     return new User(
44         dto.id,
45         dto.name,
46         dto.username,
47         dto.email_address,
48         dto.phone_number,
49         dto.student_id,
50         dto.major_id,
51         dto.links,
52         dto.start_date ? DateTime.fromISO(dto.start_date, { zone: 'UTC' }).
toJSDate() : null,
53         dto.end_date ? DateTime.fromISO(dto.end_date, { zone: 'UTC' }).toJSDate
() : null,
54         dto.is_member,
55         dto.is_extraordinary,
56         dto.is_active,
57         DateTime.fromISO(dto.created_at).toJSDate(),
58         DateTime.fromISO(dto.updated_at).toJSDate(),
59         dto.major ? MajorMapper.fromDtoToDomain(dto.major) : undefined,
60     );
61 }
62 }

```

Lampiran 28. Kode Program Implementasi Repositori UserRepositoryImpl

```

1 @injectable()
2 export class UserRepositoryImpl implements UserRepository {
3     public constructor(
4         @inject(SYMBOLS.InfinityApiDataSource)
5         private infinityApiDataSource: InfinityApiDataSource,
6     ) {}
7
8     public async getUsers(
9         includeOptions?: UserIncludeOptions,
10        filterOptions?: UserFilterOptions,
11        sortOptions?: UserSortOptions,
12        paginationOptions?: PaginationOptions,
13        abortSignal?: AbortSignal,
14        token?: string,
15    ): Promise<Either<[User[], PaginationOptions], Error>> {
16        try {
17            const response = await this.infinityApiDataSource.get('/users', {
18                signal: abortSignal,
19                headers: {
20                    ...(token ? { Authorization: 'Bearer ${token}' } : {}),
21                },
22                params: {
23                    per_page: paginationOptions?.perPage,
24                    cursor: paginationOptions?.cursor,
25                    includes: includeOptions
26                        ?.filter((value, index, self) => self.indexOf(value) === index)
27                        .join(','),
28                    'filters[sso_id]': filterOptions?.ssoId,
29                    'filters[name]': filterOptions?.name,
30                    'filters[email_address]': filterOptions?.emailAddress,
31                    'filters[phone_number]': filterOptions?.phoneNumber,

```

```

32     'filters[student_id]': filterOptions?.studentId,
33     'filters[major_id]': filterOptions?.majorId,
34     'filters[start_date]':
35       filterOptions?.startDate !== undefined
36       ? (filterOptions.startDateOperator ?? '') +
37         (filterOptions.startDate?.toISOString() ?? '')
38       : undefined,
39     'filters[end_date]':
40       filterOptions?.endDate !== undefined
41       ? (filterOptions.endDateOperator ?? '') + (filterOptions.endDate
42         ?.toISOString() ?? '')
43       : undefined,
44     'filters[is_member]': filterOptions?.isMember,
45     'filters[is_extraordinary]': filterOptions?.isExtraordinary,
46     'filters[created_at]': filterOptions?.createdAt
47       ? (filterOptions.createdAtOperator ?? '') +
48         (filterOptions.createdAt?.toISOString() ?? '')
49       : undefined,
50     'filters[updated_at]': filterOptions?.updatedAt
51       ? (filterOptions.updatedAtOperator ?? '') +
52         (filterOptions.updatedAt?.toISOString() ?? '')
53       : undefined,
54     sorts: sortOptions
55       ? Object.entries(sortOptions)
56         .map((sortOption) => {
57           const prefix = sortOption[1] === 'DESC' ? '-' : '';
58           const field = sortOption[0]
59             .split(/(?=[A-Z])/)
60             .join('_')
61             .toLowerCase();
62           return prefix + field;
63         })
64       : undefined,
65   },
66   });
67
68   const usersResponse = response.data.data.users.map(UserMapper.
69     fromDtoToDomain);
70
71   const paginationOptionsResponse = new PaginationOptions(
72     response.data.data.meta.per_page,
73     paginationOptions?.cursor,
74     response.data.data.meta.next_cursor ?? undefined,
75     response.data.data.meta.prev_cursor ?? undefined,
76   );
77
78   return right([usersResponse, paginationOptionsResponse]);
79 } catch (error) {
80   return left(handleAxiosError(error));
81 }
82
83 public async getUser(
84   id: string,
85   includeOptions?: UserIncludeOptions,
86   abortSignal?: AbortSignal,
87   token?: string,
88 ): Promise<Either<User, Error>> {
89   try {
90     const response = await this.infinityApiDataSource.get(`/users/${id}`, {

```



```

91     signal: abortSignal,
92     headers: {
93       ...(token ? { Authorization: 'Bearer ${token}' } : {}),
94     },
95     params: {
96       includes: includeOptions
97         ?.filter((value, index, self) => self.indexOf(value) === index)
98         .join(','),
99     },
100   });
101
102   const userResponse = UserMapper.fromDtoToDomain(response.data.data.user)
103   ;
104
105   return right(userResponse);
106 } catch (error) {
107   return left(handleAxiosError(error));
108 }
109
110 public async createUser(
111   user: PartialBy<
112     Omit<User, 'id' | 'isActive' | 'createdAt' | 'updatedAt'>,
113     'startDate' | 'endDate'
114   >,
115   abortSignal?: AbortSignal,
116   token?: string,
117 ): Promise<Either<User, Error>> {
118   try {
119     const response = await this.infinityApiDataSource.post(
120       '/users',
121       UserMapper.fromDomainToDto(user),
122       {
123         signal: abortSignal,
124         headers: {
125           ...(token ? { Authorization: 'Bearer ${token}' } : {}),
126         },
127       },
128     );
129
130     const userResponse = UserMapper.fromDtoToDomain(response.data.data.user)
131     ;
132
133     return right(userResponse);
134   } catch (error) {
135     return left(handleAxiosError(error));
136   }
137
138   public async updateUser(
139     id: string,
140     user: Partial<Omit<User, 'id' | 'isActive' | 'createdAt' | 'updatedAt'>>,
141     abortSignal?: AbortSignal,
142     token?: string,
143   ): Promise<Either<User, Error>> {
144     try {
145       const response = await this.infinityApiDataSource.patch(
146         `/users/${id}`,
147         UserMapper.fromDomainToDto(user),
148         {
149           signal: abortSignal,

```

```

150         headers: {
151             ...(token ? { Authorization: `Bearer ${token}` } : {}),
152         },
153     },
154 );
155
156     const userResponse = UserMapper.fromDtoToDomain(response.data.data.user)
157     ;
158
159     return right(userResponse);
160 } catch (error) {
161     return left(handleAxiosError(error));
162 }
163
164 public async deleteUser(
165     id: string,
166     abortSignal?: AbortSignal,
167     token?: string,
168 ): Promise<Either<User, Error>> {
169     try {
170         const response = await this.infinityApiDataSource.delete(`/users/${id}`
171         , {
172             signal: abortSignal,
173             headers: {
174                 ...(token ? { Authorization: `Bearer ${token}` } : {}),
175             },
176         });
177
178         const userResponse = UserMapper.fromDtoToDomain(response.data.data.user)
179         ;
180
181         return right(userResponse);
182     } catch (error) {
183         return left(handleAxiosError(error));
184     }
185 }
186
187 public async extendUserMembership(
188     id: string,
189     abortSignal?: AbortSignal,
190     token?: string,
191 ): Promise<Either<User, Error>> {
192     try {
193         const response = await this.infinityApiDataSource.post(`/users/${id}/
194         extend`, undefined, {
195             signal: abortSignal,
196             headers: {
197                 ...(token ? { Authorization: `Bearer ${token}` } : {}),
198             },
199         });
200
201         const userResponse = UserMapper.fromDtoToDomain(response.data.data.user)
202         ;
203
204         return right(userResponse);
205     } catch (error) {
206         return left(handleAxiosError(error));
207     }
208 }
209 }

```

Lampiran 29. Kode Program *Dockerfile Backend*

```
1 FROM caddy:2.11.4-builder AS caddy-builder
2
3 FROM dunglas/frankenphp:1.12.2-builder-php8.4.20 AS upstream
4
5 COPY --link --from=caddy-builder /usr/bin/xcaddy /usr/bin/xcaddy
6
7 RUN CGO_ENABLED=1 \
8     XCADDY_SETCAP=1 \
9     XCADDY_GO_BUILD_FLAGS="-ldflags='-w -s' -tags=nobadger,nomysql,nopgx" \
10    CGO_CFLAGS=$(php-config --includes) \
11    CGO_LDFLAGS="$(php-config --ldflags) $(php-config --libs)" \
12    xcaddy build \
13    --output /usr/local/bin/frankenphp \
14    --with github.com/dunglas/frankenphp=./ \
15    --with github.com/dunglas/frankenphp/caddy=./caddy/ \
16    --with github.com/dunglas/caddy-cbrotli
17
18 FROM dunglas/frankenphp:1.12.2-php8.4.20 AS base
19
20 RUN apt-get update && \
21     apt-get install -yqq --no-install-recommends \
22     curl \
23     wget \
24     ca-certificates \
25     procps \
26     unzip \
27     supervisor \
28     libsodium-dev && \
29     apt-get autoremove -yqq && \
30     apt-get clean && \
31     rm -rf /var/lib/apt/lists/* /tmp/* /var/tmp/* /var/log/lastlog /var/log/
32     faillog
33
34 RUN install-php-extensions \
35     bcmath \
36     apcu \
37     bz2 \
38     exif \
39     gd \
40     igbinary \
41     intl \
42     mbstring \
43     opcache \
44     pcntl \
45     pdo_pgsql \
46     pdo_mysql \
47     pgsql \
48     redis \
49     sockets \
50     vips \
51     ffi \
52     zip && \
53     docker-php-source delete && \
54     rm -rf /tmp/* /var/tmp/*
55
56 # Install dependencies
57 FROM composer:2.10.2 AS vendor
58
59 ENV COMPOSER_FUND=0 \
```

```

59 COMPOSER_MAX_PARALLEL_HTTP=48 \
60 COMPOSER_IGNORE_PLATFORM_REQS=1
61
62 WORKDIR /tmp
63
64 COPY --link composer.* ./
65
66 RUN composer install \
67     --classmap-authoritative \
68     --no-interaction \
69     --no-ansi \
70     --no-dev \
71     --no-scripts \
72     --prefer-dist
73
74 # Build production image
75 FROM base AS runner
76
77 COPY --link --from=upstream /usr/local/bin/frankenphp /usr/local/bin/
    frankenphp
78
79 ARG UID=1000 \
80     GID=1000 \
81     TZ=UTC \
82     APP_ENV=production
83
84 ENV USER=octane \
85     ROOT=/var/www/html \
86     OCTANE_SERVER=frankenphp \
87     TZ=${TZ} \
88     TERM=xterm-color \
89     LANG=C.UTF-8 \
90     WITH_HORIZON=false \
91     WITH_REVERB=false \
92     WITH_SCHEDULER=false \
93     APP_ENV=${APP_ENV} \
94     APP_DEBUG=false \
95     COMPOSER_FUND=0 \
96     COMPOSER_MAX_PARALLEL_HTTP=48 \
97     GODEBUG=cgocheck=0 \
98     GOMEMLIMIT=512MiB
99
100 ENV XDG_CONFIG_HOME=${ROOT}/.config \
101     XDG_DATA_HOME=${ROOT}/.data
102
103 WORKDIR ${ROOT}
104
105 SHELL ["/bin/sh", "-eou", "pipefail", "-c"]
106
107 RUN arch="$(uname -m)" && \
108     case "$arch" in \
109     armhf) _cronic_fname="supercronic-linux-arm" ;; \
110     aarch64) _cronic_fname="supercronic-linux-arm64" ;; \
111     x86_64) _cronic_fname="supercronic-linux-amd64" ;; \
112     x86) _cronic_fname="supercronic-linux-386" ;; \
113     *) echo >&2 "error: unsupported architecture: $arch"; exit 1 ;; \
114     esac && \
115     wget -q "https://github.com/aptible/supercronic/releases/download/v0.2.46/
    ${_cronic_fname}" \
116     -O /usr/bin/supercronic && \
117     chmod +x /usr/bin/supercronic && \

```

```

118     mkdir -p /etc/supercronic && \
119     echo "*/1 * * * * php ${ROOT}/artisan schedule:run --no-interaction" > /
    etc/supercronic/laravel
120
121 RUN ln -snf /usr/share/zoneinfo/${TZ} /etc/localtime && \
122     echo ${TZ} > /etc/timezone
123 RUN userdel --remove --force www-data 2>/dev/null || true && \
124     groupadd --force -g ${GID} ${USER} && \
125     useradd -ms /bin/bash --no-log-init --no-user-group -g ${GID} -u ${UID} $
    {USER}
126
127 RUN mkdir -p ${XDG_CONFIG_HOME} ${XDG_DATA_HOME} /var/log/supervisor /var/run
    /supervisor && \
128     chown -R ${UID}:${GID} ${ROOT} /var/log /var/run && \
129     chmod -R a+rw ${ROOT} /var/log /var/run && \
130     find / -perm /6000 -type f -exec chmod a-s {} + 2>/dev/null || true
131
132 # Use the default production configuration
133 RUN cp ${PHP_INI_DIR}/php.ini-production ${PHP_INI_DIR}/php.ini
134
135 USER ${USER}
136
137 COPY --link --chown=${UID}:${GID} deployment/php.ini ${PHP_INI_DIR}/conf.d
    /99-octane.ini
138 COPY --link --chown=${UID}:${GID} deployment/Caddyfile ${ROOT}/deployment/
    Caddyfile
139 COPY --link --chown=${UID}:${GID} deployment/supervisord.conf /etc/
140 COPY --link --chown=${UID}:${GID} deployment/supervisord*.conf /etc/
    supervisor/conf.d/
141 COPY --link --chown=${UID}:${GID} deployment/healthcheck /usr/local/bin/
    healthcheck
142 COPY --link --chown=${UID}:${GID} deployment/start-container /usr/local/bin/
    start-container
143 COPY --link --chown=${UID}:${GID} --from=vendor /usr/bin/composer /usr/bin/
    composer
144 COPY --link --chown=${UID}:${GID} --from=vendor /tmp/vendor ./vendor
145 COPY --link --chown=${UID}:${GID} . .
146
147 RUN composer dump-autoload \
148     --optimize \
149     --classmap-authoritative \
150     --apcu \
151     --no-dev && \
152     rm -f /usr/bin/composer && \
153     chmod +x /usr/local/bin/start-container /usr/local/bin/healthcheck
154
155 EXPOSE 8000 2019 8080
156
157 HEALTHCHECK --start-period=5s --interval=1s --timeout=3s --retries=10 CMD
    healthcheck || exit 1
158
159 ENTRYPOINT ["start-container"]

```

Lampiran 30. Kode Program Dockerfile Frontend

```

1 FROM node:24.17.0-alpine AS dependencies
2
3 # Set working directory
4 WORKDIR /app
5
6 # Copy package-related files first to leverage Docker's caching mechanism
7 COPY package.json package-lock.json* ./

```

```

8
9 # Install project dependencies with frozen lockfile for reproducible builds
10 RUN --mount=type=cache,target=/root/.npm \
11     npm ci --prefer-offline --no-audit --no-fund;
12
13 FROM node:24.17.0-alpine AS builder
14
15 # Set working directory
16 WORKDIR /app
17
18 # Copy project dependencies from dependencies stage
19 COPY --from=dependencies /app/node_modules ./node_modules
20
21 # Copy application source code
22 COPY . .
23
24 # Accept NEXT_PUBLIC_ env vars as build args so they are baked in at build
    time
25 ARG NEXT_PUBLIC_APP_URL \
26     NEXT_PUBLIC_BETTER_AUTH_URL \
27     NEXT_PUBLIC_INFINITY_API_URL \
28     NEXT_PUBLIC_INFINITY_API_TIMEOUT
29
30 # Set build stage environment variables
31 ENV NODE_ENV=production \
32     CI=1 \
33     # Disable telemetry during the build.
34     NEXT_TELEMETRY_DISABLED=1 \
35     NEXT_PUBLIC_APP_URL=${NEXT_PUBLIC_APP_URL} \
36     NEXT_PUBLIC_BETTER_AUTH_URL=${NEXT_PUBLIC_BETTER_AUTH_URL} \
37     NEXT_PUBLIC_INFINITY_API_URL=${NEXT_PUBLIC_INFINITY_API_URL} \
38     NEXT_PUBLIC_INFINITY_API_TIMEOUT=${NEXT_PUBLIC_INFINITY_API_TIMEOUT}
39
40 # Build Next.js application
41 RUN --mount=type=cache,target=/app/.next/cache \
42     npm run build
43
44 FROM oven/bun:1.3.13 AS runner
45
46 # Set working directory
47 WORKDIR /app
48
49 # Set production environment variables
50 ENV NODE_ENV=production \
51     PORT=3000 \
52     HOSTNAME="[:]" \
53     # Disable telemetry during the run time.
54     NEXT_TELEMETRY_DISABLED=1
55
56 # Copy production assets
57 COPY --from=builder --chown=bun:bun /app/public ./public
58
59 # Set the correct permission for prerender cache
60 RUN mkdir .next && \
61     chown bun:bun .next
62
63 # Automatically leverage output traces to reduce image size
64 # https://nextjs.org/docs/advanced-features/output-file-tracing
65 COPY --from=builder --chown=bun:bun /app/.next/standalone ./
66 COPY --from=builder --chown=bun:bun /app/.next/static ./next/static
67

```

```

68 # Switch to non-root user for security best practices
69 USER bun
70
71 # Expose port 3000 to allow HTTP traffic
72 EXPOSE 3000
73
74 # Start Next.js standalone server with Bun
75 CMD ["bun", "server.js"]

```

Lampiran 31. Kode Program *Pipeline Rilis Backend*

```

1 # yaml-language-server: $schema=https://json.schemastore.org/github-workflow
2 name: Release
3
4 on:
5   workflow_dispatch:
6     inputs:
7       version:
8         description: "Version"
9         required: true
10      release_type:
11        description: "Release type"
12        required: true
13        type: choice
14        default: "prod"
15        options:
16          - alpha
17          - beta
18          - rc
19          - prod
20      pre_release_version:
21        description: "Pre-release version (for non production releases)"
22        required: false
23        type: number
24
25 jobs:
26   guard:
27     name: Validate branch and release type
28     runs-on: ubuntu-latest
29     steps:
30       - name: Check if branch and release type match
31         id: guard
32         run: |
33           if [[ "$BRANCH" != "main" && "$BRANCH" != "develop" ]]; then
34             echo "Branch is not main or develop! Aborting."
35             VERSION_MISMATCH='true';
36           elif [ "$BRANCH" == "main" ] && [ "$RELEASE_TYPE" != "prod" ]; then
37             echo "Branch and release type do not match! Aborting."
38             VERSION_MISMATCH='true';
39           elif [ "$BRANCH" == "develop" ] && ( [ "$RELEASE_TYPE" == "prod" ]
40             || [ -z "$PRE_RELEASE_VERSION" ] ); then
41             echo "Branch and release type do not match! Aborting."
42             VERSION_MISMATCH='true';
43           else
44             echo "Branch and release type match! Proceeding."
45             VERSION_MISMATCH='false';
46           fi
47
48           echo "version_mismatch=$(echo $VERSION_MISMATCH)" >> "
49           $GITHUB_OUTPUT";
50     env:
51       BRANCH: ${ github.ref_name }

```

```

50     RELEASE_TYPE: ${ inputs.release_type }
51     PRE_RELEASE_VERSION: ${ inputs.pre_release_version }
52
53     - name: Fail if branch and release type do not match
54       if: ${ steps.guard.outputs.version_mismatch == 'true' }
55       uses: actions/github-script@v9.0.0
56       with:
57         script: |
58           core.setFailed('Workflow failed. Did you set the correct version,
release type, and pre-release version?');
59
60 prepare:
61   name: Prepare release
62   needs:
63     - guard
64   runs-on: ubuntu-latest
65   permissions:
66     contents: write
67   outputs:
68     version: ${ steps.version.outputs.version }
69     tags: ${ steps.meta.outputs.tags }
70     labels: ${ steps.meta.outputs.labels }
71     annotations: ${ steps.meta.outputs.annotations }
72     clean_changelog: ${ steps.changelog.outputs.clean_changelog }
73     commit_hash: ${ steps.auto-commit.outputs.commit_hash }
74   env:
75     POSTGRES_PASSWORD: postgres
76   services:
77     postgres:
78       image: postgres
79       env:
80         POSTGRES_PASSWORD: ${ env.POSTGRES_PASSWORD }
81       options: >-
82         --health-cmd pg_isready
83         --health-interval 10s
84         --health-timeout 5s
85         --health-retries 5
86       ports:
87         - 5432:5432
88   steps:
89     - name: Set version
90       id: version
91       run: |
92         if [ "$RELEASE_TYPE" != "prod" ]; then
93           VERSION="${VERSION#v}-${RELEASE_TYPE}.${PRE_RELEASE_VERSION}"
94         else
95           VERSION="${VERSION#v}"
96         fi
97         echo "version=${VERSION}" >> "$GITHUB_OUTPUT"
98       env:
99         VERSION: ${ inputs.version }
100         RELEASE_TYPE: ${ inputs.release_type }
101         PRE_RELEASE_VERSION: ${ inputs.pre_release_version }
102
103     - name: Checkout repo
104       uses: actions/checkout@v7.0.0
105       with:
106         fetch-depth: 0
107
108     - name: Set up PHP
109       uses: shivammathur/setup-php@2.37.2

```



```

110     with:
111         php-version: "8.4"
112
113     - name: Install dependencies
114       uses: ramsey/composer-install@4.0.0
115       with:
116         composer-options: "--ignore-platform-reqs --optimize-autoloader"
117
118     - name: Update app version
119       run: sed -i "s/'version' => '.*',/'version' => '${{ steps.version.
120         outputs.version }}',/g" config/app.php
121
122     - name: Generate documentations
123       run: |
124         php artisan migrate --force
125         php artisan scribe:generate --env docs
126         mv storage/app/local/scribe/* storage/app/scribe/
127       env:
128         APP_KEY: "base64:wOy0BDdP65xkwGbM6fsgH97oBV9J02zn/VvTkiTXQqE="
129         DB_CONNECTION: pgsql
130         DB_READ_HOST: localhost
131         DB_WRITE_HOST: localhost
132         DB_DATABASE: postgres
133         DB_USERNAME: postgres
134         DB_PASSWORD: ${ env.POSTGRES_PASSWORD }
135
136     - name: Generate changelog
137       id: changelog
138       uses: TriPSs/conventional-changelog-action@v6.4.0
139       with:
140         github-token: ${ secrets.GITHUB_TOKEN }
141         skip-commit: "true"
142         skip-tag: "true"
143         skip-git-pull: "true"
144         skip-version-file: "true"
145         output-file: "CHANGELOG.md"
146         tag-prefix: "v"
147
148     - name: Commit and push version change
149       if: ${ inputs.release_type == 'prod' }
150       id: auto-commit
151       uses: stefanzweifel/git-auto-commit-action@v7.2.0
152       with:
153         commit_message: "chore: Update docs and version to v${{ steps.
154           version.outputs.version }}"
155
156     - name: Upload modified files
157       if: ${ inputs.release_type != 'prod' }
158       uses: actions/upload-artifact@v7.0.1
159       with:
160         name: release-files
161         path: |
162           config/app.php
163           CHANGELOG.md
164           storage/app/scribe/
165         retention-days: 1
166
167     - name: Extract metadata for Container
168       id: meta
169       uses: docker/metadata-action@v6.2.0
170       with:

```

```

169         images: ghcr.io/${{ github.repository }}
170         tags: type=semver,pattern={{version}},value=${{ steps.version.
outputs.version }}
171         labels: |
172             org.opencontainers.image.title=Infinity Back-End
173             org.opencontainers.image.description=Infinity back-end app image,
see https://github.com/infiniteuny/infinity-backend/ for more info.
174             org.opencontainers.image.vendor=INFINITE UNY
175         annotations: |
176             org.opencontainers.image.title=Infinity Back-End
177             org.opencontainers.image.description=Infinity back-end app image,
see https://github.com/infiniteuny/infinity-backend/ for more info.
178             org.opencontainers.image.vendor=INFINITE UNY
179     env:
180         DOCKER_METADATA_ANNOTATIONS_LEVELS: manifest,index
181
182 build:
183     name: Build (${{ matrix.platform }})
184     needs:
185         - prepare
186     runs-on: ${{ matrix.runner }}
187     permissions:
188         contents: read
189         packages: write
190     strategy:
191         fail-fast: false
192     matrix:
193         include:
194             - platform: linux/amd64
195               runner: ubuntu-24.04
196               scope: amd64
197             - platform: linux/arm64
198               runner: ubuntu-24.04-arm
199               scope: arm64
200     steps:
201         - name: Checkout repo
202           uses: actions/checkout@v7.0.0
203           with:
204             ref: ${{ needs.prepare.outputs.commit_hash || github.ref_name }}
205
206         - name: Download release files
207           if: ${{ inputs.release_type != 'prod' }}
208           uses: actions/download-artifact@v8.0.1
209           with:
210             name: release-files
211             path: .
212
213         - name: Set up Docker Buildx
214           uses: docker/setup-buildx-action@v4.2.0
215
216         - name: Login to GitHub Container Registry
217           uses: docker/login-action@v4.4.0
218           with:
219             registry: ghcr.io
220             username: ${{ github.repository_owner }}
221             password: ${{ secrets.GITHUB_TOKEN }}
222
223         - name: Build and push by digest
224           id: build
225           uses: docker/build-push-action@v7.3.0
226           with:

```

```

227     context: .
228     labels: ${ needs.prepare.outputs.labels }
229     annotations: ${ needs.prepare.outputs.annotations }
230     outputs: type=image,name=ghcr.io/${ github.repository },push-by-
digest=true,name-canonical=true,push=true
231     platforms: ${ matrix.platform }
232     cache-from: type=gha,scope=${ matrix.scope }
233     cache-to: type=gha,scope=${ matrix.scope },mode=max
234
235     - name: Export digest
236       run: |
237         mkdir -p /tmp/digests
238         digest="${ steps.build.outputs.digest }"
239         touch "/tmp/digests/${ digest#sha256:}"
240
241     - name: Upload digest artifact
242       uses: actions/upload-artifact@v7.0.1
243       with:
244         name: digests-${ matrix.scope }
245         path: /tmp/digests/*
246         if-no-files-found: error
247         retention-days: 1
248
249 merge:
250   name: Create multi-arch manifest
251   needs:
252     - prepare
253     - build
254   runs-on: ubuntu-latest
255   permissions:
256     packages: write
257   outputs:
258     digest: ${ steps.manifest.outputs.digest }
259   steps:
260     - name: Download digests
261       uses: actions/download-artifact@v8.0.1
262       with:
263         path: /tmp/digests
264         pattern: digests-*
265         merge-multiple: true
266
267     - name: Prepare sources
268       id: sources
269       working-directory: /tmp/digests
270       run: |
271         {
272           echo 'sources<<EOF'
273           for f in *; do
274             echo "ghcr.io/${ github.repository }@sha256:${ f}"
275           done
276           echo EOF
277         } >> "$GITHUB_OUTPUT"
278
279     - name: Login to GitHub Container Registry
280       uses: docker/login-action@v4.4.0
281       with:
282         registry: ghcr.io
283         username: ${ github.repository_owner }
284         password: ${ secrets.GITHUB_TOKEN }
285
286     - name: Create multi-arch manifest

```

```

287     id: manifest
288     uses: int128/docker-manifest-create-action@v2.24.0
289     with:
290       tags: ${{ needs.prepare.outputs.tags }}
291       index-annotations: ${{ needs.prepare.outputs.labels }}
292       sources: ${{ steps.sources.outputs.sources }}
293
294   sign:
295     name: Sign images
296     needs:
297       - prepare
298       - merge
299     runs-on: ubuntu-latest
300     permissions:
301       id-token: write
302       packages: write
303     steps:
304       - name: Install Cosign
305         uses: sigstore/cosign-installer@v4.1.2
306
307       - name: Login to GitHub Container Registry
308         uses: docker/login-action@v4.4.0
309         with:
310           registry: ghcr.io
311           username: ${{ github.repository_owner }}
312           password: ${{ secrets.GITHUB_TOKEN }}
313
314       - name: Sign the images with GitHub OIDC token
315         run: |
316           for tag in ${TAGS//,/ }; do
317             cosign sign --yes "${tag}@${DIGEST}"
318           done
319         env:
320           DIGEST: ${{ needs.merge.outputs.digest }}
321           TAGS: ${{ needs.prepare.outputs.tags }}
322
323   release:
324     name: Create GitHub release
325     needs:
326       - guard
327       - prepare
328       - sign
329     runs-on: ubuntu-latest
330     permissions:
331       contents: write
332     steps:
333       - name: Checkout repo
334         uses: actions/checkout@v7.0.0
335         with:
336           ref: ${{ needs.prepare.outputs.commit_hash || github.ref_name }}
337
338       - name: Create release
339         uses: softprops/action-gh-release@v3.0.1
340         env:
341           GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
342         with:
343           tag_name: v${{ needs.prepare.outputs.version }}
344           name: v${{ needs.prepare.outputs.version }}
345           target_commitish: ${{ github.ref_name }}
346           prerelease: ${{ inputs.release_type != 'prod' }}
347           make_latest: "${{ inputs.release_type == 'prod' }}"

```

```
348 body: ${ needs.prepare.outputs.clean_changelog }}
```

Lampiran 32. Kode Program *Pipeline Rilis Frontend*

```
1 # yaml-language-server: $schema=https://json.schemastore.org/github-workflow
2 name: Release
3
4 on:
5   workflow_dispatch:
6     inputs:
7       version:
8         description: "Version"
9         required: true
10      release_type:
11        description: "Release type"
12        required: true
13        type: choice
14        default: "prod"
15        options:
16          - alpha
17          - beta
18          - rc
19          - prod
20      pre_release_version:
21        description: "Pre-release version (for non production releases)"
22        required: false
23        type: number
24
25 jobs:
26   guard:
27     name: Validate branch and release type
28     runs-on: ubuntu-latest
29     steps:
30       - name: Check if branch and release type match
31         id: guard
32         run: |
33           if [[ "$BRANCH" != "main" && "$BRANCH" != "develop" ]]; then
34             echo "Branch is not main or develop! Aborting."
35             VERSION_MISMATCH='true';
36           elif [ "$BRANCH" == "main" ] && [ "$RELEASE_TYPE" != "prod" ]; then
37             echo "Branch and release type do not match! Aborting."
38             VERSION_MISMATCH='true';
39           elif [ "$BRANCH" == "develop" ] && ( [ "$RELEASE_TYPE" == "prod" ]
40             || [ -z "$PRE_RELEASE_VERSION" ] ); then
41             echo "Branch and release type do not match! Aborting."
42             VERSION_MISMATCH='true';
43           else
44             echo "Branch and release type match! Proceeding."
45             VERSION_MISMATCH='false';
46           fi
47
48           echo "version_mismatch=$(echo $VERSION_MISMATCH)" >> "
49           $GITHUB_OUTPUT";
50     env:
51       BRANCH: ${ github.ref_name }
52       RELEASE_TYPE: ${ inputs.release_type }
53       PRE_RELEASE_VERSION: ${ inputs.pre_release_version }
54
55   - name: Fail if branch and release type do not match
56     if: ${ steps.guard.outputs.version_mismatch == 'true' }
57     uses: actions/github-script@v9.0.0
58     with:
```

```

57         script: |
58             core.setFailed('Workflow failed. Did you set the correct version,
release type, and pre-release version?');
59
60     prepare:
61         name: Prepare release
62         needs:
63             - guard
64         runs-on: ubuntu-latest
65         permissions:
66             contents: write
67         outputs:
68             version: ${ steps.version.outputs.version }
69             tags: ${ steps.meta.outputs.tags }
70             labels: ${ steps.meta.outputs.labels }
71             annotations: ${ steps.meta.outputs.annotations }
72             clean_changelog: ${ steps.changelog.outputs.clean_changelog }
73             commit_hash: ${ steps.auto-commit.outputs.commit_hash }
74     steps:
75         - name: Set version
76           id: version
77           run: |
78               if [ "$RELEASE_TYPE" != "prod" ]; then
79                   VERSION="${VERSION#v}-${RELEASE_TYPE}.${PRE_RELEASE_VERSION}"
80               else
81                   VERSION="${VERSION#v}"
82               fi
83               echo "version=${VERSION}" >> "$GITHUB_OUTPUT"
84         env:
85             VERSION: ${ inputs.version }
86             RELEASE_TYPE: ${ inputs.release_type }
87             PRE_RELEASE_VERSION: ${ inputs.pre_release_version }
88
89         - name: Checkout repo
90           uses: actions/checkout@v7.0.0
91           with:
92               fetch-depth: 0
93
94         - name: Update app version
95           run: 'sed -i "s/\"version\": \".*\"/,/\"version\": \"${ steps.version
.outputs.version }\"/,/g" package.json'
96
97         - name: Generate changelog
98           id: changelog
99           uses: TriPSs/conventional-changelog-action@v6.3.1
100          with:
101              github-token: ${ secrets.GITHUB_TOKEN }
102              skip-commit: "true"
103              skip-tag: "true"
104              skip-git-pull: "true"
105              skip-version-file: "true"
106              output-file: "CHANGELOG.md"
107              tag-prefix: "v"
108
109         - name: Commit and push version change
110           if: ${ inputs.release_type == 'prod' }
111           id: auto-commit
112           uses: stefanzweifel/git-auto-commit-action@v7.1.0
113           with:
114               commit_message: "chore: Update version to v${ steps.version.
.outputs.version }"

```

```

115
116   - name: Upload modified files
117     if: ${ inputs.release_type != 'prod' }
118     uses: actions/upload-artifact@v7.0.1
119     with:
120       name: release-files
121       path: |
122         package.json
123         CHANGELOG.md
124       retention-days: 1
125
126   - name: Extract metadata for Container
127     id: meta
128     uses: docker/metadata-action@v6.1.0
129     with:
130       images: ghcr.io/${ github.repository }
131       tags: type=semver,pattern={{version}},value=${ steps.version.
132         outputs.version }
133       labels: |
134         org.opencontainers.image.title=Infinity Front-End
135         org.opencontainers.image.description=Infinity front-end app image,
136         see https://github.com/infiniteuny/infinity-frontend/ for more info.
137         org.opencontainers.image.vendor=INFINITE UNY
138       annotations: |
139         org.opencontainers.image.title=Infinity Front-End
140         org.opencontainers.image.description=Infinity front-end app image,
141         see https://github.com/infiniteuny/infinity-frontend/ for more info.
142         org.opencontainers.image.vendor=INFINITE UNY
143     env:
144       DOCKER_METADATA_ANNOTATIONS_LEVELS: manifest,index
145
146 build:
147   name: Build (${ matrix.platform })
148   needs:
149     - prepare
150   runs-on: ${ matrix.runner }
151   permissions:
152     contents: read
153     packages: write
154   strategy:
155     fail-fast: false
156   matrix:
157     include:
158       - platform: linux/amd64
159         runner: ubuntu-24.04
160         scope: amd64
161       - platform: linux/arm64
162         runner: ubuntu-24.04-arm
163         scope: arm64
164   steps:
165     - name: Checkout repo
166       uses: actions/checkout@v7.0.0
167       with:
168         ref: ${ needs.prepare.outputs.commit_hash || github.ref_name }
169
170     - name: Download release files
171       if: ${ inputs.release_type != 'prod' }
172       uses: actions/download-artifact@v8.0.1
173       with:
174         name: release-files
175         path: .

```

```

173
174   - name: Set up Docker Buildx
175     uses: docker/setup-buildx-action@v4.1.0
176
177   - name: Login to GitHub Container Registry
178     uses: docker/login-action@v4.2.0
179     with:
180       registry: ghcr.io
181       username: ${github.repository_owner}
182       password: ${secrets.GITHUB_TOKEN}
183
184   - name: Build and push by digest
185     id: build
186     uses: docker/build-push-action@v7.2.0
187     with:
188       context: .
189       labels: ${needs.prepare.outputs.labels}
190       annotations: ${needs.prepare.outputs.annotations}
191       outputs: type=image,name=ghcr.io/${github.repository},push-by-
digest=true,name-canonical=true,push=true
192       platforms: ${matrix.platform}
193       cache-from: type=gha,scope=${matrix.scope}
194       cache-to: type=gha,scope=${matrix.scope},mode=max
195       build-args: |
196         NEXT_PUBLIC_APP_URL=${secrets.NEXT_PUBLIC_APP_URL}
197         NEXT_PUBLIC_BETTER_AUTH_URL=${secrets.NEXT_PUBLIC_APP_URL}
198         NEXT_PUBLIC_INFINITY_API_URL=${secrets.
NEXT_PUBLIC_INFINITY_API_URL}
199         NEXT_PUBLIC_INFINITY_API_TIMEOUT=${secrets.
NEXT_PUBLIC_INFINITY_API_TIMEOUT}
200
201   - name: Export digest
202     run: |
203       mkdir -p /tmp/digests
204       digest="${steps.build.outputs.digest}"
205       touch "/tmp/digests/${digest#sha256:}"
206
207   - name: Upload digest artifact
208     uses: actions/upload-artifact@v7.0.1
209     with:
210       name: digests-${matrix.scope}
211       path: /tmp/digests/*
212       if-no-files-found: error
213       retention-days: 1
214
215 merge:
216   name: Create multi-arch manifest
217   needs:
218     - prepare
219     - build
220   runs-on: ubuntu-latest
221   permissions:
222     packages: write
223   outputs:
224     digest: ${steps.manifest.outputs.digest}
225   steps:
226     - name: Download digests
227       uses: actions/download-artifact@v8.0.1
228       with:
229         path: /tmp/digests
230         pattern: digests-*

```



```

231         merge-multiple: true
232
233     - name: Prepare sources
234       id: sources
235       working-directory: /tmp/digests
236       run: |
237         {
238           echo 'sources<<EOF'
239           for f in *; do
240             echo "ghcr.io/${{ github.repository }}@sha256:${f}"
241           done
242           echo EOF
243         } >> "$GITHUB_OUTPUT"
244
245     - name: Login to GitHub Container Registry
246       uses: docker/login-action@v4.2.0
247       with:
248         registry: ghcr.io
249         username: ${{ github.repository_owner }}
250         password: ${{ secrets.GITHUB_TOKEN }}
251
252     - name: Create multi-arch manifest
253       id: manifest
254       uses: int128/docker-manifest-create-action@v2.24.0
255       with:
256         tags: ${{ needs.prepare.outputs.tags }}
257         index-annotations: ${{ needs.prepare.outputs.labels }}
258         sources: ${{ steps.sources.outputs.sources }}
259
260   sign:
261     name: Sign images
262     needs:
263       - prepare
264       - merge
265     runs-on: ubuntu-latest
266     permissions:
267       id-token: write
268       packages: write
269     steps:
270       - name: Install Cosign
271         uses: sigstore/cosign-installer@v4.1.2
272
273       - name: Login to GitHub Container Registry
274         uses: docker/login-action@v4.2.0
275         with:
276           registry: ghcr.io
277           username: ${{ github.repository_owner }}
278           password: ${{ secrets.GITHUB_TOKEN }}
279
280       - name: Sign the images with GitHub OIDC token
281         run: |
282           for tag in ${TAGS//,/ }; do
283             cosign sign --yes "${tag}@${DIGEST}"
284           done
285       env:
286         DIGEST: ${{ needs.merge.outputs.digest }}
287         TAGS: ${{ needs.prepare.outputs.tags }}
288
289   release:
290     name: Create GitHub release
291     needs:

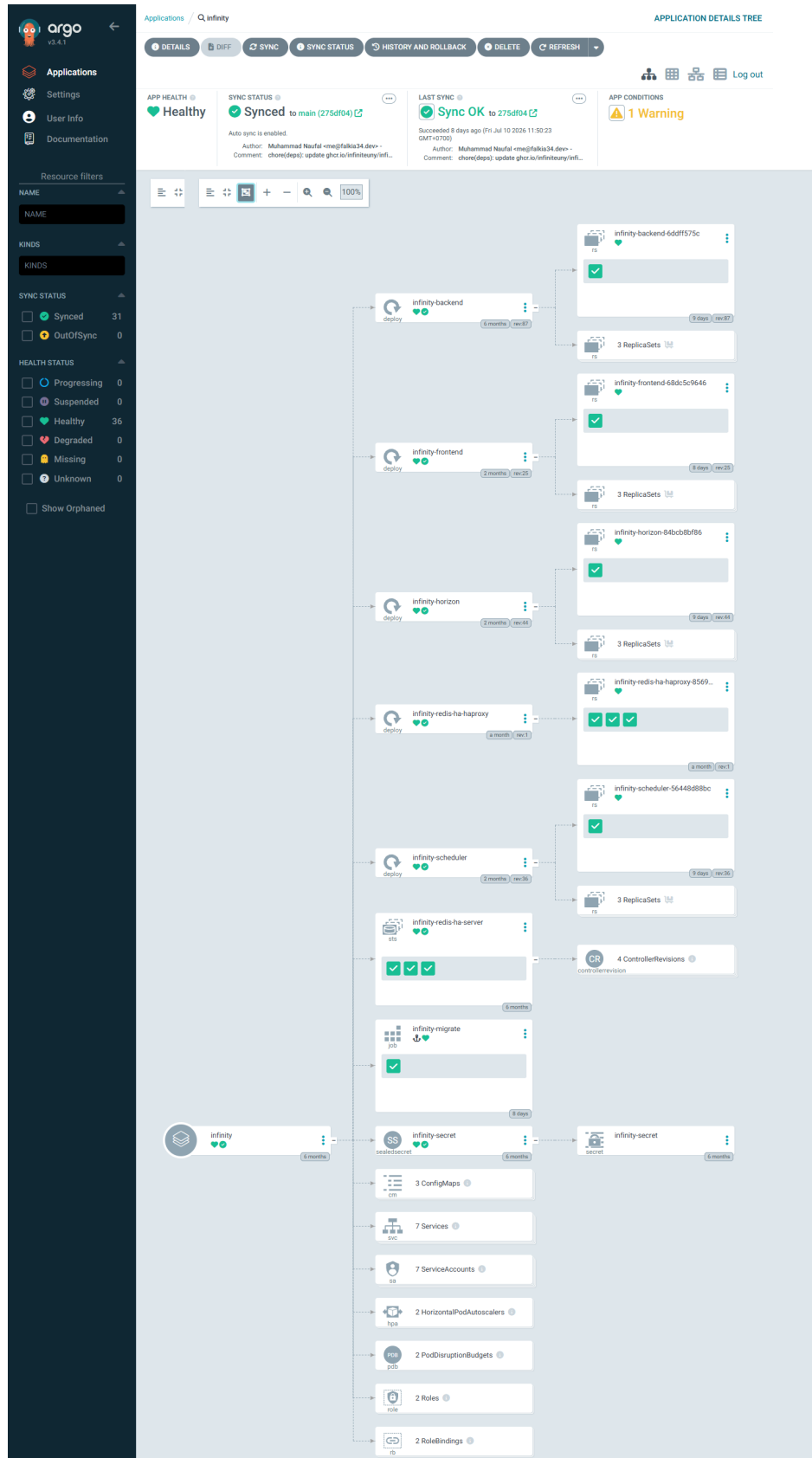
```

```

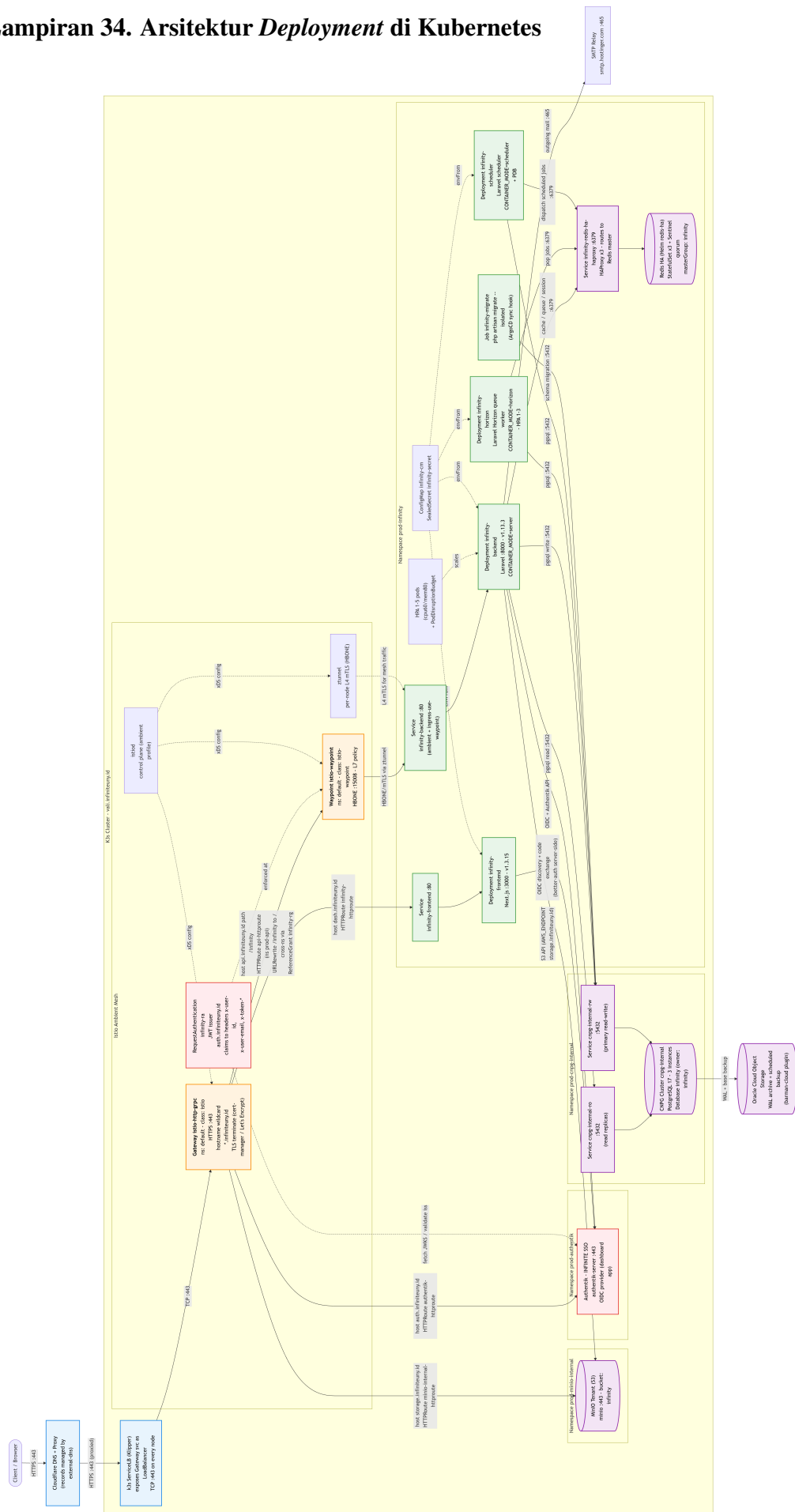
292     - guard
293     - prepare
294     - sign
295 runs-on: ubuntu-latest
296 permissions:
297     contents: write
298 steps:
299     - name: Checkout repo
300       uses: actions/checkout@v7.0.0
301       with:
302         ref: ${ needs.prepare.outputs.commit_hash || github.ref_name }
303
304     - name: Create release
305       uses: softprops/action-gh-release@v3.0.1
306       env:
307         GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
308       with:
309         tag_name: v${ needs.prepare.outputs.version }
310         name: v${ needs.prepare.outputs.version }
311         target_commitish: ${ github.ref_name }
312         prerelease: ${ inputs.release_type != 'prod' }
313         make_latest: "${ inputs.release_type == 'prod' }"
314         body: ${ needs.prepare.outputs.clean_changelog }

```

Lampiran 33. Deployment dengan ArgoCD



Lampiran 34. Arsitektur *Deployment* di Kubernetes



Lampiran 35. Hasil Pengujian *Functional Suitability* dan *Interaction Capability* Penguji 1

Angket Instrumen *Product Quality*

Judul Penelitian: Pengembangan Sistem Informasi Organisasi INFINITE dengan Laravel *Headless*, Next.js, dan Kontainer

Identitas Peneliti
Nama: Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM: 22051130001
Departemen: Pendidikan Teknik Elektronika dan Informatika
Program Studi: Teknologi Informasi

The respondent's email (xxxxxxxxxxxxxxxxxxxx) was recorded on submission of this form.

Nama *

Ikhwan

Jenis Kelamin *

Laki-laki

Dropdown

Usia *

24

Jabatan *

Software Engineer

Instansi *

UNU Jogja

Functional Suitability

Petunjuk Pengisian

1. Buka situs [INFINITE Dashboard \(https://dash.infiniteuny.id/\)](https://dash.infiniteuny.id/).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62).

Penilaian *Functional Suitability* *

	Ya	Tidak
Sistem dapat melakukan autentikasi melalui SSO INFINITE	☒	☐
Sistem dapat mengelola data akademik (strata, fakultas, program studi)	☒	☐
Sistem dapat mengelola data anggota (User)	☒	☐
Sistem dapat mengelola data kategori anggota (Persona)	☒	☐
Sistem dapat mengelola data kepengurusan (Core Team)	☒	☐
Sistem dapat mengelola data anggota kepengurusan (Core Team Member) berupa divisi dan foto	☒	☐
Sistem dapat mengelola data kelompok komunitas (Community Group)	☒	☐
Sistem dapat mengelola data anggota kelompok komunitas (Community Group Member)	☒	☐
Sistem dapat mengelola data administrator kelompok komunitas (Community Group Admin)	☒	☐
Sistem dapat mengelola data anggota administrator kelompok komunitas (Community Group Admin Member) berupa kelompok dan foto	☒	☐
Sistem dapat mengelola data seri kompetisi (Competition) dan edisi pelaksanaan kompetisi (Competition Instance)	☒	☐
Sistem dapat mengelola data tim (Team) dan anggota tim (Team Member) untuk keperluan kompetisi	☒	☐
Sistem dapat mengelola data prestasi (Achievement) beserta status persetujuan	☒	☐
Sistem dapat mengelola data pengajuan dana (Fund Application) beserta status persetujuan	☒	☐
Sistem dapat mengelola data testimoni (Testimonial) dan galeri proyek (Project Gallery)	☒	☐
Sistem dapat mengelola hak akses pengguna melalui grup (Group) dan izin (Permission)	☒	☐
Sistem dapat mengelola konfigurasi sistem (Config) termasuk pengaturan daftar ulang keanggotaan	☒	☐
Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan	☒	☐
Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (Pending/Accepted/Rejected)	☒	☐

Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar	☒	☐
Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data	☒	☐
Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar	☒	☐
Sistem menampilkan halaman pengaturan (Settings) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi	☒	☐

Usability

Petunjuk Pengisian

1. Buka situs [INFINITE Dashboard \(https://dash.infiniteuny.id/\)](https://dash.infiniteuny.id/).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Keterangan kolom penilaian:

- **STS:** Sangat Tidak Setuju (1)
- **TS:** Tidak Setuju (2)
- **N:** Netral (3)
- **S:** Setuju (4)
- **SS:** Sangat Setuju (5)

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62).

Penilaian Usability *

	STS (1)	TS (2)	N (3)	S (4)	SS (5)
Sistem membantu saya menjadi lebih efektif	☐	☐	☐	☒	☐
Sistem membantu saya menjadi lebih produktif	☐	☐	☐	☒	☐
Sistem ini berguna	☐	☐	☐	☒	☐
Sistem memberi saya lebih banyak kendali atas aktivitas saya	☐	☐	☐	☒	☐
Sistem mempermudah penyelesaian hal-hal yang ingin saya selesaikan	☐	☐	☐	☒	☐
Sistem menghemat waktu saya saat menggunakannya	☐	☐	☐	☐	☒
Sistem ini memenuhi kebutuhan saya	☐	☐	☐	☒	☐
Sistem melakukan semua hal yang saya harapkan	☐	☐	☐	☒	☐
Sistem ini mudah digunakan	☐	☐	☐	☒	☐
Sistem ini simpel untuk digunakan	☐	☐	☐	☒	☐
Sistem ini ramah pengguna (user friendly)	☐	☐	☐	☒	☐
Sistem hanya membutuhkan sedikit langkah untuk menyelesaikan apa yang ingin saya lakukan	☐	☐	☐	☒	☐
Sistem ini fleksibel	☐	☐	☐	☐	☒
Menggunakan sistem ini tidak membutuhkan banyak usaha	☐	☐	☐	☒	☐
Saya dapat menggunakan sistem ini tanpa petunjuk tertulis	☐	☐	☐	☒	☐
Saya tidak menemukan adanya inkonsistensi saat menggunakan sistem ini	☐	☐	☐	☒	☐
Pengguna awam (occasional) maupun pengguna rutin (regular) akan menyukai sistem ini	☐	☐	☐	☒	☐

Saya dapat memulihkan (memperbaiki) kesalahan dengan cepat dan mudah	☐	☐	☐	☒	☐
Saya dapat menggunakan sistem ini dengan sukses setiap saat	☐	☐	☐	☒	☐
Saya belajar menggunakan sistem ini dengan cepat	☐	☐	☐	☒	☐
Saya mudah mengingat cara menggunakan sistem ini	☐	☐	☐	☒	☐
Sistem ini mudah dipelajari cara penggunaannya	☐	☐	☐	☒	☐
Saya dengan cepat menjadi mahir menggunakan sistem ini	☐	☐	☐	☒	☐
Saya merasa puas dengan sistem ini	☐	☐	☐	☒	☐
Saya akan merekomendasikan sistem ini kepada teman	☐	☐	☐	☒	☐
Sistem ini menyenangkan untuk digunakan	☐	☐	☐	☒	☐
Sistem bekerja sesuai dengan yang saya inginkan	☐	☐	☐	☒	☐
Sistem ini luar biasa	☐	☐	☐	☒	☐
Saya merasa perlu memiliki sistem ini	☐	☐	☐	☒	☐
Sistem ini menyenangkan/nyaman digunakan	☐	☐	☐	☒	☐

Komentar atau Saran *

Warna sangat monokrom

Tanda Tangan *

☒ Saya menyatakan telah mengisi angket ini dengan sebenar-benarnya.

This form was created inside Universitas Negeri Yogyakarta.

Google Forms

Penguji 2

Angket Instrumen *Product Quality*

Judul Penelitian: **Pengembangan Sistem Informasi Organisasi INFINITE dengan Laravel *Headless*, Next.js, dan Kontainer**

Identitas Peneliti
Nama: Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM: 22051130001
Departemen: Pendidikan Teknik Elektronika dan Informatika
Program Studi: Teknologi Informasi

The respondent's email (xxxxxxxxxxxxxxxxxxxx) was recorded on submission of this form.

Nama *

Zainal Ma'ruf Abidin

Jenis Kelamin *

Laki-laki

Dropdown

Usia *

23

Jabatan *

UI UX Designer

Instansi *

PT Digdaya Inovasi Gemilang

Functional Suitability

Petunjuk Pengisian

1. Buka situs **INFINITE Dashboard** (<https://dash.infiniteuny.id/>).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Apabila ada keraguan, silakan hubungi peneliti via **WA** (<https://wa.me/62>) .

Penilaian *Functional Suitability* *

	Ya	Tidak
Sistem dapat melakukan autentikasi melalui SSO INFINITE	☒	☐
Sistem dapat mengelola data akademik (strata, fakultas, program studi)	☒	☐
Sistem dapat mengelola data anggota (User)	☒	☐
Sistem dapat mengelola data kategori anggota (Persona)	☒	☐
Sistem dapat mengelola data kepengurusan (Core Team)	☒	☐
Sistem dapat mengelola data anggota kepengurusan (Core Team Member) berupa divisi dan foto	☒	☐
Sistem dapat mengelola data kelompok komunitas (Community Group)	☒	☐
Sistem dapat mengelola data anggota kelompok komunitas (Community Group Member)	☒	☐
Sistem dapat mengelola data administrator kelompok komunitas (Community Group Admin)	☒	☐
Sistem dapat mengelola data anggota administrator kelompok komunitas (Community Group Admin Member) berupa kelompok dan foto	☒	☐
Sistem dapat mengelola data seri kompetisi (Competition) dan edisi pelaksanaan kompetisi (Competition Instance)	☒	☐
Sistem dapat mengelola data tim (Team) dan anggota tim (Team Member) untuk keperluan kompetisi	☒	☐
Sistem dapat mengelola data prestasi (Achievement) beserta status persetujuan	☒	☐
Sistem dapat mengelola data pengajuan dana (Fund Application) beserta status persetujuan	☒	☐
Sistem dapat mengelola data testimoni (Testimonial) dan galeri proyek (Project Gallery)	☒	☐
Sistem dapat mengelola hak akses pengguna melalui grup (Group) dan izin (Permission)	☒	☐
Sistem dapat mengelola konfigurasi sistem (Config) termasuk pengaturan daftar ulang keanggotaan	☒	☐
Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan	☒	☐
Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (Pending/Accepted/Rejected)	☒	☐

Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar	☒	☐
Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data	☒	☐
Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar	☒	☐
Sistem menampilkan halaman pengaturan (Settings) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi	☒	☐

Usability

Petunjuk Pengisian

1. Buka situs [INFINITE Dashboard \(https://dash.infiniteuny.id/\)](https://dash.infiniteuny.id/).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Keterangan kolom penilaian:

- **STS:** Sangat Tidak Setuju (1)
- **TS:** Tidak Setuju (2)
- **N:** Netral (3)
- **S:** Setuju (4)
- **SS:** Sangat Setuju (5)

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62).

Penilaian Usability *

	STS (1)	TS (2)	N (3)	S (4)	SS (5)
Sistem membantu saya menjadi lebih efektif	☐	☐	☐	☒	☐
Sistem membantu saya menjadi lebih produktif	☐	☐	☐	☒	☐
Sistem ini berguna	☐	☐	☐	☐	☒
Sistem memberi saya lebih banyak kendali atas aktivitas saya	☐	☐	☐	☐	☒
Sistem mempermudah penyelesaian hal-hal yang ingin saya selesaikan	☐	☐	☐	☒	☐
Sistem menghemat waktu saya saat menggunakannya	☐	☐	☐	☒	☐
Sistem ini memenuhi kebutuhan saya	☐	☐	☐	☐	☒
Sistem melakukan semua hal yang saya harapkan	☐	☐	☐	☒	☐
Sistem ini mudah digunakan	☐	☐	☐	☐	☒
Sistem ini simpel untuk digunakan	☐	☐	☐	☐	☒
Sistem ini ramah pengguna (user friendly)	☐	☐	☐	☐	☒
Sistem hanya membutuhkan sedikit langkah untuk menyelesaikan apa yang ingin saya lakukan	☐	☐	☐	☐	☒
Sistem ini fleksibel	☐	☐	☐	☒	☐
Menggunakan sistem ini tidak membutuhkan banyak usaha	☐	☐	☐	☒	☐
Saya dapat menggunakan sistem ini tanpa petunjuk tertulis	☐	☐	☐	☐	☒
Saya tidak menemukan adanya inkonsistensi saat menggunakan sistem ini	☐	☐	☐	☒	☐
Pengguna awam (occasional) maupun pengguna rutin (regular) akan menyukai sistem ini	☐	☐	☐	☒	☐

Saya dapat memulihkan (memperbaiki) kesalahan dengan cepat dan mudah	☐	☐	☐	☒	☐
Saya dapat menggunakan sistem ini dengan sukses setiap saat	☐	☐	☐	☐	☒
Saya belajar menggunakan sistem ini dengan cepat	☐	☐	☐	☐	☒
Saya mudah mengingat cara menggunakan sistem ini	☐	☐	☐	☐	☒
Sistem ini mudah dipelajari cara penggunaannya	☐	☐	☐	☐	☒
Saya dengan cepat menjadi mahir menggunakan sistem ini	☐	☐	☐	☐	☒
Saya merasa puas dengan sistem ini	☐	☐	☐	☒	☐
Saya akan merekomendasikan sistem ini kepada teman	☐	☐	☐	☒	☐
Sistem ini menyenangkan untuk digunakan	☐	☐	☐	☒	☐
Sistem bekerja sesuai dengan yang saya inginkan	☐	☐	☐	☒	☐
Sistem ini luar biasa	☐	☐	☐	☒	☐
Saya merasa perlu memiliki sistem ini	☐	☐	☐	☒	☐
Sistem ini menyenangkan/nyaman digunakan	☐	☐	☐	☒	☐

Komentar atau Saran *

Semua fitur pada sistem sudah berjalan dengan baik.

Page overview bisa ditambahkan ringkasan informasi/data sistem saat ini (salah satunya mungkin terkait informasi pendanaan yg masih dalam proses)

Tanda Tangan *

☒ Saya menyatakan telah mengisi angket ini dengan sebenar-benarnya.

Penguji 3

Angket Instrumen *Product Quality*

Judul Penelitian: **Pengembangan Sistem Informasi Organisasi INFINITE dengan Laravel *Headless*, Next.js, dan Kontainer**

Identitas Peneliti
Nama: Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM: 22051130001
Departemen: Pendidikan Teknik Elektronika dan Informatika
Program Studi: Teknologi Informasi

The respondent's email (xxxxxxxxxxxxxxxxxxxx) was recorded on submission of this form.

Nama *

Damar Albaribin Syamsu

Jenis Kelamin *

Laki-laki

Dropdown

Usia *

25

Jabatan *

AI Software Engineer

Instansi *

Pelgo Incorporated

Functional Suitability

Petunjuk Pengisian

1. Buka situs **INFINITE Dashboard** (<https://dash.infiniteuny.id/>).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Apabila ada keraguan, silakan hubungi peneliti via **WA** (<https://wa.me/62>) .

Penilaian *Functional Suitability* *

	Ya	Tidak
Sistem dapat melakukan autentikasi melalui SSO INFINITE	☒	☐
Sistem dapat mengelola data akademik (strata, fakultas, program studi)	☒	☐
Sistem dapat mengelola data anggota (User)	☒	☐
Sistem dapat mengelola data kategori anggota (Persona)	☒	☐
Sistem dapat mengelola data kepengurusan (Core Team)	☒	☐
Sistem dapat mengelola data anggota kepengurusan (Core Team Member) berupa divisi dan foto	☒	☐
Sistem dapat mengelola data kelompok komunitas (Community Group)	☒	☐
Sistem dapat mengelola data anggota kelompok komunitas (Community Group Member)	☒	☐
Sistem dapat mengelola data administrator kelompok komunitas (Community Group Admin)	☒	☐
Sistem dapat mengelola data anggota administrator kelompok komunitas (Community Group Admin Member) berupa kelompok dan foto	☒	☐
Sistem dapat mengelola data seri kompetisi (Competition) dan edisi pelaksanaan kompetisi (Competition Instance)	☒	☐
Sistem dapat mengelola data tim (Team) dan anggota tim (Team Member) untuk keperluan kompetisi	☒	☐
Sistem dapat mengelola data prestasi (Achievement) beserta status persetujuan	☒	☐
Sistem dapat mengelola data pengajuan dana (Fund Application) beserta status persetujuan	☒	☐
Sistem dapat mengelola data testimoni (Testimonial) dan galeri proyek (Project Gallery)	☒	☐
Sistem dapat mengelola hak akses pengguna melalui grup (Group) dan izin (Permission)	☒	☐
Sistem dapat mengelola konfigurasi sistem (Config) termasuk pengaturan daftar ulang keanggotaan	☒	☐
Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan	☒	☐
Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (Pending/Accepted/Rejected)	☒	☐

Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar	☒	☐
Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data	☒	☐
Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar	☒	☐
Sistem menampilkan halaman pengaturan (Settings) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi	☒	☐

Usability

Petunjuk Pengisian

1. Buka situs [INFINITE Dashboard \(https://dash.infiniteuny.id/\)](https://dash.infiniteuny.id/).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Keterangan kolom penilaian:

- **STS:** Sangat Tidak Setuju (1)
- **TS:** Tidak Setuju (2)
- **N:** Netral (3)
- **S:** Setuju (4)
- **SS:** Sangat Setuju (5)

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62).

Penilaian Usability *

	STS (1)	TS (2)	N (3)	S (4)	SS (5)
Sistem membantu saya menjadi lebih efektif	☐	☐	☐	☐	☒
Sistem membantu saya menjadi lebih produktif	☐	☐	☐	☐	☒
Sistem ini berguna	☐	☐	☐	☐	☒
Sistem memberi saya lebih banyak kendali atas aktivitas saya	☐	☐	☐	☐	☒
Sistem mempermudah penyelesaian hal-hal yang ingin saya selesaikan	☐	☐	☐	☐	☒
Sistem menghemat waktu saya saat menggunakannya	☐	☐	☐	☐	☒
Sistem ini memenuhi kebutuhan saya	☐	☐	☐	☐	☒
Sistem melakukan semua hal yang saya harapkan	☐	☐	☐	☐	☒
Sistem ini mudah digunakan	☐	☐	☐	☐	☒
Sistem ini simpel untuk digunakan	☐	☐	☐	☐	☒
Sistem ini ramah pengguna (user friendly)	☐	☐	☐	☐	☒
Sistem hanya membutuhkan sedikit langkah untuk menyelesaikan apa yang ingin saya lakukan	☐	☐	☐	☐	☒
Sistem ini fleksibel	☐	☐	☐	☐	☒
Menggunakan sistem ini tidak membutuhkan banyak usaha	☐	☐	☐	☐	☒
Saya dapat menggunakan sistem ini tanpa petunjuk tertulis	☐	☐	☐	☐	☒
Saya tidak menemukan adanya inkonsistensi saat menggunakan sistem ini	☐	☐	☐	☐	☒
Pengguna awam (occasional) maupun pengguna rutin (regular) akan menyukai sistem ini	☐	☐	☐	☐	☒

Saya dapat memulihkan (memperbaiki) kesalahan dengan cepat dan mudah	☐	☐	☐	☐	☒
Saya dapat menggunakan sistem ini dengan sukses setiap saat	☐	☐	☐	☐	☒
Saya belajar menggunakan sistem ini dengan cepat	☐	☐	☐	☐	☒
Saya mudah mengingat cara menggunakan sistem ini	☐	☐	☐	☐	☒
Sistem ini mudah dipelajari cara penggunaannya	☐	☐	☐	☐	☒
Saya dengan cepat menjadi mahir menggunakan sistem ini	☐	☐	☐	☐	☒
Saya merasa puas dengan sistem ini	☐	☐	☐	☐	☒
Saya akan merekomendasikan sistem ini kepada teman	☐	☐	☐	☐	☒
Sistem ini menyenangkan untuk digunakan	☐	☐	☐	☐	☒
Sistem bekerja sesuai dengan yang saya inginkan	☐	☐	☐	☐	☒
Sistem ini luar biasa	☐	☐	☐	☐	☒
Saya merasa perlu memiliki sistem ini	☐	☐	☐	☐	☒
Sistem ini menyenangkan/nyaman digunakan	☐	☐	☐	☐	☒

Komentar atau Saran *

Sudah bagus

Tanda Tangan *

☒ Saya menyatakan telah mengisi angket ini dengan sebenar-benarnya.

This form was created inside Universitas Negeri Yogyakarta.

Google Forms

Penguji 4

Angket Instrumen *Product Quality*

Judul Penelitian: **Pengembangan Sistem Informasi Organisasi INFINITE dengan Laravel *Headless*, Next.js, dan Kontainer**

Identitas Peneliti
Nama: Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM: 22051130001
Departemen: Pendidikan Teknik Elektronika dan Informatika
Program Studi: Teknologi Informasi

The respondent's email (xxxxxxxxxxxxxxxxxxxx) was recorded on submission of this form.

Nama *

Satya Adhiyaksa Ardy

Jenis Kelamin *

Laki-laki

Dropdown

Usia *

25

Jabatan *

System Software Engineer

Instansi *

Sillicon XPandas

Functional Suitability

Petunjuk Pengisian

1. Buka situs **INFINITE Dashboard** (<https://dash.infiniteuny.id/>).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Apabila ada keraguan, silakan hubungi peneliti via **WA** (<https://wa.me/62>) .

Penilaian *Functional Suitability* *

	Ya	Tidak
Sistem dapat melakukan autentikasi melalui SSO INFINITE	☒	☐
Sistem dapat mengelola data akademik (strata, fakultas, program studi)	☒	☐
Sistem dapat mengelola data anggota (User)	☒	☐
Sistem dapat mengelola data kategori anggota (Persona)	☒	☐
Sistem dapat mengelola data kepengurusan (Core Team)	☒	☐
Sistem dapat mengelola data anggota kepengurusan (Core Team Member) berupa divisi dan foto	☒	☐
Sistem dapat mengelola data kelompok komunitas (Community Group)	☒	☐
Sistem dapat mengelola data anggota kelompok komunitas (Community Group Member)	☒	☐
Sistem dapat mengelola data administrator kelompok komunitas (Community Group Admin)	☒	☐
Sistem dapat mengelola data anggota administrator kelompok komunitas (Community Group Admin Member) berupa kelompok dan foto	☒	☐
Sistem dapat mengelola data seri kompetisi (Competition) dan edisi pelaksanaan kompetisi (Competition Instance)	☒	☐
Sistem dapat mengelola data tim (Team) dan anggota tim (Team Member) untuk keperluan kompetisi	☒	☐
Sistem dapat mengelola data prestasi (Achievement) beserta status persetujuan	☒	☐
Sistem dapat mengelola data pengajuan dana (Fund Application) beserta status persetujuan	☒	☐
Sistem dapat mengelola data testimoni (Testimonial) dan galeri proyek (Project Gallery)	☒	☐
Sistem dapat mengelola hak akses pengguna melalui grup (Group) dan izin (Permission)	☒	☐
Sistem dapat mengelola konfigurasi sistem (Config) termasuk pengaturan daftar ulang keanggotaan	☒	☐
Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan	☒	☐
Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (Pending/Accepted/Rejected)	☒	☐

Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar	☒	☐
Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data	☒	☐
Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar	☒	☐
Sistem menampilkan halaman pengaturan (Settings) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi	☒	☐

Usability

Petunjuk Pengisian

1. Buka situs [INFINITE Dashboard \(https://dash.infiniteuny.id/\)](https://dash.infiniteuny.id/).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Keterangan kolom penilaian:

- **STS**: Sangat Tidak Setuju (1)
- **TS**: Tidak Setuju (2)
- **N**: Netral (3)
- **S**: Setuju (4)
- **SS**: Sangat Setuju (5)

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62).

Penilaian Usability *

	STS (1)	TS (2)	N (3)	S (4)	SS (5)
Sistem membantu saya menjadi lebih efektif	☐	☐	☐	☐	☒
Sistem membantu saya menjadi lebih produktif	☐	☐	☐	☐	☒
Sistem ini berguna	☐	☐	☐	☐	☒
Sistem memberi saya lebih banyak kendali atas aktivitas saya	☐	☐	☐	☐	☒
Sistem mempermudah penyelesaian hal-hal yang ingin saya selesaikan	☐	☐	☐	☐	☒
Sistem menghemat waktu saya saat menggunakannya	☐	☐	☐	☐	☒
Sistem ini memenuhi kebutuhan saya	☐	☐	☐	☐	☒
Sistem melakukan semua hal yang saya harapkan	☐	☐	☐	☒	☐
Sistem ini mudah digunakan	☐	☐	☐	☒	☐
Sistem ini simpel untuk digunakan	☐	☐	☒	☐	☐
Sistem ini ramah pengguna (user friendly)	☐	☐	☐	☒	☐
Sistem hanya membutuhkan sedikit langkah untuk menyelesaikan apa yang ingin saya lakukan	☐	☐	☐	☒	☐
Sistem ini fleksibel	☐	☐	☐	☐	☒
Menggunakan sistem ini tidak membutuhkan banyak usaha	☐	☐	☐	☒	☐
Saya dapat menggunakan sistem ini tanpa petunjuk tertulis	☐	☐	☒	☐	☐
Saya tidak menemukan adanya inkonsistensi saat menggunakan sistem ini	☐	☐	☐	☐	☒
Pengguna awam (occasional) maupun pengguna rutin (regular) akan menyukai sistem ini	☐	☐	☐	☒	☐

Saya dapat memulihkan (memperbaiki) kesalahan dengan cepat dan mudah	☐	☐	☐	☐	☒
Saya dapat menggunakan sistem ini dengan sukses setiap saat	☐	☐	☐	☐	☒
Saya belajar menggunakan sistem ini dengan cepat	☐	☐	☐	☒	☐
Saya mudah mengingat cara menggunakan sistem ini	☐	☐	☐	☒	☐
Sistem ini mudah dipelajari cara penggunaannya	☐	☐	☐	☒	☐
Saya dengan cepat menjadi mahir menggunakan sistem ini	☐	☐	☐	☒	☐
Saya merasa puas dengan sistem ini	☐	☐	☐	☐	☒
Saya akan merekomendasikan sistem ini kepada teman	☐	☐	☐	☐	☒
Sistem ini menyenangkan untuk digunakan	☐	☐	☐	☐	☒
Sistem bekerja sesuai dengan yang saya inginkan	☐	☐	☐	☐	☒
Sistem ini luar biasa	☐	☐	☐	☐	☒
Saya merasa perlu memiliki sistem ini	☐	☐	☐	☐	☒
Sistem ini menyenangkan/nyaman digunakan	☐	☐	☐	☐	☒

Komentar atau Saran *

Sistem sudah cukup baik dan sangat membantu kelancaran administrasi komunitas. Sebagai saran, mungkin ke depannya bisa ditambahkan fitur panduan singkat (tooltips atau pop-up tour) pada beberapa menu pengaturan. Hal ini akan sangat membantu administrator baru agar bisa langsung menggunakan sistem dengan maksimal tanpa kebingungan di awal.

Tanda Tangan *

☒ Saya menyatakan telah mengisi angket ini dengan sebenar-benarnya.

Penguji 5

Angket Instrumen *Product Quality*

Judul Penelitian: **Pengembangan Sistem Informasi Organisasi INFINITE dengan Laravel Headless, Next.js, dan Kontainer**

Identitas Peneliti
Nama: Muhammad Naufal Khoirul Imamilhaq Alhifdi
NIM: 22051130001
Departemen: Pendidikan Teknik Elektronika dan Informatika
Program Studi: Teknologi Informasi

The respondent's email (xxxxxxxxxxxxxxxxxxxx) was recorded on submission of this form.

Nama *

Dzul Fadli Rahman, S.Kom., M.Sc.

Jenis Kelamin *

Laki-laki

Dropdown

Usia *

31

Jabatan *

Dosen

Instansi *

Universitas Negeri Yogyakarta

Functional Suitability

Petunjuk Pengisian

1. Buka situs **INFINITE Dashboard** (<https://dash.infiniteuny.id/>).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62).

Penilaian *Functional Suitability* *

	Ya	Tidak
Sistem dapat melakukan autentikasi melalui SSO INFINITE	☒	☐
Sistem dapat mengelola data akademik (strata, fakultas, program studi)	☒	☐
Sistem dapat mengelola data anggota (User)	☒	☐
Sistem dapat mengelola data kategori anggota (Persona)	☒	☐
Sistem dapat mengelola data kepengurusan (Core Team)	☒	☐
Sistem dapat mengelola data anggota kepengurusan (Core Team Member) berupa divisi dan foto	☒	☐
Sistem dapat mengelola data kelompok komunitas (Community Group)	☒	☐
Sistem dapat mengelola data anggota kelompok komunitas (Community Group Member)	☒	☐
Sistem dapat mengelola data administrator kelompok komunitas (Community Group Admin)	☒	☐
Sistem dapat mengelola data anggota administrator kelompok komunitas (Community Group Admin Member) berupa kelompok dan foto	☒	☐
Sistem dapat mengelola data seri kompetisi (Competition) dan edisi pelaksanaan kompetisi (Competition Instance)	☒	☐
Sistem dapat mengelola data tim (Team) dan anggota tim (Team Member) untuk keperluan kompetisi	☒	☐
Sistem dapat mengelola data prestasi (Achievement) beserta status persetujuan	☒	☐
Sistem dapat mengelola data pengajuan dana (Fund Application) beserta status persetujuan	☒	☐
Sistem dapat mengelola data testimoni (Testimonial) dan galeri proyek (Project Gallery)	☒	☐
Sistem dapat mengelola hak akses pengguna melalui grup (Group) dan izin (Permission)	☒	☐
Sistem dapat mengelola konfigurasi sistem (Config) termasuk pengaturan daftar ulang keanggotaan	☒	☐
Sistem menampilkan data profil anggota secara akurat sesuai data yang tersimpan	☒	☐
Sistem menampilkan status persetujuan prestasi dan pengajuan dana secara tepat (Pending/Accepted/Rejected)	☒	☐

Sistem memfilter menu navigasi berdasarkan hak akses pengguna yang sedang login secara benar	☒	☐
Sistem menyediakan fitur pencarian dan filter data pada setiap daftar entitas untuk memudahkan pengelolaan data	☒	☐
Sistem menyediakan paginasi pada daftar data untuk memudahkan navigasi data dalam jumlah besar	☒	☐
Sistem menampilkan halaman pengaturan (Settings) yang terorganisir dalam kategori untuk memudahkan akses konfigurasi	☒	☐

Usability

Petunjuk Pengisian

1. Buka situs **INFINITE Dashboard** (<https://dash.infiniteuny.id/>).
2. Masuk melalui INFINITE SSO dengan menggunakan akun yang diberikan kepada Anda.
3. Berilah penilaian pada tabel di bawah yang sesuai berdasarkan pengalaman pengoperasian sistem.

Keterangan kolom penilaian:

- **STS**: Sangat Tidak Setuju (1)
- **TS**: Tidak Setuju (2)
- **N**: Netral (3)
- **S**: Setuju (4)
- **SS**: Sangat Setuju (5)

Apabila ada keraguan, silakan hubungi peneliti via [WA \(https://wa.me/62\)](https://wa.me/62) .

Penilaian Usability *

	STS (1)	TS (2)	N (3)	S (4)	SS (5)
Sistem membantu saya menjadi lebih efektif	☐	☐	☐	☐	☒
Sistem membantu saya menjadi lebih produktif	☐	☐	☐	☒	☐
Sistem ini berguna	☐	☐	☐	☐	☒
Sistem memberi saya lebih banyak kendali atas aktivitas saya	☐	☐	☐	☒	☐
Sistem mempermudah penyelesaian hal-hal yang ingin saya selesaikan	☐	☐	☐	☐	☒
Sistem menghemat waktu saya saat menggunakannya	☐	☐	☐	☐	☒
Sistem ini memenuhi kebutuhan saya	☐	☐	☐	☐	☒
Sistem melakukan semua hal yang saya harapkan	☐	☐	☐	☒	☐
Sistem ini mudah digunakan	☐	☐	☐	☐	☒
Sistem ini simpel untuk digunakan	☐	☐	☐	☐	☒
Sistem ini ramah pengguna (user friendly)	☐	☐	☐	☐	☒
Sistem hanya membutuhkan sedikit langkah untuk menyelesaikan apa yang ingin saya lakukan	☐	☐	☐	☐	☒
Sistem ini fleksibel	☐	☐	☐	☐	☒
Menggunakan sistem ini tidak membutuhkan banyak usaha	☐	☐	☐	☐	☒
Saya dapat menggunakan sistem ini tanpa petunjuk tertulis	☐	☐	☐	☐	☒
Saya tidak menemukan adanya inkonsistensi saat menggunakan sistem ini	☐	☐	☐	☐	☒
Pengguna awam (occasional) maupun pengguna rutin (regular) akan menyukai sistem ini	☐	☐	☐	☐	☒

Saya dapat memulihkan (memperbaiki) kesalahan dengan cepat dan mudah	☐	☐	☐	☐	☒
Saya dapat menggunakan sistem ini dengan sukses setiap saat	☐	☐	☐	☐	☒
Saya belajar menggunakan sistem ini dengan cepat	☐	☐	☐	☐	☒
Saya mudah mengingat cara menggunakan sistem ini	☐	☐	☐	☐	☒
Sistem ini mudah dipelajari cara penggunaannya	☐	☐	☐	☐	☒
Saya dengan cepat menjadi mahir menggunakan sistem ini	☐	☐	☐	☐	☒
Saya merasa puas dengan sistem ini	☐	☐	☐	☐	☒
Saya akan merekomendasikan sistem ini kepada teman	☐	☐	☐	☐	☒
Sistem ini menyenangkan untuk digunakan	☐	☐	☐	☐	☒
Sistem bekerja sesuai dengan yang saya inginkan	☐	☐	☐	☐	☒
Sistem ini luar biasa	☐	☐	☐	☐	☒
Saya merasa perlu memiliki sistem ini	☐	☐	☐	☐	☒
Sistem ini menyenangkan/nyaman digunakan	☐	☐	☐	☐	☒

Komentar atau Saran *

Bisa ditambahkan fitur chatbot berbasis AI, agar lebih menarik.

Tanda Tangan *

☒ Saya menyatakan telah mengisi angket ini dengan sebenar-benarnya.

This form was created inside Universitas Negeri Yogyakarta.

Google Forms

Lampiran 36. Hasil Pengujian *Performance Efficiency Frontend*

No.	URL	FCP	LCP	TBT	CLS	SI	Skor
1	/	100	95	95	100	40	91
2	/achievements	100	99	65	100	72	86
3	/achievements/[id]	100	100	65	100	83	88
4	/achievements/[id]/edit	100	100	65	100	61	86
5	/achievements/new	100	99	98	100	78	97
6	/community-group-admins	99	94	68	100	74	86
7	/community-group-admins/[id]	100	97	73	100	86	90
8	/community-group-admins/[id] /edit	98	96	71	100	77	88
9	/community-group-admins/[id] /members	100	99	82	100	75	92
10	/community-group-admins/[id] /members/[memberId]	100	92	67	99	78	86
11	/community-group-admins/[id] /members/[memberId]/edit	100	53	91	100	70	83
12	/community-group-admins/[id] /members/new	100	100	99	100	59	96
13	/community-group-admins/new	100	100	100	100	94	99
14	/community-groups	100	100	42	100	80	81
15	/community-groups/[id]	99	77	80	100	84	87
16	/community-groups/[id]/edit	99	85	67	99	63	82
17	/community-groups/[id]/members	100	98	66	100	73	87
18	/community-groups/[id] /members/new	100	100	49	100	71	82
19	/community-groups/new	100	94	100	100	91	98
20	/competition-organizer-types	100	93	100	100	76	96
21	/competition-organizer-types/ [id]	99	88	62	100	77	83
22	/competition-organizer-types/ [id]/edit	100	98	100	100	87	98
23	/competition-organizer-types/ new	100	96	100	100	91	98
24	/competition-outputs	98	99	74	100	74	89
25	/competition-outputs/[id]	99	88	58	100	76	82
26	/competition-outputs/[id]/edit	100	100	47	100	67	81
27	/competition-outputs/new	100	91	99	100	88	96
28	/competition-ranks	99	99	55	100	66	83
29	/competition-ranks/[id]	99	84	71	100	73	84
30	/competition-ranks/[id]/edit	69	87	88	100	44	84
31	/competition-ranks/new	100	99	100	100	95	99
32	/competition-scales	99	91	63	100	62	83
33	/competition-scales/[id]	98	93	61	100	61	82
34	/competition-scales/[id]/edit	98	94	58	100	66	82
35	/competition-scales/new	100	94	100	100	93	98
36	/competition-time-ranges	99	89	67	100	63	84
37	/competition-time-ranges/[id]	100	100	44	100	83	82
38	/competition-time-ranges/[id] /edit	99	99	47	100	69	81
39	/competition-time-ranges/new	100	96	100	100	95	99
40	/competitions	100	75	75	100	66	83
41	/competitions/[id]	100	98	48	100	70	81
42	/competitions/[id]/edit	100	100	57	100	82	85
43	/competitions/[id]/instances	100	92	97	100	72	94
44	/competitions/[id]/instances/ [instanceId]	100	99	69	100	86	89

No.	URL	FCP	LCP	TBT	CLS	SI	Skor
45	/competitions/[id]/instances/[instanceId]/edit	100	68	100	100	66	89
46	/competitions/[id]/instances/new	100	95	100	100	78	97
47	/competitions/new	100	100	100	100	93	99
48	/core-team-divisions	100	76	82	100	73	86
49	/core-team-divisions/[id]	100	98	60	100	75	85
50	/core-team-divisions/[id]/edit	100	99	58	100	82	85
51	/core-team-divisions/new	100	99	100	100	95	99
52	/core-teams	100	98	72	100	79	89
53	/core-teams/[id]	100	99	57	100	76	84
54	/core-teams/[id]/edit	100	97	58	100	77	84
55	/core-teams/[id]/members	100	100	55	100	68	83
56	/core-teams/[id]/members/[memberId]	100	89	57	99	67	81
57	/core-teams/[id]/members/[memberId]/edit	100	72	100	98	83	91
58	/core-teams/[id]/members/new	100	98	100	100	68	96
59	/core-teams/new	100	100	100	100	81	98
60	/degrees	100	99	59	100	68	84
61	/degrees/[id]	100	99	76	100	83	91
62	/degrees/[id]/edit	100	100	64	100	80	87
63	/degrees/new	100	95	100	100	94	98
64	/faculties	100	83	66	100	66	82
65	/faculties/[id]	100	78	63	100	77	81
66	/faculties/[id]/edit	100	99	63	100	81	87
67	/faculties/new	100	93	100	100	94	98
68	/fund-applications	100	98	54	100	63	82
69	/fund-applications/[id]	100	100	62	100	73	86
70	/fund-applications/[id]/edit	100	100	54	100	45	81
71	/fund-applications/new	100	99	100	100	77	97
72	/groups	88	92	90	100	48	89
73	/groups/[id]	100	100	57	100	56	83
74	/groups/[id]/edit	100	90	100	100	93	97
75	/groups/[id]/permissions	100	100	67	100	74	88
76	/groups/[id]/permissions/new	100	100	83	100	76	93
77	/groups/new	95	98	100	100	72	96
78	/login	100	99	78	100	84	92
79	/majors	100	94	100	100	78	96
80	/majors/[id]	100	100	76	100	80	91
81	/majors/[id]/edit	100	100	68	100	71	88
82	/majors/new	100	91	100	100	91	97
83	/permissions	100	100	80	99	52	89
84	/permissions/[id]	100	100	57	100	81	85
85	/permissions/[id]/edit	100	100	71	100	85	90
86	/permissions/new	100	91	100	100	92	97
87	/personas	100	89	62	100	83	84
88	/personas/[id]	100	95	66	100	79	86
89	/personas/[id]/edit	100	78	74	100	74	84
90	/personas/new	100	94	100	100	92	98
91	/project-galleries	100	100	61	100	85	87
92	/project-galleries/[id]	100	96	71	99	80	88
93	/project-galleries/[id]/edit	100	79	72	100	51	81
94	/project-galleries/new	100	98	100	100	85	98
95	/settings	100	99	63	100	82	87
96	/settings/profile	99	97	78	100	67	89
97	/settings/profile/community-groups	100	99	85	100	83	94

No.	URL	FCP	LCP	TBT	CLS	SI	Skor
98	/settings/profile/ community-groups/new	100	99	100	100	90	99
99	/settings/profile/edit	100	100	65	100	70	87
100	/settings/profile/groups	100	99	95	100	84	97
101	/settings/profile/groups/new	100	99	100	100	89	99
102	/settings/profile/permissions	100	98	97	100	83	97
103	/settings/profile/permissions/ new	100	94	99	100	89	97
104	/settings/profile/personas	100	72	89	100	72	87
105	/settings/profile/personas/new	100	89	99	100	82	95
106	/settings/reregistration	100	85	91	100	84	92
107	/settings/system	100	94	100	100	84	97
108	/team-types	100	90	93	100	67	92
109	/team-types/[id]	100	86	100	100	89	95
110	/team-types/[id]/edit	100	96	100	100	90	98
111	/team-types/new	100	94	100	100	93	98
112	/teams	100	97	91	100	87	95
113	/teams/[id]	100	99	88	100	85	95
114	/teams/[id]/edit	100	96	94	100	71	94
115	/teams/[id]/members	100	90	97	100	79	95
116	/teams/[id]/members/new	100	99	96	100	86	97
117	/teams/new	100	94	100	100	82	97
118	/testimonials	100	89	93	100	77	93
119	/testimonials/[id]	100	66	99	99	78	89
120	/testimonials/[id]/edit	100	83	95	100	82	92
121	/testimonials/new	100	84	100	100	93	95
122	/users	100	99	57	99	71	84
123	/users/[id]	100	97	87	100	68	92
124	/users/[id]/community-groups	100	94	97	100	75	95
125	/users/[id]/community-groups/ new	100	99	95	100	76	96
126	/users/[id]/edit	100	99	97	100	66	95
127	/users/[id]/groups	100	99	91	100	82	95
128	/users/[id]/groups/new	100	91	98	100	76	95
129	/users/[id]/permissions	100	90	97	100	67	93
130	/users/[id]/permissions/new	100	90	97	100	82	95
131	/users/[id]/personas	100	99	86	100	79	93
132	/users/[id]/personas/new	100	99	99	100	87	98
133	/users/new	98	99	100	100	57	95
Rata-rata		99,48	93,81	81,08	99,93	76,83	90,42

Lampiran 37. Hasil Pengujian *Performance Efficiency Backend*



Report: 2026-06-21 14:57:29

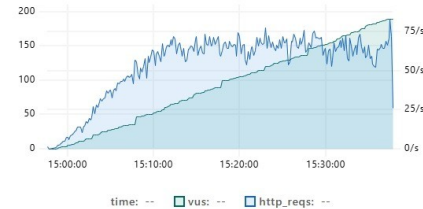
Overview

This chapter provides an overview of the most important metrics of the test run. Graphs plot the value of metrics over time.

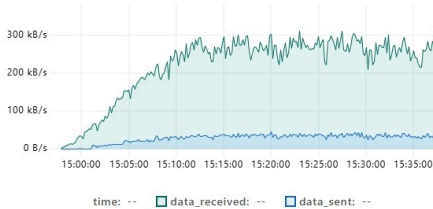
HTTP Performance overview



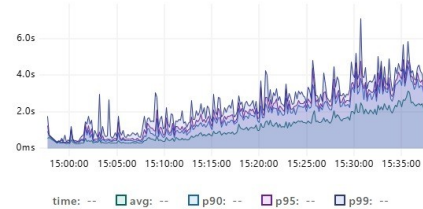
VUs



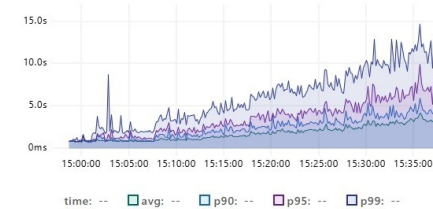
Transfer Rate



HTTP Request Duration



Iteration Duration



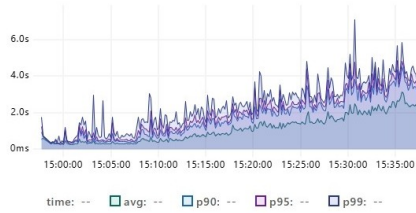
Timings

This chapter provides an overview of test run HTTP timing metrics. Graphs plot the value of metrics over time.

HTTP

These metrics are generated only when the test makes HTTP requests.

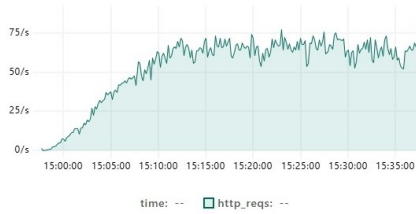
Request Duration



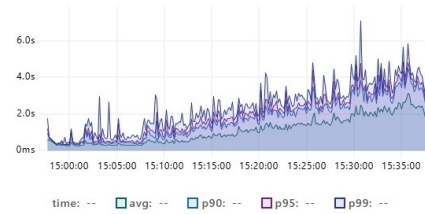
Request Failed Rate



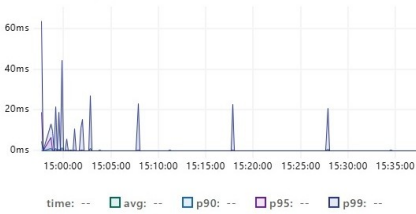
Request Rate



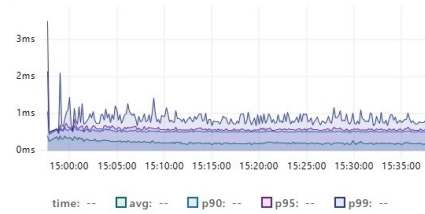
Request Waiting



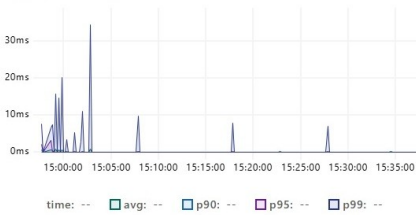
TLS handshaking



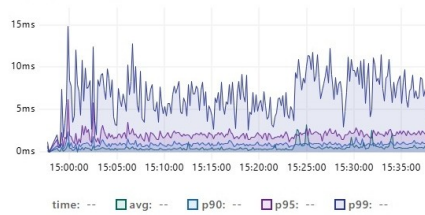
Request Sending



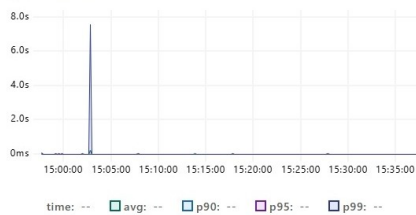
Request Connecting



Request Receiving



Request Blocked



Summary

This chapter provides a summary of the test run metrics. The tables contains the aggregated values of the metrics for the entire test run.

Trends

metric	avg	max	med	min	p90	p95	p99
achievements_response_time	1s	7s	1s	266ms	2s	3s	4s
api_response_time	1s	7s	1s	204ms	2s	2s	3s
community_groups_response_time	1s	7s	1s	214ms	2s	2s	3s
competitions_response_time	1s	5s	988ms	204ms	2s	2s	3s
configs_response_time	1s	6s	1s	215ms	2s	2s	3s
fund_applications_response_time	1s	7s	1s	260ms	2s	3s	4s
group_duration	1s	18s	1s	210ms	2s	3s	7s
http_req_blocked	878µs	7s	0ms	0ms	0ms	0ms	0ms
http_req_connecting	22µs	36ms	0ms	0ms	0ms	0ms	0ms
http_req_duration	1s	7s	1s	204ms	2s	2s	3s
http_req_receiving	538µs	1s	0ms	0ms	971µs	2ms	7ms
http_req_sending	210µs	27ms	0ms	0ms	518µs	582µs	866µs
http_req_tls_handshaking	41µs	164ms	0ms	0ms	0ms	0ms	0ms
http_req_waiting	1s	7s	1s	204ms	2s	2s	3s
iteration_duration	1s	19s	1s	710ms	3s	4s	8s
setup_response_time	570ms	1s	381ms	290ms	926ms	1s	1s
teams_response_time	1s	5s	1s	239ms	2s	2s	3s
teardown_response_time	376ms	541ms	351ms	283ms	482ms	496ms	532ms
testimonials_response_time	1s	7s	1s	220ms	2s	2s	3s
user_permissions_response_time	1s	7s	1s	216ms	2s	2s	3s
users_response_time	1s	7s	1s	220ms	2s	2s	3s

Counters

metric	count	rate
achievements_throughput	7.9k	3.27/s
api_throughput	129.9k	53.6/s
community_groups_throughput	5.8k	2.39/s
competitions_throughput	20.6k	8.48/s
configs_throughput	13.3k	5.5/s
data_received	534 MB	220 kB/s
data_sent	68.8 MB	28.4 kB/s
fund_applications_throughput	10.9k	4.51/s
http_reqs	129.9k	53.6/s
iterations	113.9k	47/s
setup_throughput	18	0.01/s
teams_throughput	5.4k	2.21/s
teardown_throughput	18	0.01/s
testimonials_throughput	5.9k	2.42/s
user_permissions_throughput	26.5k	10.92/s
users_throughput	33.6k	13.87/s

Rates

metric	rate
achievements_error_rate	0.0%
api_error_rate	0.0%
checks	100.0%
community_groups_error_rate	0.0%
competitions_error_rate	0.0%
configs_error_rate	0.0%
fund_applications_error_rate	0.0%
http_req_failed	0.0%
setup_error_rate	0.0%
teams_error_rate	0.0%
teardown_error_rate	0.0%
testimonials_error_rate	0.0%
user_permissions_error_rate	0.0%
users_error_rate	0.0%

Gauges

metric	value
vus	0
vus_max	200

Select a time interval by holding down the mouse on any graph to zoom. To cancel zoom, double click on any graph.